Drishti दृष्टि

Drishti VaR engine
Architecture, data contract and how VaR works

Any set of trades or positions, in any reporting currency

See what your data hasn't told you yet.

ANY DATA. ANY DOMAIN. ONE GRAMMAR.

Ashutosh Sinha

November 2026

Historical simulation over 509 scenario days, window and confidence configurable

IN THIS DECK

1–4 Why, how it works, data in, computed
5 Scale, capacity, concurrency
6–7 Storage, operations, model risk
8 Editions: Risk, Market, World
9–10 Regulatory reporting, roadmap
A Appendix: reference

TL;DR

Executive summary

What is it?	Explainable and controlled risk aggregation over the engines you already have: historical simulation over 509 scenario days, VaR 99% and ES 97.5%, on demand for any set of trades or positions: the bank, a legal entity, desk, book, portfolio, rule or basket.
Every input	**One engine, every input: scenario PVs or path P&L, by trade or by position, any currency, files or Kafka. Source systems keep their own formats.**
How does it work?	It never reprices. The pricing systems send scenario PVs (in each trade's or position's own currency) or path P&L, at trade or position level, through files or Kafka. Drishti turns each into one internal form, a P&L vector per trade or position, converting PVs with scenario FX into any reporting currency. A scope's vector is the sum of its trades' or positions' vectors, and VaR, ES and every contribution are read from that one vector.
Can I trust it?	Every answer names its inputs' versions, scenario set and method; components add up exactly; parametric VaR sits beside it as a cross-check; official records are kept 7 years; anything not built says so.
Does it scale?	Measured: 1 million rows summed in 58 ms[1] (heap tier); 100,000 × 5 dates on the tiered store, a node in 0.48 ms[2]; 700 limit letters build in the same time[3]. Totals over every position of any scope; sized for a large bank (e.g. 2.3M positions, 15,000 portfolios)[4]; VaR sharded across a cluster is built.

[1] Measured: VarScaleBenchmark, 1M trades (VAR_GUIDE §27.2) [2] VarTierBenchmark, 100,000 × 5 dates (§27.3) [3] 10,000 trades (§15.14) [4] derivation A-1

Why Drishti VaR

01

WHY IT MATTERS

VaR at a large bank: scale, sources, currencies, intraday

≈ 2.3M[1]
positions valued every business date

≈ 15,000[1]
portfolios, each a VaR on demand

5[2]
reporting currencies built ahead: USD, EUR, CAD, JPY, GBP

≈ 700[1]
limit letters: a hierarchy configured by columns

Illustrative volumes of a large bank (an example, not a client's figures); every other figure in this deck is Drishti's.

Scale

- A node's VaR from its float64 aggregate in under a millisecond, whatever its size
- Position grain keeps rows = positions; portfolios are sums of positions

Pricing systems keep their formats

- Scenario PVs or P&L, by trade or by position, files or Kafka
- Each system its own form; systems mix on one date and may switch day to day

Multi-currency reporting

- PVs in each trade's or position's own currency, converted with each scenario's FX
- A EUR or CAD VaR is not the USD VaR × today's rate: it carries the currency risk

Intraday

- Only the changed trades or positions, applied as new − old up every tree
- The official end-of-day record stays the full run's; intraday never restates it

Status words

Built things carry no label; only what is not built yet does.

PLANNED	DESIGNED	NOT BUILT	NOT RUN	PROPOSED	MEASURED	ESTIMATED	ASSUMPTION
not built yet	design only	open, not built	not yet run	this deck's proposal	on a workstation	scaled from measured	bank to confirm

[1] Illustrative volumes of a large bank, the product owner's sizing case: ASSUMPTION, for the bank to confirm [2] drishti.var.currencies: the five built ahead by default

WHERE DRISHTI SITS

The real-time risk and analytics layer next to the bank's trading systems

What it is

- the trader's risk workstation: VaR, ES, exposure, sensitivities, on any book, desk or ad-hoc set of trades, live
- pre-trade checks: the pre-deal check against credit limits and tenor buckets, audited
- a live, editable blotter: planned
- every figure explained: where it came from, why it moved

What it is not

- not an order or execution system: it never routes, books or amends a trade at a venue
- not the system of record: the trading and pricing systems stay the source; Drishti reads their files, streams and databases
- not a second risk silo: one engine, one grammar, the same answer on every screen

Trading and pricing systems
own the trades and the prices

›

Feeds
files, Kafka, databases: bound by configuration

›

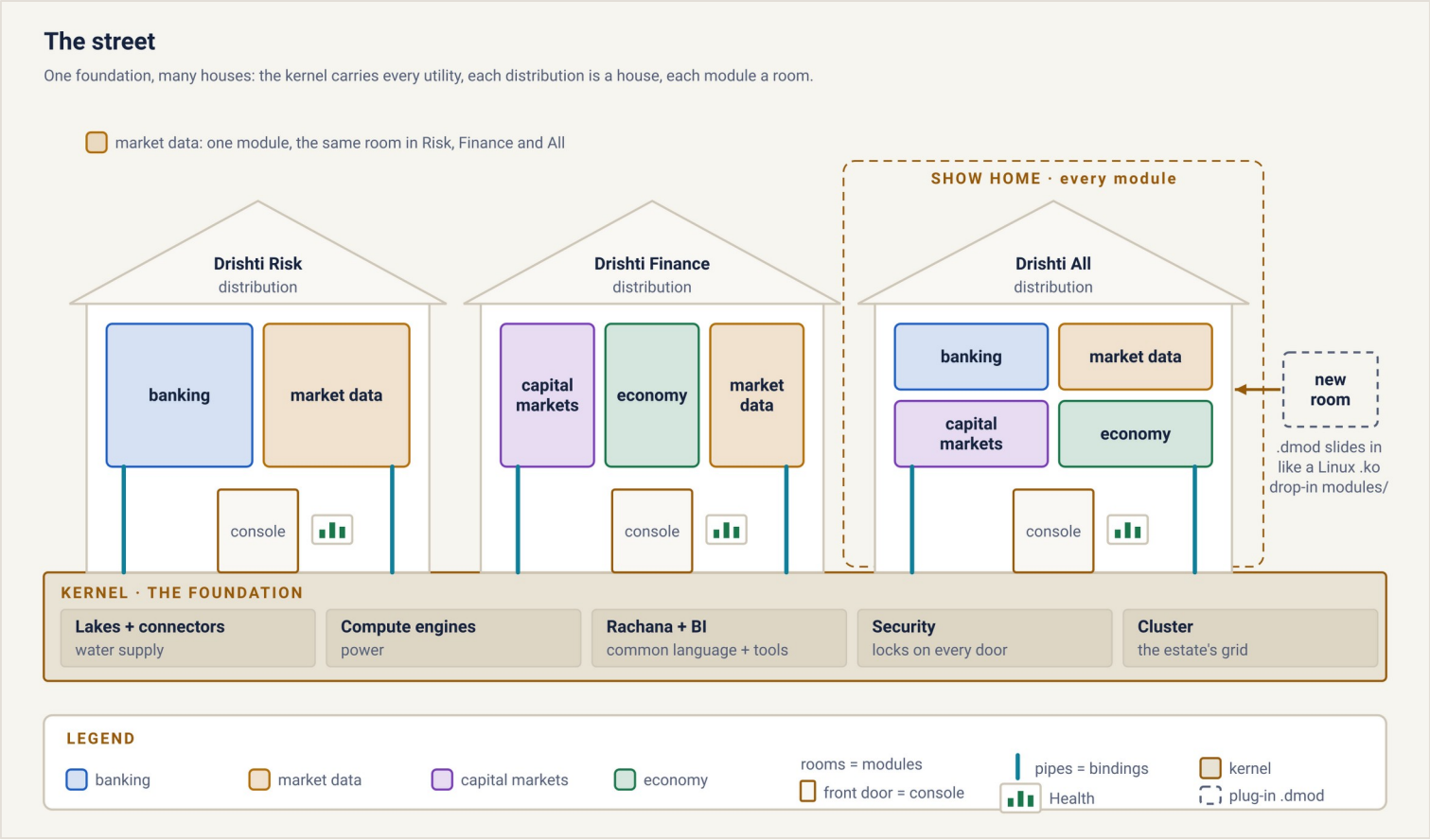
Drishti kernel
lakes, engines, one grammar

›

Workstation
risk, pre-trade, explain, live

Next to the trading systems, not inside them: read-only on their data, so Drishti can be added, upgraded or removed without touching an order path.

One foundation, many houses: kernel, modules, distributions



The kernel

lakes and connectors, compute engines, Rachana and BI, security, the cluster. It knows no industry.

Modules

banking, market data, capital markets, economy: .dmod packages with packs, compute, command words, lenses and pages.

Distributions

Drishti Risk for traders and risk, Drishti Finance for research and economics, Drishti All with every room: one kernel build, one console build.

One rule

every function has one room, every room one door, every setting one switch.

Like Linux: the kernel, drivers (.dmod modules built apart and checked against the kernel), and distributions with their own name, theme and set of modules.

How VaR works

02

How VaR works in Drishti: add vectors, then read the tail

1 A P&L vector per trade or position
509 numbers: its P&L if the market moved as on each scenario day

→

2 Add the scope's vectors
a set of trades or positions; overlaps once; longs and shorts net

→

3 Sort the 509 P&Ls
worst first; equal P&Ls: the older scenario first

→

4 Read the tail
VaR: minus the 6th worst; ES: minus the mean of the 13 worst

Worked example

ten scenarios, α = 80% so k = 2; one book holding P1 (T1 long, T2 short) and P2 (T3)

Vector	0	1	2	3	4	5	6	7	8	9
T1	5	−8	3	−12	6	−2	4	−5	7	1
T2	−2	3	−1	4	−3	1	−2	2	−3	0
P2 = T3	−4	2	−6	1	−9	3	−1	0	2	−7
P1 = T1 + T2	3	−5	2	−8	3	−1	2	−3	4	1
Book = P1 + P2	−1	−3	−4	**−7**	**−6**	2	1	−3	6	**−6**

VaR
6
2nd worst of the book: −6

ES
6.5
mean of −7 and −6

Component VaR
1.26 + 4.74
P1 + P2 = 6: Euler, adds up

Diversification
6
standalone 5 + 7 − 6

Incremental, P1
−1
a hedge: removing it raises VaR

The measures at N = 509

Measure	Definition
VaR 99%	minus the 6th worst P&L
ES 97.5%	minus the mean of the 13 worst
Component VaR	Euler on ranks 4–8; adds up to VaR
Component ES	mean over the 13 worst; adds up to ES
Incremental	VaR(scope) − VaR(scope − part)
Marginal	component ÷ |size| × 1,000,000
Diversification	Σ standalone VaR − VaR
Limits	per node, portfolio; amber 75%, red 90%
Horizon, confidence	1 or 10 days (√10, or sum of 10 days); 95–99.5%

Kupiec test (Paul Kupiec, 1995): is the number of VaR breaches (losses bigger than VaR) consistent with 99%, about 2–3 in 250 days?
Christoffersen test (Peter Christoffersen, 1998): do breaches cluster together, a sign the model reacts too slowly when markets turn?
Basel traffic light: breaches in 250 days at 99%: 0–4 green, 5–9 amber, 10+ red.

CONFIGURABLE

The window, the confidence, the currencies and the names are settings

Scenario window

`drishti.var.scenario-window: N | all`

The newest N scenarios of each date's set: 250 as shipped (one year); all keeps the whole set (509 in the demo). Per request, window= shows another view beside it: labelled on every answer, never official.

Window	VaR 99%	ES 97.5%
250 (shipped)	3rd worst	mean of 7
all (509)	6th worst	mean of 13

Confidence

`DRISHTI_VAR_CONFIDENCE · DRISHTI_VAR_ES_CONFIDENCE`

VaR 99% by default, 95% by configuration; ES 97.5% by default; both also per request. Every label follows the answer (VaR 95% · 1 day). Backtesting stays at 99%.

VaR at	509 days	250 days
99% (default)	6th worst	3rd worst
95%	26th worst	13th worst

Choosing the window banks typically use 1 year (≈ 250 days, the regulatory minimum) or 2 years (≈ 500). A shorter window reacts faster to new volatility; a longer one is steadier and keeps more stress days. Compare both with window= before changing the setting.

Currencies, built ahead

`drishti.var.currencies`
USD, EUR, CAD, JPY, GBP, built in the background after the USD book

`currency-retention-dates`
5: the newest dates kept; older ones built again on first ask

Names

`DRISHTI_VAR_ROOT_NAME`
the top of every VaR tree (root-names: per hierarchy)

`config/demo/names.yaml`
the demo's bank and legal-entity names

Inputs and intraday

`drishti.var.inputs`
optional pin per source system: form and grain

`intraday.kafka.window`
5m; early flush at max-trades (500)

10-day VaR two ways, and any offered confidence, from the same vectors



VAR BANK 10D SUM: the Horizon, Confidence and ES pickers beside the reporting currency; the tiles and the method strip follow the lens.

Method	How	Scenarios
√10, the default	every 1-day P&L × √10: VaR and ES × 3.1623	the same 509
10-day sum	each window the sum of 10 consecutive days	509 − 9 = 500

Demo bank, 99%	VaR	Rank
1-day	112.0m	6th of 509
10-day √10	354.3m	× √10
10-day sum	274.5m	5th of 500

The confidence chosen

`k = ⌈N × (1 − α)⌉`: 99% of 509 is the 6th worst, 95% the 26th, 99.5% the 3rd: another rank of the same sorted vector, nothing stored

- VaR from 95%, 97.5%, 99%, 99.5%; ES from 95%, 97.5%, 99% (configurable)
- Limit use only like for like: a 1-day 99% limit is not compared with a 10-day figure
- Backtesting stays 1-day; the official records stay 1-day at the configured confidence
- Bit-equal whatever the shards and threads that summed the 1-day vector

Figures: the demo bank on 2026-09-30, USD, the committed synthetic test data (VAR_GUIDE 18.6, VarTestDataPrecisionTest)

Stressed VaR: today's positions under a stress period's moves

The same historical simulation, on a second scenario set

Today's positions
today's trades, sizes and sensitivities

→

A stress period's moves
a fixed 12 months, e.g. 2008-07 to 2009-06: about 250 days

→

Stressed P&L vectors
the pricing system revalues today's trades under each stress day

- Not the positions as they were then: a bond bought last month has a stressed P&L for 2008-10-10
- A second set of the business date, through the same feed and contracts: SET_TYPE STRESS (or the HS-STRESS- prefix)
- Its own book and hot file, loaded on the first stressed question; never written to the lake
- VaR at any offered confidence, ES, components, contributions; 1-day and 10-day (√10 or the 241 windows)

On every official record: SVAR, SES and the stress set's id, beside VaR and ES

The 12-week average of a node's official stressed VaR, for the capital rule's max(latest, multiplier × average); official dates without one are named, never counted as zero

The demo's stress set is synthetic. 2008-like moves made by the test-data generator, not market history: the set says so and so does every stressed answer (the method strip reads **synthetic**).

Normal or Stressed, beside the other pickers



Bank, 99%	Normal	Stressed
1-day	112.0m	213.6m
10-day √10	354.3m	675.6m
10-day sum	274.5m	293.3m

- Reported in USD, the stress set's own currency
- No limit use: VaR limits are set against normal VaR
- Not backtested; intraday batches update the normal set
- Commands: VAR BANK SVAR 10D SUM; the API's set=stressed; the CLI's --stressed

Figures: the demo bank on 2026-09-30, USD; stressed on the synthetic stress set HS-STRESS-2008, 250 days (VAR_GUIDE 18.8, VarComputeFirstTest)

From scenario PVs in their own currency to VaR in any currency

Inputs

- Pricing systems send each trade's or position's 509 scenario PVs and base PV in its own currency (`scenario-pv`). The USD P&L-vector form stays supported; a date may mix both, each row in one form.
- Scenario FX: the batch's own (`fx-scenario`) when sent, else derived from the FX history. Every cross goes via USD. Horizon: 1 day.
- Converted when the date loads; positions, nodes, captures and official records do not know which form a row came in.

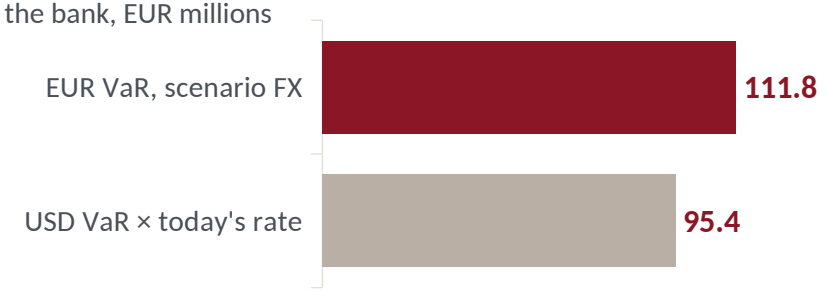
```
USD P&L      PV(s)·FX_c(s) − PV(base)·FX_c(base)
FX derived   FX(s) = FX(base) · H(d_s) / H(d_s − 1)
In r         (p + P&L_USD(s)) / FX_r(s) − p / FX_r(base)
Node in X    Σ_c [PVsum_c(s)·FX_c→X(s) − PVbase_c·FX_c→X(base)]
```

Reporting currencies

- Built ahead in the background: USD · EUR · CAD · JPY · GBP (`drishti.var.currencies`); the newest 5 dates kept; any other currency with FX on first ask.
- Every node keeps its PVs summed per trade currency: a headline in any currency is exact to the cent.
- Trade pages say the Trade currency (EUR · as priced); official records in USD, others per setting.

Why EUR VaR ≠ USD VaR × today's rate[1,2]

the bank, EUR millions



The bank on 2026-09-30 holds a net 3.65 bn USD of value; on the scenario days it moves against the euro, a risk a euro-based owner carries and a dollar-based one does not.

Ccy	Bank VaR 99%[1]	Bank ES 97.5%[1]
USD	112,041,646	160,143,809
EUR	111,837,045	137,631,546
CAD	189,801,404	230,680,413
JPY	20,883,226,318	23,578,418,999
GBP	104,139,554	125,595,958

On the demo: the 1,339 Endur trades come as scenario PVs (1,089 USD, 246 EUR, 4 CAD). Bank VaR 112,035,906 → 112,041,646 and ES 160,132,107 → 160,143,809: the FX move of the 250 EUR and CAD trades. A USD trade reproduces its vector exactly.

[1] Computed by the engine on the committed synthetic demo data, 2026-09-30 (VAR_GUIDE §15.9–15.11) [2] 95.4 = the USD VaR at today's EUR rate, as the guide quotes it

Intraday: only what changed, applied as new – old up the tree

Morning full run
RUN_KIND FULL: every live trade or position; the date loads; official records taken from it

→

Changed rows
Trades or positions amended, new, cancelled; same scenario set; own BATCH_ID, later COMPUTED_AT

JSONL files

→

Lake: appended batch
Files: drishti.py var intraday --load (lake build --mode append). Kafka: one batch per window

Kafka micro-batches

→

Apply: new – old
Into the book, every node above (every hierarchy), the currency totals and built currencies

→

Answers say intraday
Badge in the status line, intraday on every answer; morning hot file never changed

Case	What the engine does
Same trade or position, several batches	latest COMPUTED_AT wins across files and Kafka; an equal one keeps the first
COMPUTED_AT before the morning run	ignored and counted
Cancel (CANCELLED, vector null)	removed from every scope
New trade	book from TT_BOOK, position by the position rules
Moved to another book or position	old ordinal cancelled, added in its new place
Another scenario set	row refused (DRS-5203); the date stays as it is
A full run of the date	date reloads; the intraday layer is dropped

Cost of a batch ∝ rows changed × nodes above × currencies built, never the size of the book.

Kafka, as built

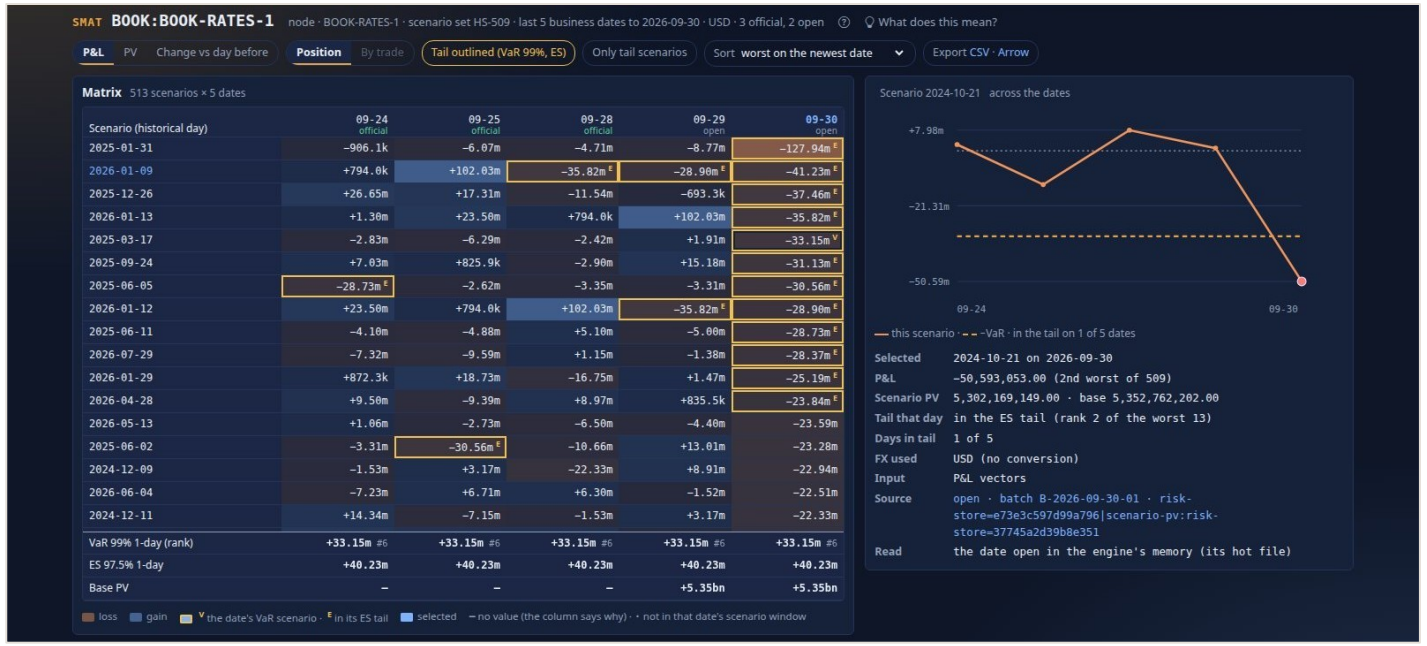
- A message is JSONL text: one or more records, PVs or P&L, a cancel as a marked line
- A window every 5 min (drishti.var.intraday.kafka.window); early flush at 500 trades; admin “Apply now”
- Each window written to the lake as ONE intraday batch (on Iceberg, one append), then applied like a file
- Replaying the topic after a restart writes nothing twice

The guarantee incremental = the same data loaded as one run, to the cent, on every node

Measured[1]: 200 batches of 3 trades on one date; a restart equal too; batch time does not grow. Official numbers come only from full runs.

[1] VarIntradayApiTest: 200 batches of 3 trades, equal to the cent to the same data loaded as one run, on every node (VAR_GUIDE §8.4)

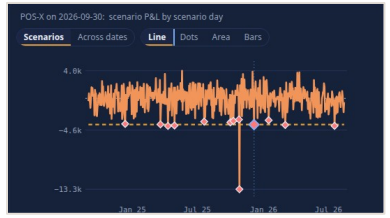
SMAT: one trade, position or node, scenario by scenario, across dates



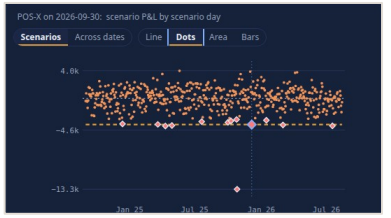
SMAT BOOK:BOOK-RATES-1 5: scenario days down, dates across; V the date's VaR scenario, E its ES tail

- Answers: is this loss scenario new, or in the tail all month? Did VaR move because the book changed or the window rolled?
- P&L, PV or the change against the day before; only the tail scenarios; By trade for a position; CSV or Arrow export
- Each date's numbers name their source: open in memory, read from the lake by id, the official record, or the engine

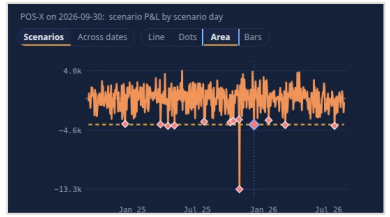
The chart: Scenarios or Across dates



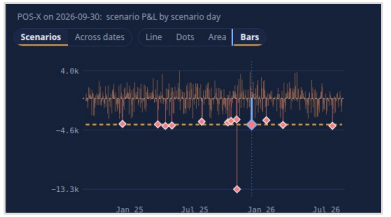
Line



Dots



Area



Bars

- One click switches the type, for both views; remembered in the browser
- The reference line: −VaR for P&L, PV at VaR for PV; the ES tail as diamonds
- The legend names exactly what is drawn

One explorer for any scope, and the same answers everywhere else



VAR DESK DESK-RATES IN EUR: the scope tree and tiles in EUR, the basket tray, the P&L path, and How this number was built naming the reporting currency, the input form, the scenario FX and the crosses via USD (2026-09-30).

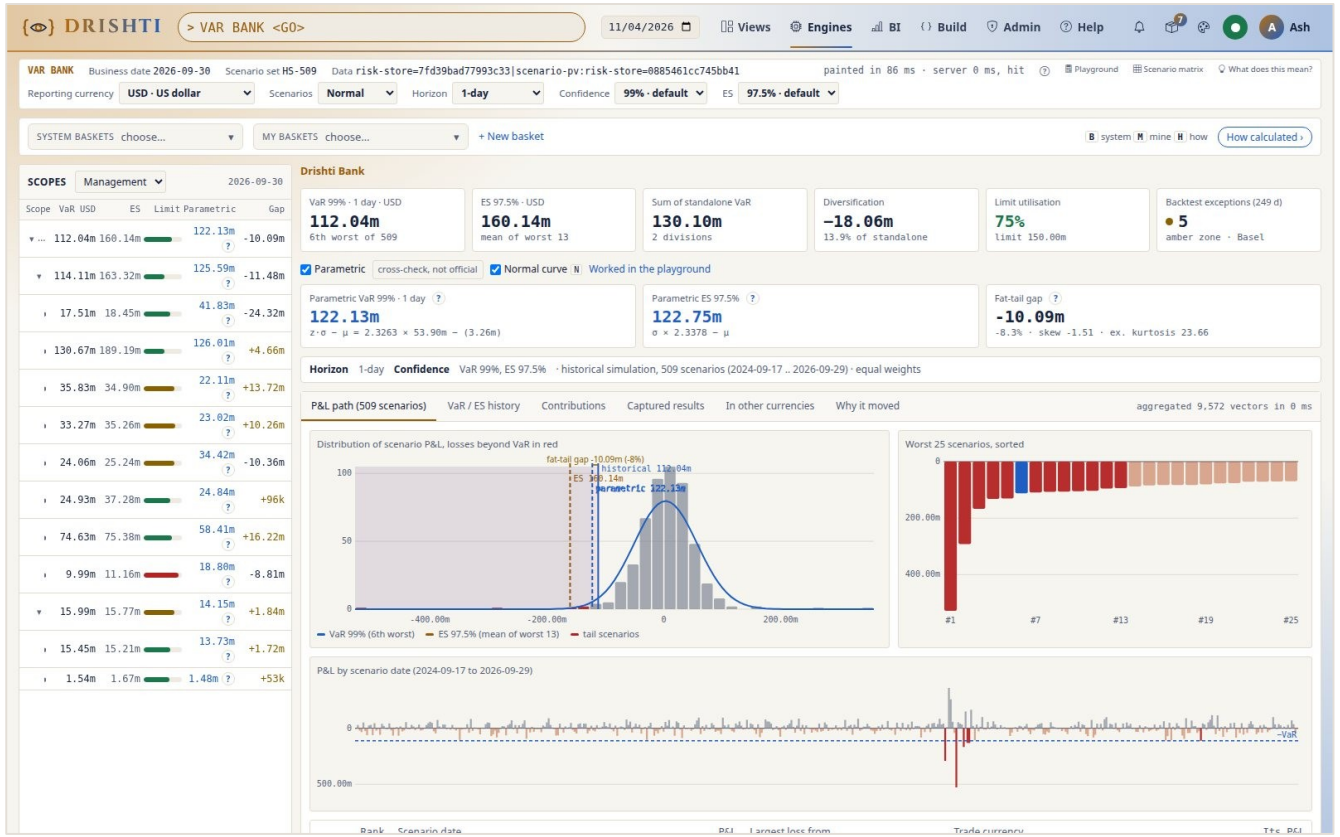
9–14 ms[1] click to painted, a desk, answers fetched ahead (target 300 ms)

390 px on a phone the three columns stack, no sideways scroll

- **Scope tree and six tiles** MGMT, LEGAL or the limit letters (a tree by columns) with VaR, ES and limit bars; VaR, ES, sum of standalone, diversification, limit use, backtest exceptions.
- **Five tabs** P&L path (509 days, worst 25, who lost on a day) · VaR/ES history and backtest · contributions · captured results · In other currencies.
- **Contributions as Arrow** the by-trade and by-position tables travel as Apache Arrow, 61–65% fewer bytes[2] than JSON.
- **Reporting currency picker** USD, EUR, CAD, JPY, GBP and More…; every panel follows; kept in the link; VAR … IN EUR.
- **How this number was built** the scope as resolved, its trades and positions, hidden and unknown ids, batch, data versions, method, tier, timing.
- **Baskets and pre-deal what-if** any mix of portfolios, positions, trades, counterparties and rules, live; a booked trade's vector × a factor through the Python engine.
- **Captures, official records** immutable captures, compare with now; each node's official history.
- **Everywhere else** trade (Trade currency), position, portfolio, desk and LE pages; drishti.py var … --ccy; /api/v1/var; VRES and VWIF; the BI VaR report; Home facts.
- **Governed** root name drishti.var.root-name; per-node positions or aggregate access; masks; every answer audited.

[1] Measured: 9–14 ms fetched ahead, else 30–35 ms (VAR_GUIDE §27.2) [2] Measured: 631,873 → 244,304 B by trade; 692,324 → 243,392 B by position (DATA_FORMATS §6)

Parametric VaR beside historical: a cross-check, never the official figure



VAR BANK with Parametric on: the tiles, the tree's Parametric and Gap columns, the normal curve over the P&L histogram (demo data, 2026-09-30)

Reading it. Historical VaR 112.04m against parametric 122.13m at 99%, one day: a gap of −10.09m. The gap is the signal: a scope whose tail is fatter or thinner than the normal says so at once.

A toggle, not a second screen

Parametric on the VaR explorer adds parametric VaR and ES, the fat-tail gap, the tree's columns and the curve (key N). PVAR opens the explorer with it on.

The method, shown

A normal fitted to the scope's own scenario P&L: VaR = z·σ − μ; ES = σ·φ(z)/(1 − c) − μ; equal weights or EWMA (λ 0.94). Every figure has a ? with the server's own working.

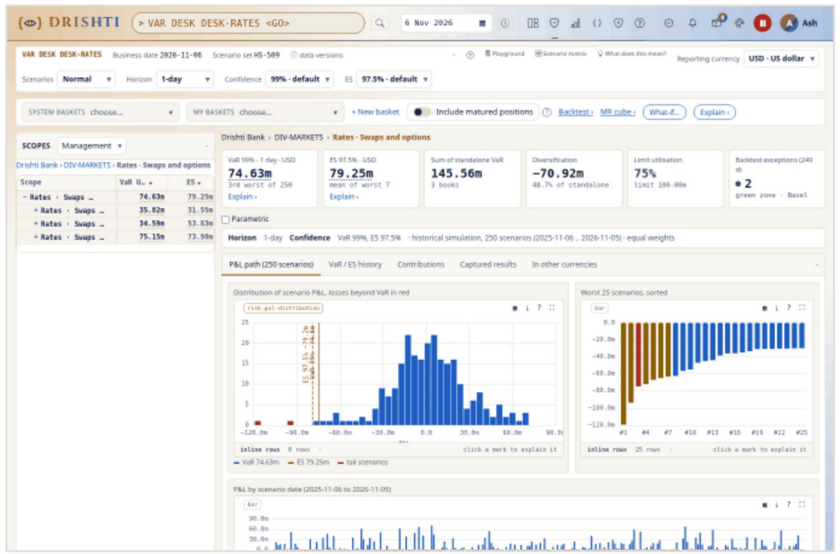
By hand, and in Excel

The VaR Playground's Parametric tab: each step a formula, each position's component adding up exactly; the workbook recalculates to the engine's figure.

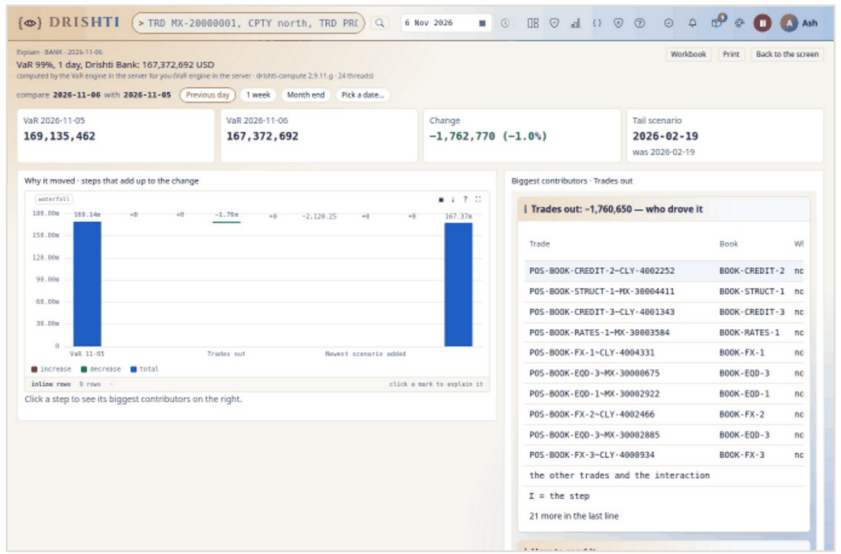
Never official

Limits, backtests and official records stay historical simulation. pvar() also works in the pivot and the MR cube.

Scoped tree, Explain on every figure, and Why it moved against any date



The VaR page of a desk: the tree shows the chosen scope only



Why it moved: steps that add up to the change

- Scoped tree: the bank, a node and below, or a basket's own positions by any hierarchy; each part's VaR from the engine
- Explain on every row and tile, on every page that shows VaR or ES
- Why it moved: compare with the previous day, a week, month end or a picked date
- four figures, the waterfall first and wide; click a step for its biggest contributors

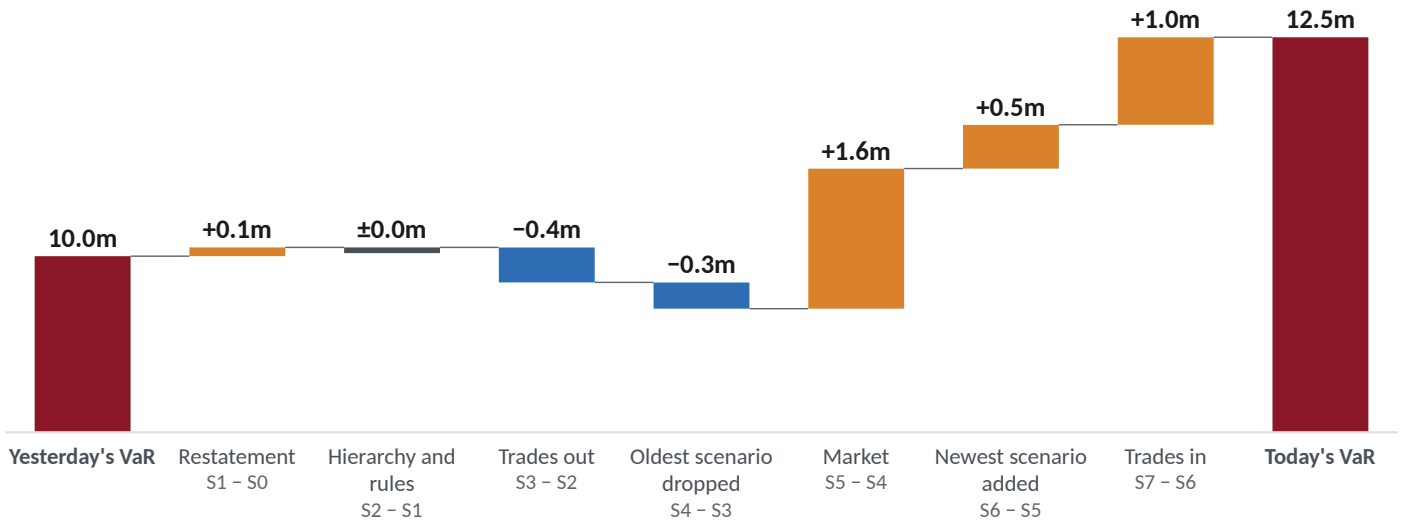
Why the steps add up exactly

Drishti moves one thing at a time from yesterday's state to today's (restatements, hierarchy, trades out, the oldest scenario dropped, the market, the newest scenario added, trades in) and recomputes VaR at each stage; each step is the difference between two stages. The pictures are the demo data.

Why VaR moved: one change at a time

The method works by moving one thing at a time from yesterday's state to today's and recomputing VaR at each stage, which is why the steps always add up exactly.

From yesterday's VaR to today's, a step for each cause (illustrative figures)[1]



A rise in VaR orange, a fall blue; the value axis starts at 8.0m. The steps add up to +2.5m exactly.

The states: each one a full VaR

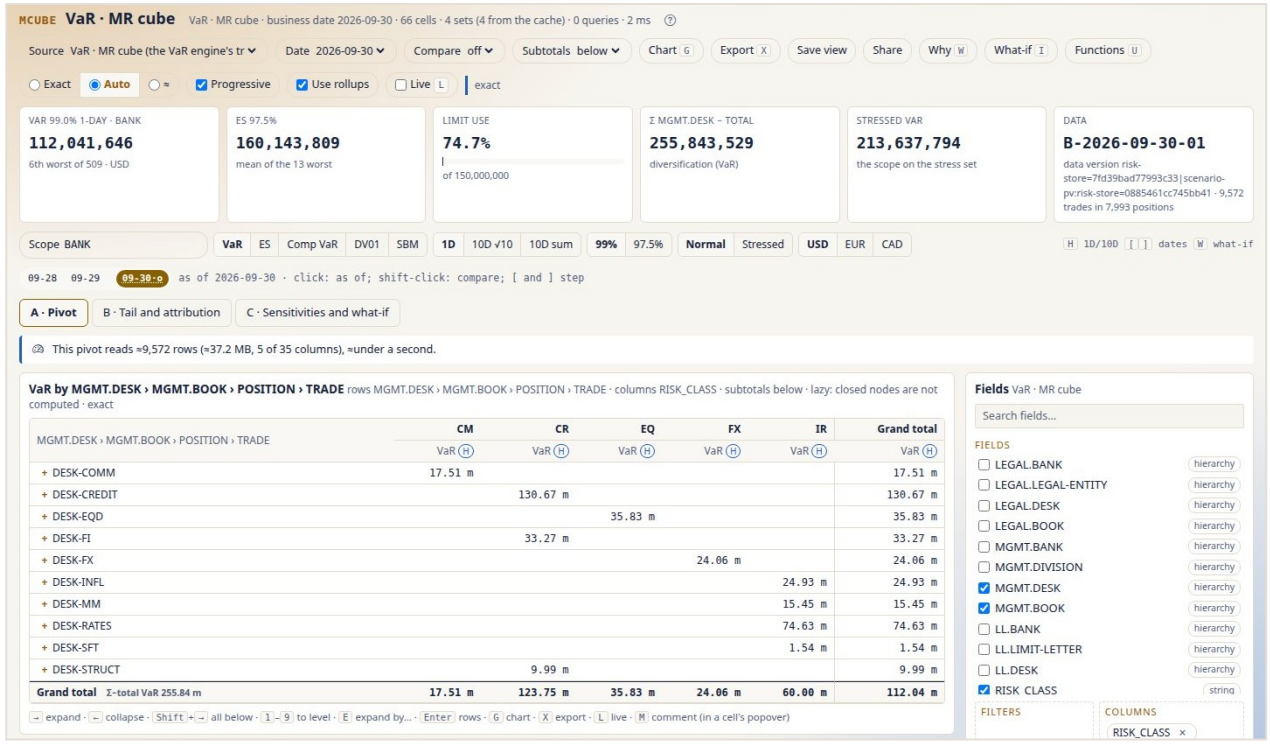
State	What changes from the one before
S0	yesterday, as officially captured
S1	yesterday's data as it is now (restated)
S2	today's hierarchy and rules
S3	only the trades still live today
S4	only the scenario days common to both windows
S5	today's market: today's vectors
S6	today's scenario set: the newest days added
S7	today's trades: today's VaR

A step is one state's VaR less the one before: S1 − S0 … S7 − S6. Their sum is S7 − S0: today's VaR less yesterday's.

One answer, three views: Waterfall · Matrix (every node's bridge) · Sentences (the answer in words). **Download the working (.xlsx)** to check every state by hand. Each step names its drivers: trades by incremental VaR, desks by component VaR, the scenario days that left or entered. **`VAR EXPLAIN <scope> <GO>`**

[1] Illustrative figures, invented to show the shape, not from the demo data (VAR_GUIDE.md section 3.11)

The MR cube, FRTB-SA and SIMM, computed when asked



MCUBE: VaR by desk × risk class; each cell the VaR engine's figure for its own scope (demo data, 2026-09-30)

In every cell: click for its provenance (scope, members, data version, method); Why splits a move into steps that add up; What-if sets an unwind beside the original.

The MR cube (MCUBE)

VaR, ES and sensitivities pivoted by any dimension on the pivot engine. A cell is never a sum: it is the VaR engine's answer for that cell's trades; the gap shows as diversification.

VaR for any scope

Any node, portfolio, basket, ad-hoc expression or trade column (by trader, say): one engine, the same figure on every screen.

FRTB-SA (FRSA), computed

From position sensitivities: delta, vega and curvature with the three correlation scenarios, the DRC and the RRAO; Basel MAR21–23 parameters versioned and hashed into every answer; PLAY FRTB.

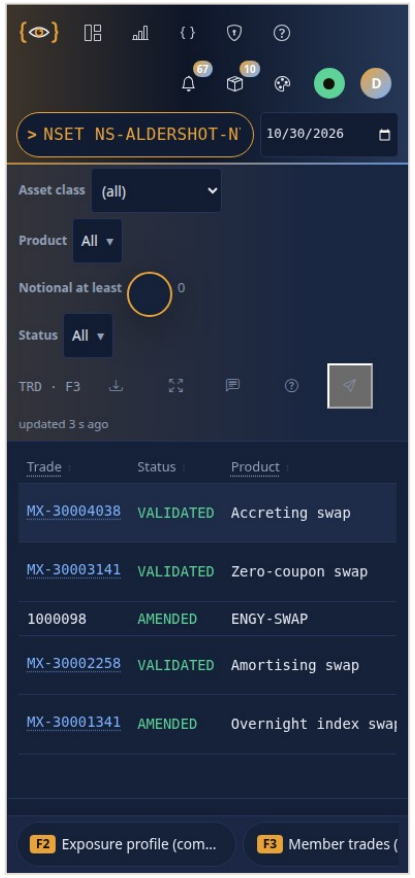
SIMM, per netting set

Collect and post, against the counterparty's call; PLAY SIMM. ISDA's calibrated numbers are licensed: the parameter file ships empty and the bank installs its own.

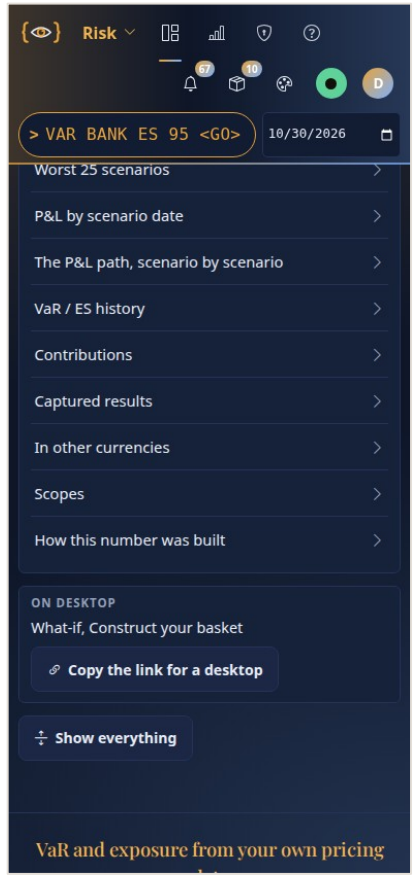
On a phone: must, glance, more and desk

must	**shown in full, first, loaded with the page, live** the header, key figures, breaches
glance	**a compact form: five rows, a chart as a sparkline** a long table as its first rows, with Show all
more	**listed under More on this page, loaded on tap** contributions, the P&L path, provenance
desk	**not on a phone: Copy the link for a desktop** what-if, Construct your basket, designers

- a tier per panel, set in configuration: the panel kind, the Sutra, then the site, pack or edition
- a phone loads only must and glance panels; masks and roles unchanged; desktop untouched; Show everything for the session



glance: a netting set's trades

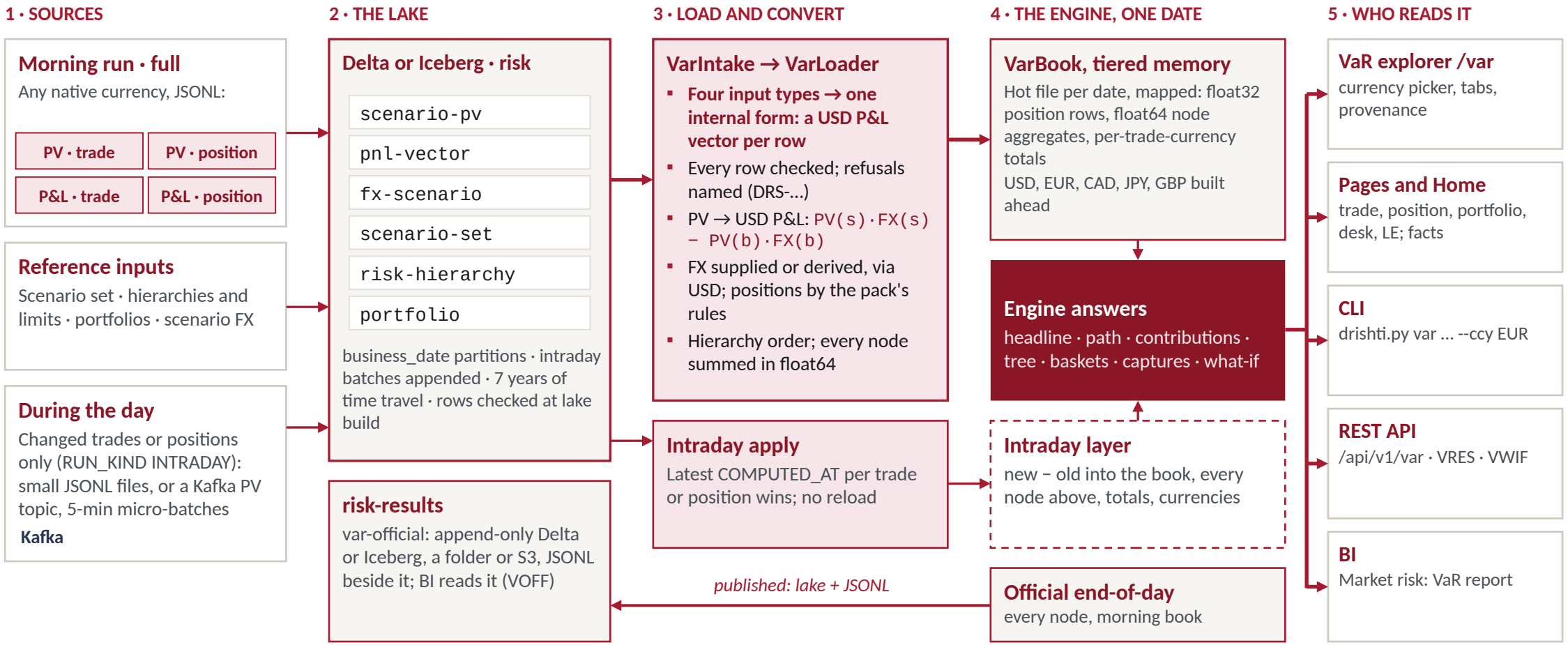


more and desk: the VaR page

Data in

03

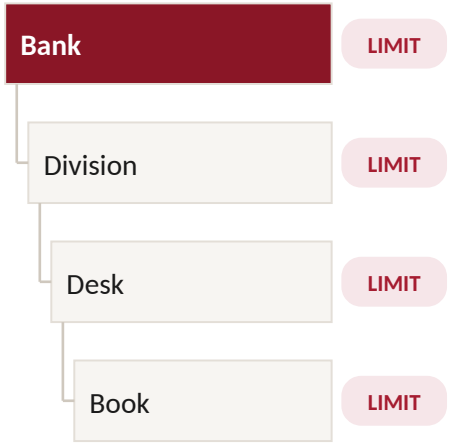
From the pricing systems to every screen, in one direction



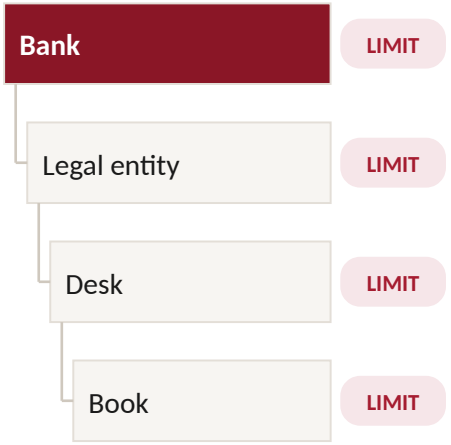
Each batch's data-load signal `POST /api/v1/packs/market-risk/loads` *tells the engine: a full run reloads the date; an intraday batch is applied on top. The lake is the record: a restart rebuilds everything, the intraday layer included.*

Three trees: management, legal entity and the limit letters

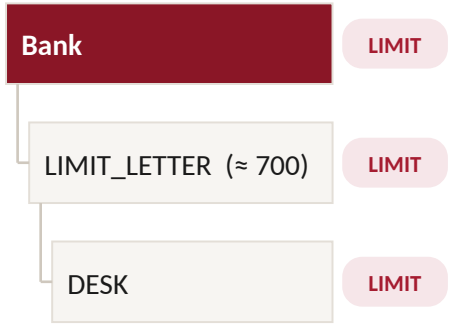
MGMT · management



LEGAL · legal entity



LL · limit letters



Configured by columns

- Levels are columns every row carries (LIMIT_LETTER > DESK); nodes are the distinct paths of the values
- A letter may hold part of a book; a position under two letters is split (<position>@<letter>) and counted
- A source without the column skips that hierarchy alone, with a DRS-5211 warning; the date, MGMT, LEGAL and exposure answer as before
- Official records for every hierarchy; exposure builds the same trees

```
# the market-risk pack's hierarchies: block, as built
hierarchies:
  MGMT:  { title: Management, source: risk-hierarchy }
  LEGAL: { title: Legal entities, source: risk-hierarchy }
  LL:    { title: Limit letters, levels: "LIMIT_LETTER > DESK", word: LL, limits: var-limit }
```

Limits and the picker

- var-limit: HIERARCHY, NODE_ID, LIMIT_VAR (USD); no row: no limit, never zero
- Limit bar amber 75%, red 90%; LIMIT and LIMIT_USE in official records
- The explorer's hierarchy picker lists the date's trees (hierarchy=LL kept in the link)

MEASURED **700 letters build in the same time as none, and the VaR of all 1,412 nodes takes about 30 ms** (10,000 trades; build 65–99 ms against 53–197 ms without). Scope word LL:LL-0003; node ids are paths, LL~LL-0003/DESK-RATES.

One engine, every input: source systems keep their own formats

	Scenario PVs native currency	Path P&L vectors any currency
By trade	✓ in the trade's currency	✓ a currency a row
By position	✓ in the position's currency	✓ a currency a row

Delivery routes

Files, JSONL
the morning run; small intraday batches appended

Kafka, micro-batched
JSONL messages; 5-min window, early flush, Apply now

Intraday, either route
changed trades or positions, as new − old; latest wins

```
{"VALUE_TYPE":"PV",  "GRAIN":"TRADE",    "TRADE_ID":"1000279", "CURRENCY":"EUR", "BASE_PV":…, "PVS":[…509…], "COMPUTED_AT":"…", …}
{"VALUE_TYPE":"PV",  "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"JPY", "BOOK":"…", "BASE_PV":…, "PVS":[…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"TRADE",    "TRADE_ID":"MX-…",    "CURRENCY":"USD", "PNL":[…509…], …}
{"VALUE_TYPE":"PNL", "GRAIN":"POSITION", "POSITION_ID":"POS-…", "CURRENCY":"USD", "PNL":[…], …}
```

Mix and switch

- Each system sends its own form and grain; systems mix on one date.
- A system may switch day to day; each date loads as it stands.
- Old files: `drishti.py convert --set …`

```
# optional cross-check
drishti.var.inputs:
  ENDUR:   {form: pv,  grain: trade}
  CALYPSO: {form: pnl, grain: trade}
  default: {form: any, grain: any}
```

Guarantees

- One form and grain per system per date (DRS-5210): no double counting
- Fields missing or wrong (DRS-5208), or against the pin (DRS-5209): refused, never guessed
- Provenance shows what each system sent

Caveat a P&L vector sent without its PV converts to another reporting currency only approximately (at each scenario's rate, without the FX move of its base value; counted as `tradesWithoutPv`). Scenario PVs give exact multi-currency VaR.

Six input contracts, JSONL on the wire, every check named

JSONL: one JSON object a line, UTF-8, .jsonl or .jsonl.gz; each line exactly its contract's fields. Key fields of the as-built contracts:

`scenario-pv` SPV · a trade × date × set	
`TRADE_ID … TT_BOOK`	trade; LE, desk, book, letter
`CURRENCY · BASE_PV`	PVs' currency; value today
`SCENARIO_PV [509]`	per scenario; null = failed
`RUN_KIND`	FULL or INTRADAY
`SCENARIO_SET_ID`	and BATCH_ID: the run

`pnl-vector` PNLV · a trade × date × set	
`VECTOR_ID · TRADE_ID`	PNL-<trade>-HS-509
`PNL [509]`	USD P&L, gain positive
`PV · REPORTING_CCY`	USD value today; USD
`STATUS`	ok, proxied or failed
`RUN_KIND`	as scenario-pv

`fx-scenario` FXS · a currency × date	
`FX_SCENARIO_ID`	FXS-<CCY>-HS-509
`CURRENCY`	AUD, CAD, EUR, JPY …
`RATE`	today, USD per one unit
`RATES [509]`	per scenario, the set's order
`not sent?`	derived from the FX history

`scenario-set` SSET · one row a date	
`SCENARIO_SET_ID`	HS-509
`SCENARIO_DATES`	509 days, oldest first
`WINDOW_START … END`	2024-09-17 … 2026-09-29
`HORIZON_DAYS`	1
`REVALUATION`	full or sensitivities

`risk-hierarchy` HIER · a node × date	
`NODE_ID`	MGMT~<id> or LEGAL~<id>
`HIERARCHY · LEVEL`	MGMT, LEGAL…; open values
`PARENT_ID · NAME`	the tree; display name
`BOOKS`	every book beneath
`limits`	in var-limit (VLIM), USD

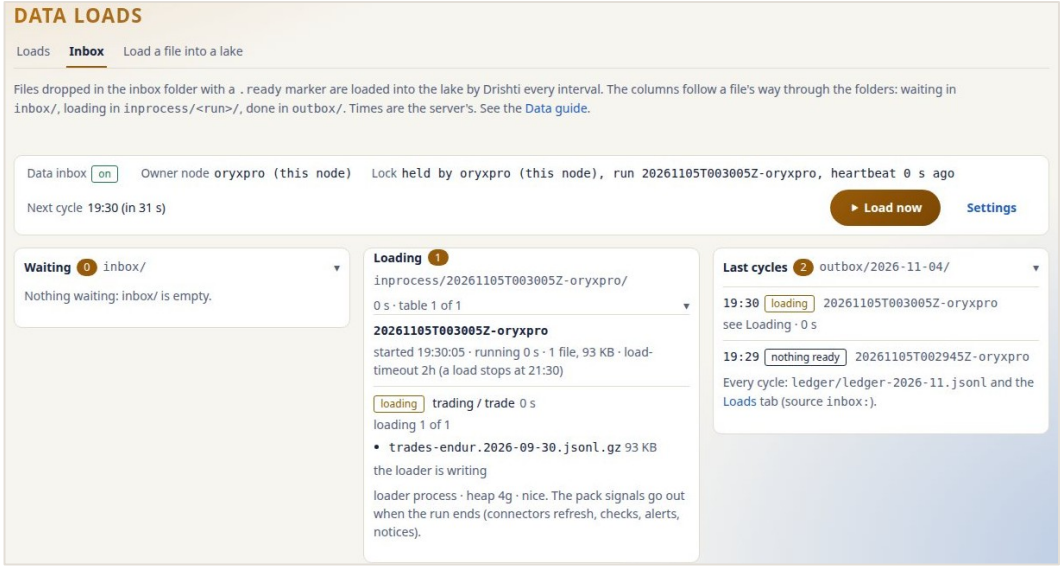
`portfolio` PORT · a portfolio × date	
`PORTFOLIO_ID · NAME`	PF-…; scope PORT:<id>
`TYPE`	HIERARCHY, RULE or BASKET
`DEFINITION`	NODE, RULE or MEMBERS
`OWNER`	a user or a team
`limit`	var-limit, hierarchy PORTFOLIO

Required on every record: `VALUE_TYPE`, `GRAIN` with its id, `CURRENCY`, `SOURCE_SYSTEM`; GRAIN matches the id. Old files: `convert --set`

DRS-5201 length, null, NaN	**DRS-5206** a PV failed	**DRS-5207** no scenario FX	**DRS-5208** record fields wrong	**DRS-5209** against the pin	**DRS-5210** 2nd form or grain	**DRS-5202** trade twice	**DRS-5203** two runs

Data in by clear rules: the file says its table and its date

Rule	As built
Table and date	from the name: <table>.<YYYY-MM-DD>.jsonl.gz, its .ready file, or the command line; never a row
Several at once	a tar of <table>/<date>/ folders: all of it loads, or none
Append by default	mode: replace replaces a date; every row carries its load id
Undo	drishti.py dataload undo <load id> removes one load, in its own lake
One file per lake	config/lakes/<lake>.yaml: partitions, load mode, retention, maintenance; lake check proves it
Everywhere the same	the Python and Java loaders, the inbox, every store loader, the demo and test data



Admin → Data loads → Inbox: a file waiting, loading, and the last cycles

Why it matters. A file loaded twice, a date guessed from a row or a half-loaded set are the usual sources of wrong numbers. Here the name decides, a set loads whole, and any load can be taken back.

The data inbox

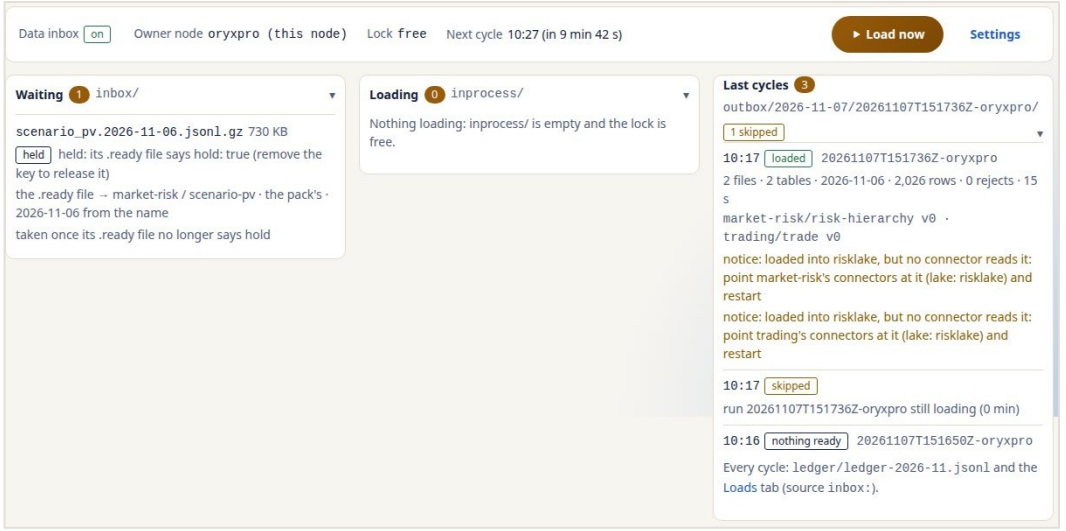
Drop files with a .ready marker: the Java loader loads them every 10 minutes, moves them to the outbox with a report and tells the cluster. Off as shipped.

The in-memory lake

Delta tables held only in memory, for feeds that by contract may never touch a disk: a hard cap, refilled by server jobs, proved by a marker scan. Off as shipped.

Every lake in its own folder: real data beside the demo, never over it

What	As built
A folder per lake	data/delta/<lake>/ by default; a lake file may name any folder, s3:// or mem://
The demo lake	data/delta/demo; an older demo lake moves there once at the first start, measured, rolled back on any difference, never deleting
Shipped empty	risklake, newslake, worldlake, regulators, capitalmarket: lake files with no tables; nothing shipped points at them
lake: in .ready	names the lake a file goes to; the first load creates the lake file
Said, not guessed	a load into a lake no connector reads says so, naming the connectors to point at it
Never mixed	once a kind reads a real lake, the demo samples stop answering for it; Health flags a pack whose connectors read two lakes



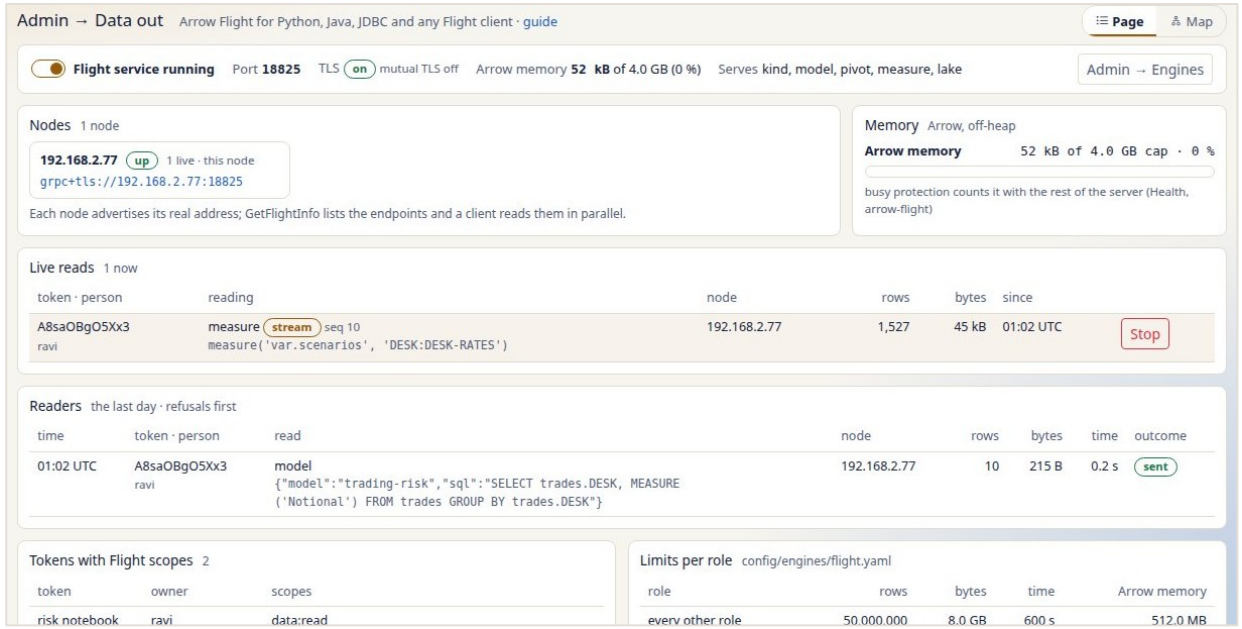
Admin → Data loads → Inbox: a day loaded into risklake, with the notice that no connector reads it yet

Why it matters. A pilot loads the bank's own files next to the demo, and a screen never answers from a mix of the two. One guide covers lakes and loading end to end; tutorial 20 runs it on a real day.

Undo in the load's own lake

Admin → Inbox → Undo a load and drishti.py dataload undo <load id> find the load's lake in the ledger and take back exactly that load.

Drishti's data to any client, as the person it belongs to



Admin → Data out: the service, its nodes, Arrow memory, a live VaR stream and the readers

Client	How it reads
Python	pyarrow: a token to a DataFrame, VaR scenario P&L, a live stream
Java	the Arrow Flight client: kinds, models, pivots, measures
JDBC and ODBC	Flight SQL: read-only SQL over the BI models
Excel	through Python; working examples in examples/clients

What it serves

The kinds of every pack, BI model queries, pivot views, VaR's scenario P&L vectors and the in-memory lake: Arrow record batches, no JSON step, no copy of the lake.

The same controls as the screens

Tokens with Flight scopes, entitlements and masks per reader, limits per role, Arrow memory counted by busy protection, every read audited. Off as shipped.

Every node an endpoint

Each node advertises its real address; a client reads the endpoints in parallel; a live stream resumes from its sequence number.

NOT RUN **Under construction**

Three nodes end to end (endpoints on every node, a node stopped mid-read, a VaR stream through an intraday batch) is not measured yet. The single-node reads, the stream, Stop and resume are run.

Feeds and public data

04

One feed file per source, pulled daily into the lakes

Piece	As built
The feed file	config/feeds/<feed>.yaml: datasets, the pack and table each loads, the lake, the schedule, the licence with its internal-only flag; API keys as environment names only
The pull	incremental and resumable; one polite User-Agent naming the product
What it writes	JSON Lines per dataset and business date under data/datafeeds/<feed>/, a manifest with rows and SHA-256, a run ledger
Three load routes	the Java loader, the Python loader, or the data inbox
Run and watch	drishti.py feed list | run | status | schedule; Admin → Data loads → Feeds; an alert when a run fails

Thirteen feeds ship

Public sector

World Bank WDI, central banks (FRED, Bank of Canada), Statistics Canada, the OECD, official press releases, GDELT world events

Reference and markets

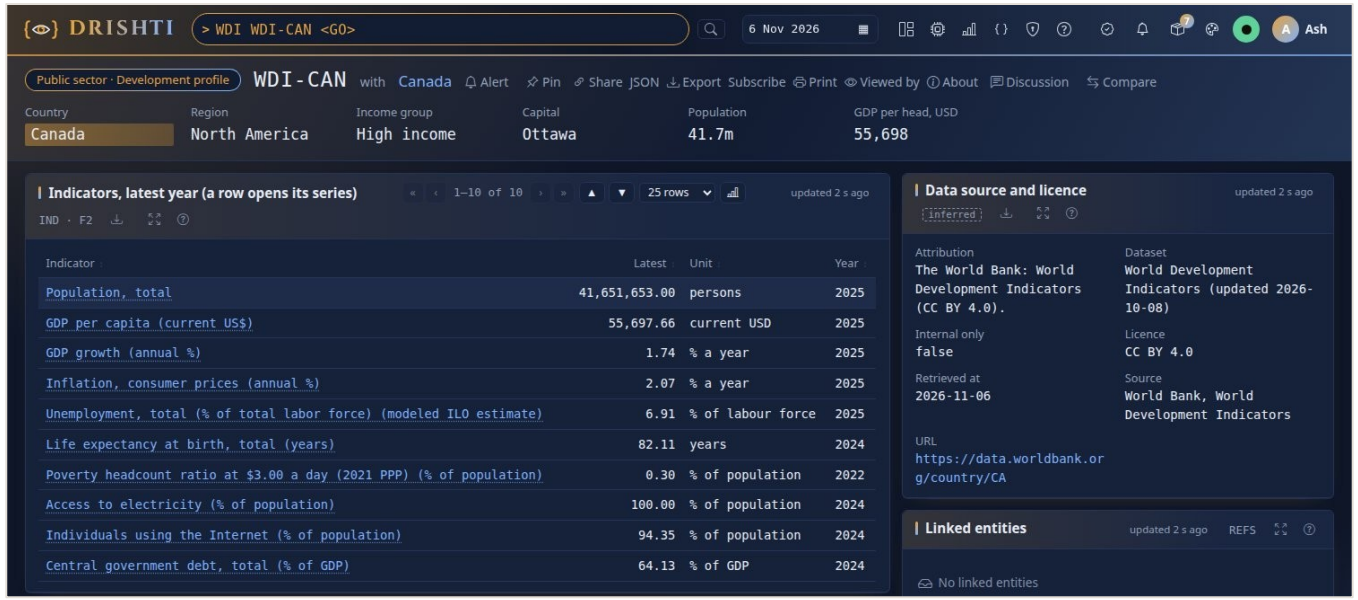
GLEIF legal entities, US equities, funds and ETFs, market data

Internal use only

the IMF forecasts, Yahoo prices (off as shipped), market headlines: each release says so

Why it matters. Outside data arrives the way a bank's own data does: by the same load rules, into named lakes, with every licence recorded and the internal-only sources marked, so nothing restricted reaches a client by accident. A daily entry for cron, systemd or Windows is one command (feed schedule).

Six packs and ten functions in the economy module, with a lens of its own



WDI WDI-CAN: a country's development indicators, with the source and its licence (tutorial 14, data of 2026-11-06)

Function	What it answers
WDI · WDIS	World Bank development indicators; the world map by GDP per head
CBANK · CBSER	central banks' series from FRED and the Bank of Canada
NSTAT	Statistics Canada
OECD · OECDS	the OECD's indicators and series
PRESS · PREL	official press releases: Fed, ECB, Bank of Canada, Bank of England, SEC, OSFI, Federal Register, Canada Gazette
GDELT	world events by country and tone

Built the way every domain will be

Packs in their domain folder, functions owned by the module, a lens, a guide and a tutorial, and data that arrives through drishti-feeds. The free news is in the World lens too.

Why it matters. The second industry proves the first was a module, not a special case.

Build and deploy

05



From JSON Lines files to a deployed module, in one workspace

1 Data
drop .jsonl files; every record is read on the server, none sampled

2 Kind and keys
discriminators give one screen per combination; the ID is checked across every file

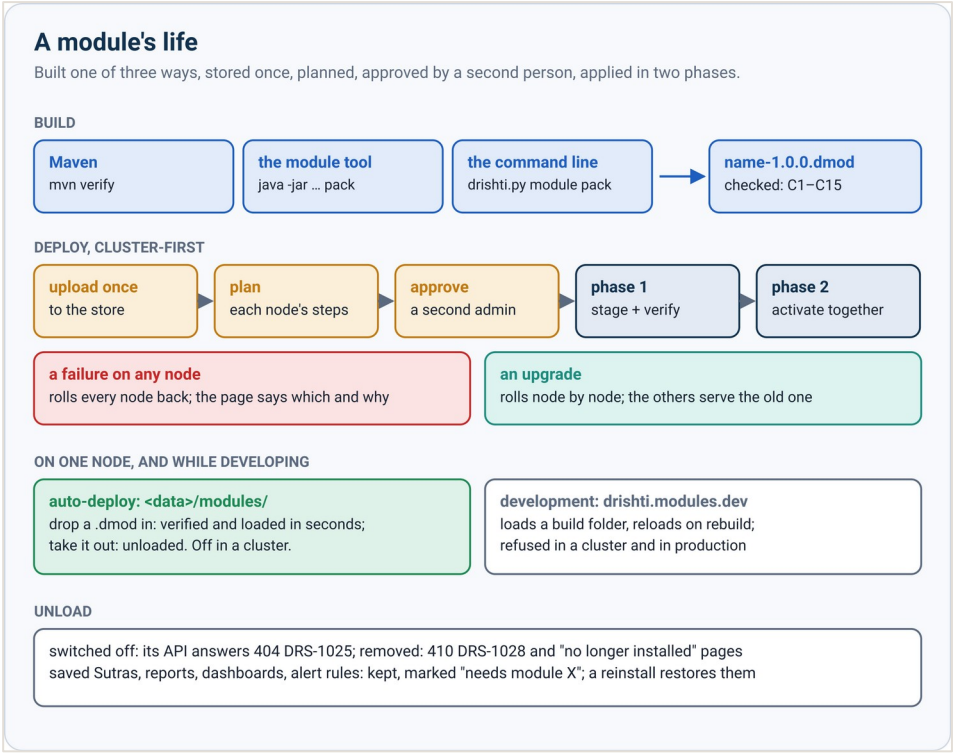
3 Source
name, version, connector and one data binding per kind

4 Design
each Sutra's YAML beside its live screen on real rows

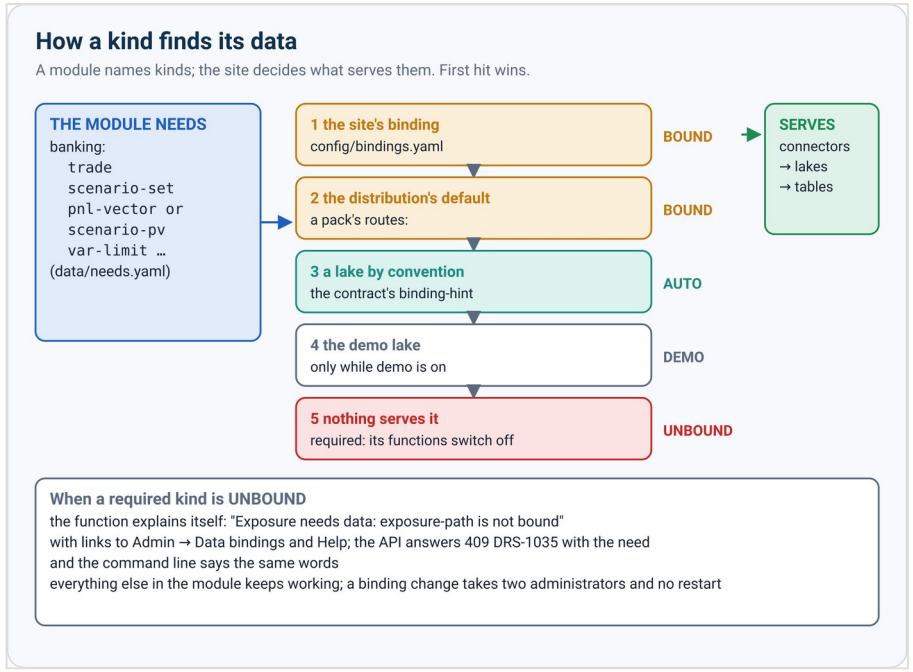
5 Finalize
the pack linted and tested, packed as a .dmod, fifteen checks

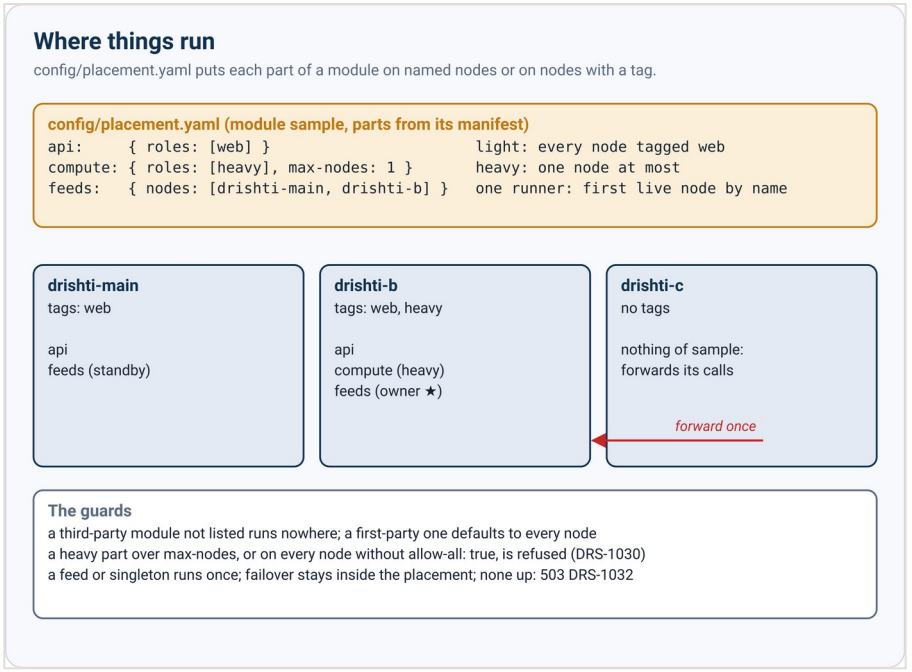
6 Deploy
one module deploy: planned, approved by a second administrator, on every placed node

- Build > New pack at /build/pack/designer, with the same engine as the command line (drishti.py pack new)
- three ways to build a module, one packer: Maven archetype, the standalone module tool, or Python and Java only
- a failure rolls every node back; an upgrade goes node by node; on one node it installs at once
- a second administrator approves every deploy: audited, no restart



Bindings say where data comes from; placement says where it runs





Bindings (config/bindings.yaml)

- one binding per kind: connectors to read, union, live source, where a produced kind is written
- first hit wins: site binding, pack default, lake by convention, demo, else unbound
- an unbound kind switches off only its functions, and they say why

Placement (config/placement.yaml)

- each module part runs by node name or tag: light, heavy, feed, singleton
- a singleton has one owner and fails over inside the placement; heavy work is capped
- a node that does not hold a part forwards the call once, for the same person

Computed, never stored

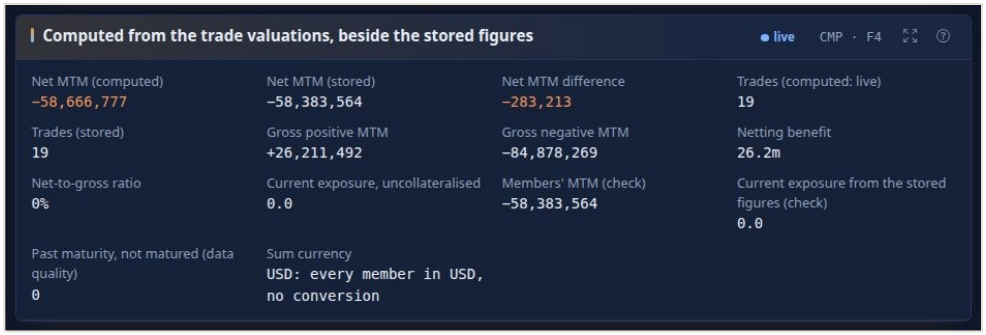
06

Computed on demand from trade-grain inputs, never stored

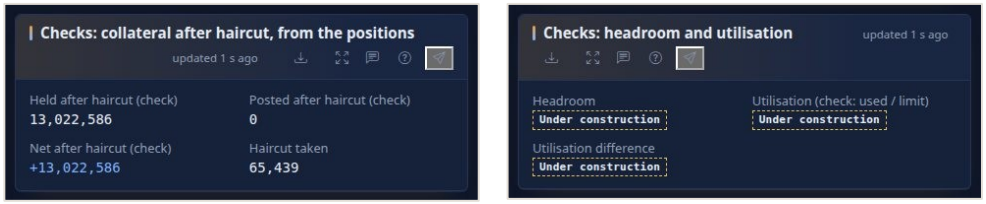
The principle. A total is computed from the trades it adds up, once per business date and data version, and shared by everyone who may open the page. A stored copy is shown beside it with the difference, until it is retired.

Computed	What it is
`NSMTM`	a netting set's MtM from its live trades: net, gross parts, netting benefit, CE, past maturity
`CPMTM`	a counterparty's MtM and its netting sets, from the trade valuations
`SSPNL`	a stress scenario's P&L: every desk, its limit use, breaches
`Checks panels`	collateral after haircut, stress limit use; stored SA-CCR and limit usage: Under construction
`Exposure engine`	members and MtM from trade valuations; collateral after haircut; limit from credit-limit

Live is one rule everywhere: a live status and a maturity not before the valuation date. A live trade past maturity is in no total, and is listed as a data-quality warning.



A netting set: computed, stored and the difference



Checks: the collateral balance and the credit limit

Why

- One definition per figure: no second copy to drift and no batch job to forget
- Every total traceable to the trades it adds up; a stored copy is checked, not trusted

Reconciled to the stored figures, retired fields guarded, VaR unchanged

55 of 76[1]
netting sets: derived net MtM equals the stored one

21 explained[1]
827 live trades past maturity (MtM 16.7m); added back, 76 of 76 agree to the dollar

112,041,646[1]
VAR BANK, USD, 2026-09-30: unchanged, no VaR input moved

Exposure on its real inputs[1]

collateral after haircut, 2026-09-30, USD

Scope · figure	Stored inputs	Computed inputs
BANK · peak PFE 95%	2,858,060,180	2,897,620,439
BANK · EPE	945,577,981	947,202,991
BANK · CE	1,532,007,701	1,586,811,699
CP-ALDERSHOT · CE	217,565,006	220,250,163
NS-ALDERSHOT-NY · CE	212,580,686	214,706,654

Holdings 6,304.4m at market value, 6,229.6m after haircut: less held raises a collateralised set's exposure, less posted lowers it.

Retired fields, guarded

- `netting-set.UTILISATION`, `risk-hierarchy.LIMIT_VAR`, `portfolio.LIMIT_VAR`
- The build fails if one returns (tools/contracts.policy.yaml, removed:); VaR limits live only in var-limit

The difference is shown, not hidden

- The netting set's panel: computed, stored, difference
- CP-ALDERSHOT: stored 520,405,666, derived 518,314,005 (18 trades past maturity)

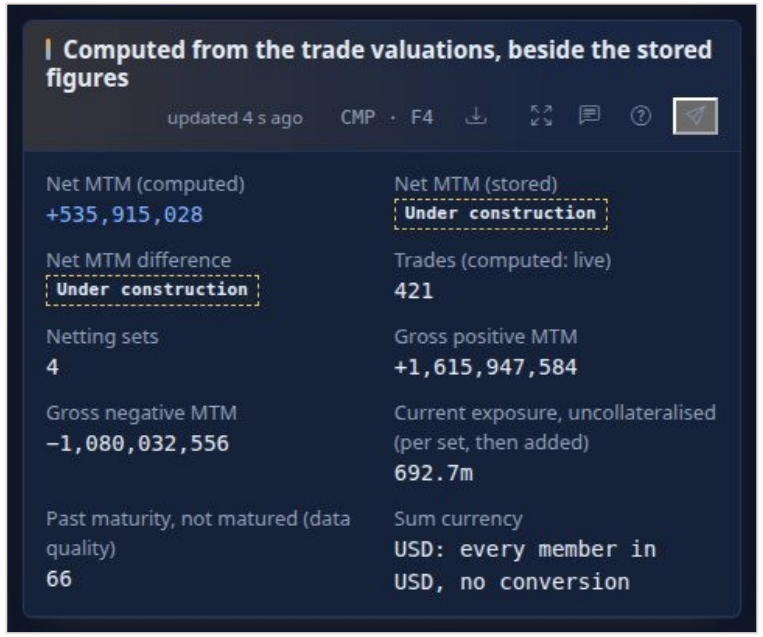
NOT BUILT

Still open

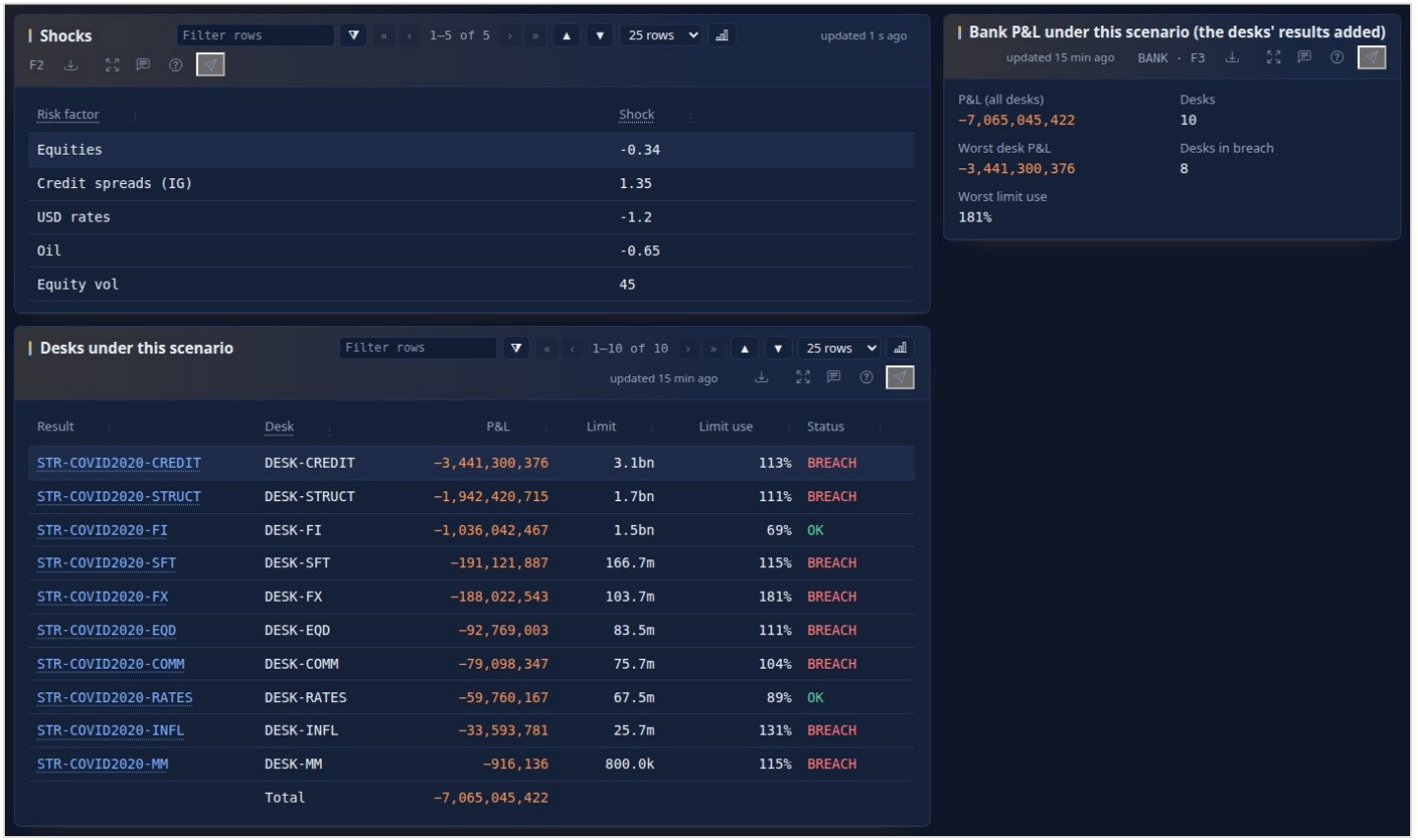
retiring the stored netting-set and counterparty MtM with the netting-set engine; currency conversion on derived totals (all USD today); BI's MtM measures filtered to live trades; stored SA-CCR figures and credit-limit usage shown as under construction until their engines compute them.

[1] Measured on the committed test data, 2026-09-30 (DATA_ARCHITECTURE.md section 12)

On the pages: the counterparty and a stress scenario, computed



A counterparty, computed; its stored copy: Under construction



A stress scenario: the bank's P&L and every desk under it (SSPNL)

Read from the trades, not copied. A page shows the computed figure with its source, its sum currency and its checks; a stored copy no engine computes says Under construction in place of its number.

Pre-deal and one limit rule

07

Does the new trade fit the limit? Approve, refer or decline, with the reasons

```
PREDEAL CP-ALDERSHOT LIKE 1000001 250M
PREDEAL CP-ALDERSHOT EQUITY 768495000 2031-10-20 MTM 9227785
```

Decision	When
approve	exact data; the limit and every tenor bucket within their limits after the trade
decline	exact data; the limit or a tenor bucket over its limit after the trade
refer	anything less than exact, each reason said: a proxy answer, no limit on record, a stale run, netting sets the person does not see, no exposure on the date

How it is computed

A booked trade's simulated paths copied and scaled (LIKE), or the trade described with a conservative proxy; added to its netting set path by path; held to the counterparty's credit limit and each tenor bucket (0-1Y … 10Y+).

The answer

The decision and its reasons; the limit and each bucket before and after; the headroom; EPE and EAD before and after. A proxy never approves and never declines.

Computed as the person asking

Their netting sets only; nothing kept between requests. Every check audited, and explained by the one Explain.

One service

Counterparties → Pre-deal check, POST /api/v1/risk/exposure/predeal, and the Trading landing's new-trade check all ask the same service.

Under construction: SA-CCR's incremental EAD in the answer, said so on the screen.

Within, near or breach: said in the answer, coloured the same everywhere

Within
below 80% of the limit

Near
from 80% up to 100%: still within, said in amber

Breach
over 100% of the limit

Where	Held to the rule
VaR	every scope and hierarchy node with a limit; the VaR what-if
The official record	the end-of-day figures and their limits
Pre-deal	the counterparty's credit limit and each tenor bucket
Sensitivities	the sensitivity ladder's limits

One setting

drishti.risk.limit-near (0.8 as shipped), in the banking module's VaR engine file.

One colour per status

Every bar, tile, pill and heat-map cell takes its colour from the server's status: the screens' own thresholds are gone.

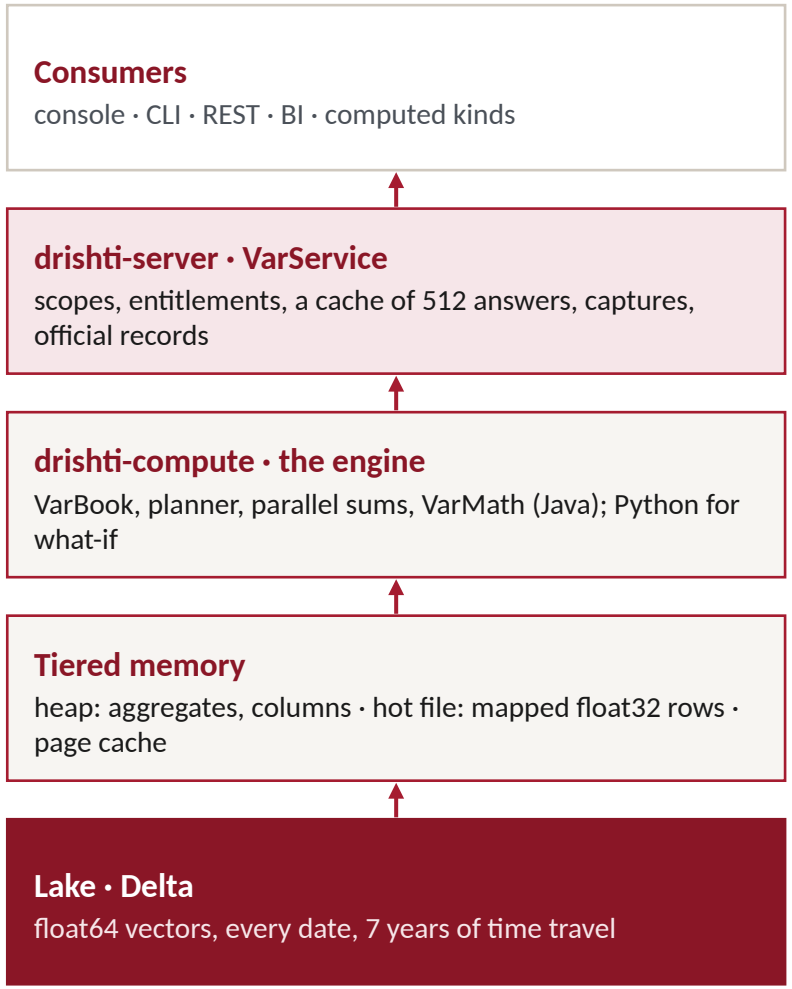
Why it matters. The same use of a limit can no longer be amber on one screen and red on another. The status comes with the number, so a report, an export and a screen agree, and Near never changes a decision: it is the warning before the limit.

How it scales

08



Why it is fast: vectors add, one date stays hot, sums run on every core



Vector math[1]
509 doubles = 4,072 B a trade or position; hot rows float32 (2,036 B); sums in float64

Memory-mapped hot file
one file a date; only touched pages resident; 2 dates open, 60 kept on disk

Planner over aggregates
a node is its aggregate; Markets minus three trades = 4 vector operations

Parallel sums
slices on every core, added in a fixed order: the same bits every time

Currencies built ahead[2]
five books in the background; 21.4 MB a currency a date on the demo

Per-trade-currency totals
a headline in any currency from a node's few currency sums, exact

Intraday increments
cost ∝ rows changed; Kafka: one commit and one apply per window

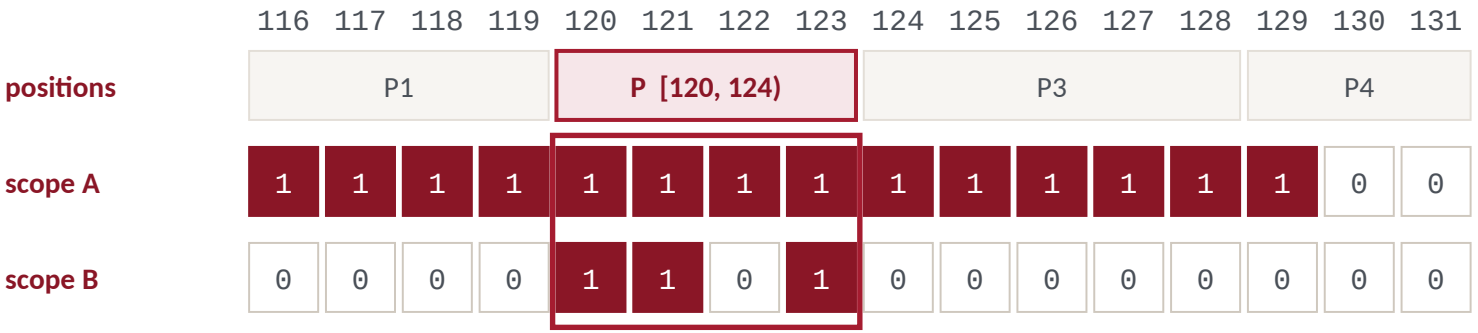
Cluster and history
Drishti*net replicas; VaR sharded across servers; lake history

Measured[3]	Time	On
A node's VaR and ES	0.48 ms	tiered store, 100,000 trades
The whole bank from float32 rows	3.4 ms (≈ 60 GB/s)	tiered store, 100,000 trades
The whole bank from float64 rows	58 ms (≈ 70 GB/s)	heap tier, 1,000,000 trades
A basket of 50,000 scattered positions	2.6 ms + ≈ 45 ms ids	tiered store, 100,000 trades
First load of a date (after the lake read)	0.8–1.2 s	tiered store, 100,000 trades
Loading the demo date	572 ms	9,572 trades, the server

[1]–[3] How these figures were derived: A-1, in the appendix (How it scales)

A scope is a set of ordinals; a position is one run

A business date's trades, numbered 0 … N − 1, sorted by hierarchy cell and position



Is P whole in the scope? `nextClearBit(from) ≥ to`

A: the first ordinal from 120 not in A is 130; 130 ≥ 124, so A holds P whole.

B: the first ordinal from 120 not in B is 122 < 124: B holds P in part, its trades the runs [120, 122) and [123, 124).

A scan of the position's own bits, (to − from) ÷ 64 words, with nothing allocated: the cost of a position is its own trades, whatever the size of the date.

Ordinals

a date's trades numbered in hierarchy and position order: a position's trades are one contiguous run [from, to); an intraday batch's trades follow, listed with their position

A scope is bits[1]

one bit a trade, N ÷ 8 bytes: 25 KB at 200,000 trades, 290 KB at 2.3M; union, difference and intersection word by word; a node is one run

Whole or in part

whole: the position's cached standalone figures; in part: only its runs inside the scope

Cost[2]

per position: its trades ÷ 64 words; per scope: its positions. Nothing grows with the date

[1–2] How these figures were derived: A-1, in the appendix (How it scales: the algorithms)

Totals over every position; the row limit is for display only

A scope of P positions: one pass, in ordinal order

Each whole position
standalone VaR read from the cache: computed once per date and lens, the first time a scope holds the position, then kept

Each position held in part
the VaR of its trades inside the scope, summed on the fly

Missing from the cache
256 or more: computed in parallel on the engine's sum threads

→

Totals, over every part

`Σ standalone`
the pass on the left

`Σ component = VaR`
no pass at all: the parts partition the scope, so their components add up to its own

`diversification`
`= Σ standalone − VaR`

The row limit: max-parts, 5,000

5,000 of 18,567 positions shown (the largest by gross size); totals are over all 18,567

Position	Standalone	Component
POS-…-0001	4.21m	2.97m
POS-…-0002	3.88m	2.41m
… 4,997 more rows		
Totals, all 18,567	Σ 231.4m	Σ = VaR

- The rows: the largest by gross size, chosen with a heap of max-parts entries, O(P log max-parts), kept in ordinal order
- No total depends on the limit; the answer says truncated, total and shown

Worked: a basket of three positions

Standalone VaRs 100, 80 and 50; the basket's VaR 170.

Σ standalone 230 · Σ component 170 · diversification 60.

With a limit of 2 rows the table lists the two largest by gross size and says 2 of 3 shown; the three totals stay 230, 170 and 60.

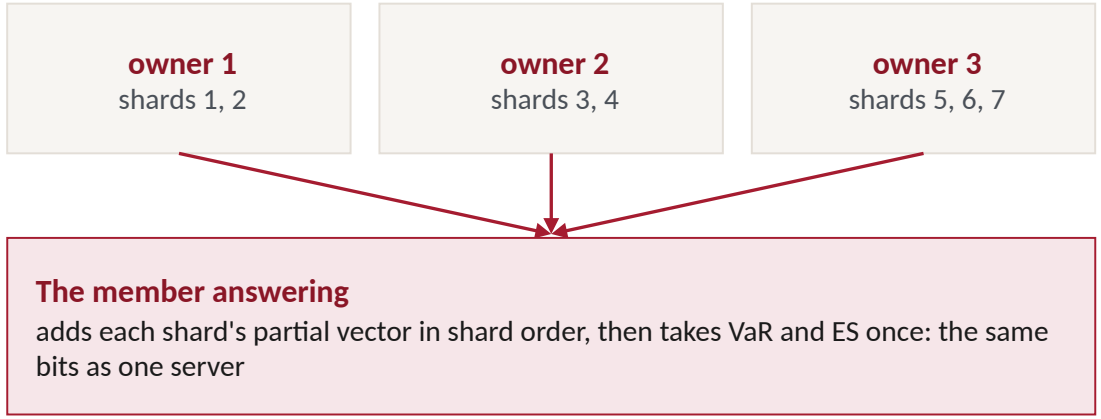
Memory and the k worst[1,2]

8 bytes a position and lens: 1.6 MB at 200,000 positions, 18 MB at 2.3M, in the one cache budget, released with its date. A standalone VaR keeps only the k worst values (k = 6 at 99% of 509) while reading the vector once.

[1–2] How these figures were derived: A-1, in the appendix (How it scales: the algorithms) Figures in the table: illustrative

Sums added shard by shard; a date's files decoded on several threads

In a sharded cluster: partial sums, added in shard order



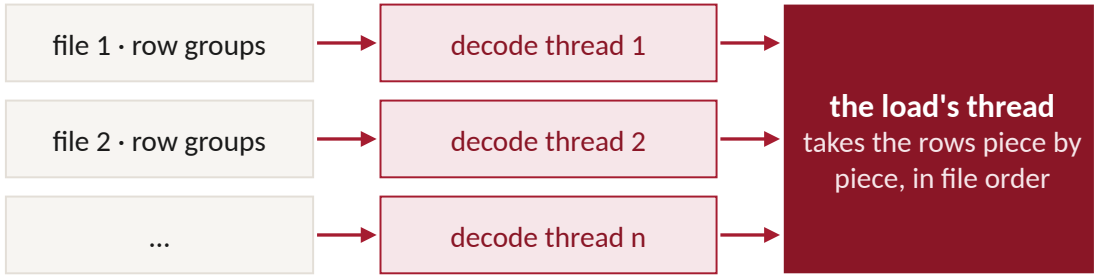
- Exactly one answer a shard; no live owner of a needed shard: refused in words, never a partial VaR
- Every node's aggregate is summed bottom-up as on one server: the bank's VaR is the same bits on every member
- The totals over the parts: computed by the member answering from each position's vector, cached as on one server

Per-shard Σ standalone

DESIGNED

each owner returns its shard's partial Σ standalone over the scope's whole positions from its own cache; the sums are additive across shards and, added in shard order, the same on every member

A date's first open: decoded on several threads, in file order



- Only the columns the load uses are read: the id, the vector, the trade's text columns
- The intake, sort, sums and hot file see exactly the rows one thread would give, in the same order: the same book to the bit
- Bounded: drishti.var.read.threads (half the cores, at most 8) and read.buffer-mb (64 MB, at most 1/32 of the heap)
- A piece that cannot be read fails the load with its error, as one thread would

Measured[1]	First open	The read
50,000 positions, one server	1.8 s	0.8 s
200,000 positions, one server	4.5 s	70–73% of CPU
1,000,000 positions, one server	19–21 s	65–69% of CPU
200,000 positions, cluster of 3	10.8 s	by the owners

[1] Measured on one workstation, 24 hardware threads: 50,000 positions at -Xmx4g (VAR_GUIDE 27a.6); 200,000 and 1,000,000: BENCHMARK.md section 4

Ten dates in memory; the least recently used leaves first

1. **Ten dates in memory**
   a setting (drishti.var.hot-dates, default 10: two weeks of business dates); exposure the same
2. **The least recently used leaves first**
   every request stamps the date it reads; one more date open closes the date used longest ago
3. **Always kept**
   the date in use, the newest business date and the pinned dates
4. **A byte budget as the safety net**
   a date's currency books leave first, then its stress book, then whole dates; never a kept one
5. **Refusals politely remembered**
   a date refused for room answers 503 at once with Retry-After for five minutes: fifty requests at once make one load, not fifty

Pinned from the calendar: New York and Toronto

Pin	The last business day of	Example
month-end	each month, in every calendar	2026-05-29
quarter-end	March, June, September, December	2026-09-29
year-end	December	2026-12-31
fiscal year-end	the bank's fiscal year	2026-10-30

- a holiday in either city moves the date back: 2026-09-29, as the 30th is Toronto's National Day for Truth and Reconciliation
- each rule switched on by the bank; kept on disk (built ahead, opens in seconds) or held in memory; the stress window and named dates too

At 1,000,000 positions[1]

a date in memory takes about 0.6–0.9 GB of heap; its hot file (3.14 GB) is mapped and on local disk; every date stays in the lake for the years kept

Memory grows with the dates open at once, a setting, never with the years kept.

[1] Measured: one server, -Xmx6g, Java 21, G1, two dates open (BENCHMARK.md section 4)

Evidence: scale and capacity

09

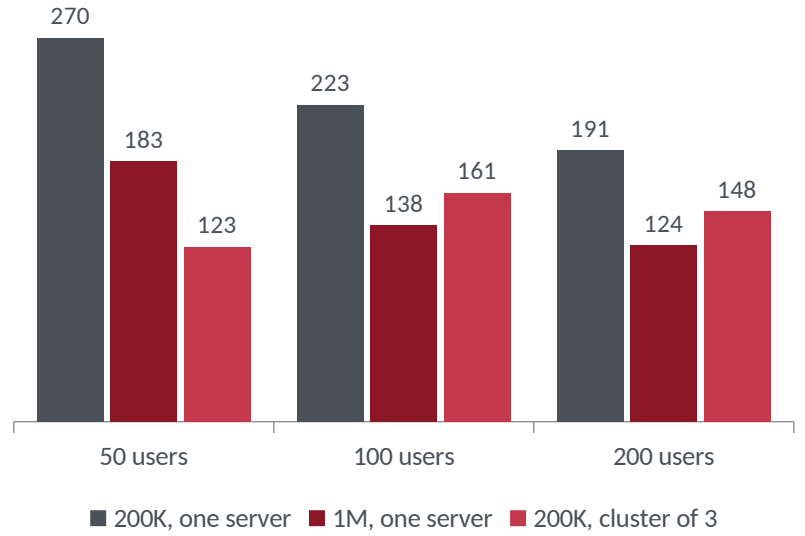
Measured: 200,000 and 1,000,000 positions side by side

One workstation: one server at each size, and a cluster of three at 200,000[1]

Measure	200K, one server	1M, one server	200K, cluster of 3
Requests a second, 50 / 100 / 200 users	270 / 223 / 191	183 / 138 / 124	123 / 161 / 148
p99 a VaR node, 50 / 200 users	0.72 / 5.39 s	1.08 / 7.33 s	0.46 / 5.08 s
p99 the bank minus a desk, 50 / 200 users	0.92 / 5.44 s	1.17 / 7.19 s	1.11 / 5.15 s
A date's first open	4.5 s	19–21 s	10.8 s
Load: five kinds, two dates, Java loader	13.9 s	28.4 s	the same lake
Heap after GC, two dates open	0.88 GB	2.2 GB	1.03 / 0.82 / 0.53 GB
Resident memory (RSS) at the end	5.0 GB	11.2 GB	4.9 / 5.6 / 5.4 GB
Disk a date: lake / hot file	0.29 / 0.56 GB	1.43 / 3.14 GB	the same lake

- **A cluster of three at 1,000,000 positions:** not yet measured[2]
- Mix: half VaR (nodes, the bank minus a desk, 50,000-position baskets, contributions, P&L paths), half trade pages; 60 s a step after a 45 s warm-up
- Single server -Xmx8g at 200K, -Xmx6g at 1M; the cluster -Xmx4g a member, VaR sharded, 2 replicas a shard; Java 21, G1

Requests a second, by users



- 200K, one server: 270 a second at 50 users, every p99 under 1 s; at 200 users the wait is queueing on 24 threads
- 1M, one server at -Xmx6g: 183 a second at 50 users with no failure at any step; every kind of question under 1.4 s at p99 with 50 users

[1] Measured on one workstation (24 threads, 61 GB, NVMe), Java 21, G1: BENCHMARK.md section 4 [2] Three servers at 1,000,000 positions: not yet run

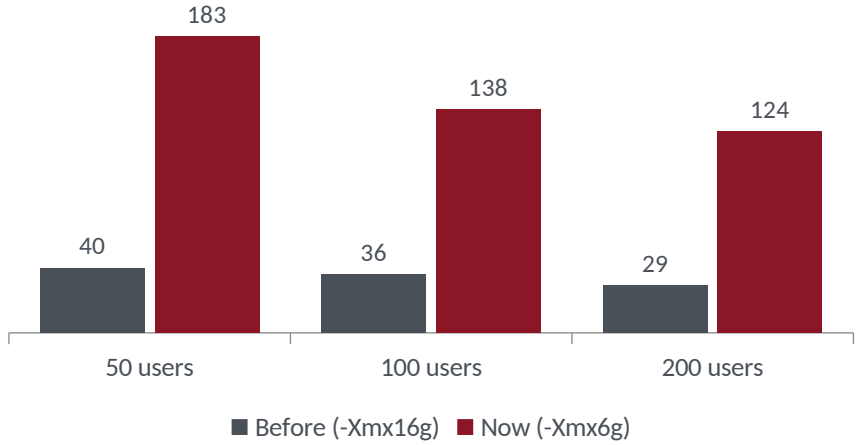
One server in 6 GB: under a third of the heap, four times the requests

The same data, machine and questions, measured twice[1]

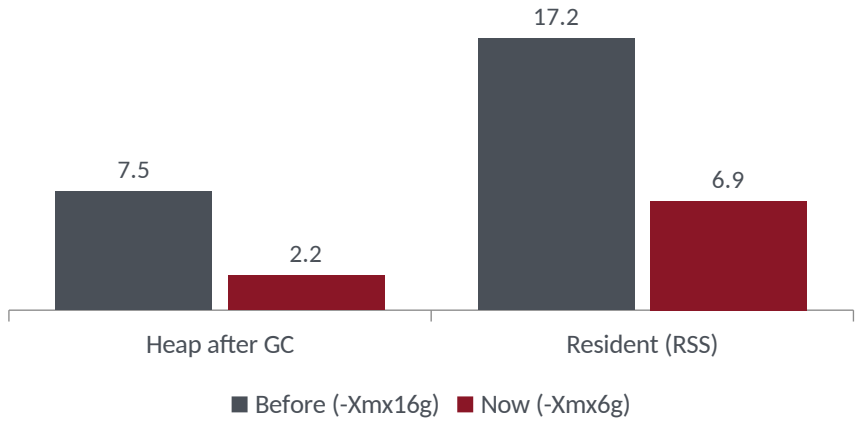
1,000,000 positions, one server	Before	Now
Heap setting (-Xmx)	16 GB	6 GB
Heap after GC, two dates open	7.5 GB	2.2 GB
Heap a date	about 3.2 GB	about 0.6–0.9 GB
Resident memory (RSS), two dates	17.2 GB	6.9 GB
A date's first open	25–27 s	19–21 s
Requests a second, 50 / 100 / 200 users	40 / 36 / 29	183 / 138 / 124
Failed requests, 50 / 100 / 200 users	0 / 23 / 77	0 / 0 / 0
The bank minus a desk, p50, 50 users	5.7 s	0.20 s
VaR answers, both dates	the reference	identical to the dollar

- **Node members as runs:** a node's members are kept as the sorted runs of its books' ordinals, not a bitmap as long as the date
- **Aggregates read from the hot file:** mapped like the position rows, not copied onto the heap
- **A scope's parts summed run by run:** the scan that took nine tenths of the CPU under load is gone

Requests a second, by users



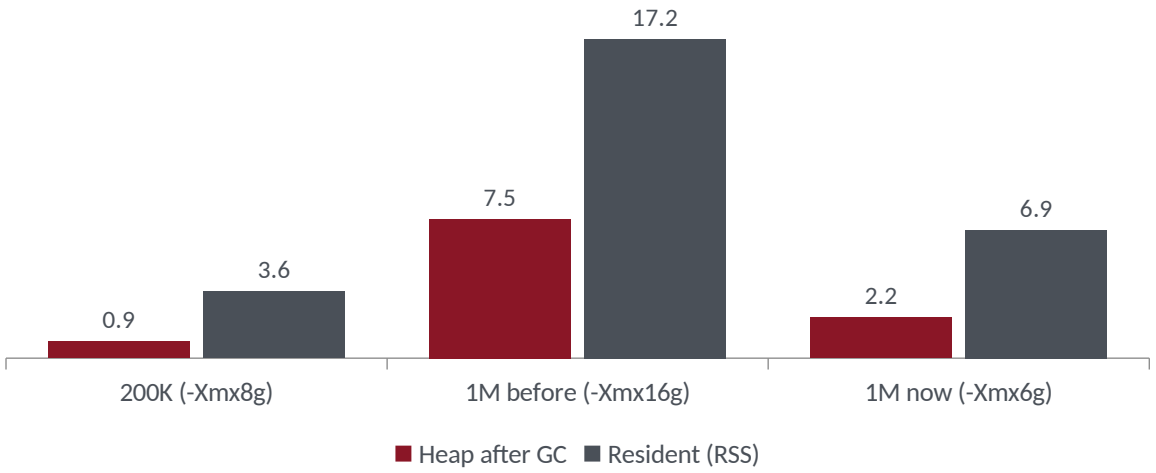
Memory with two dates open, GB



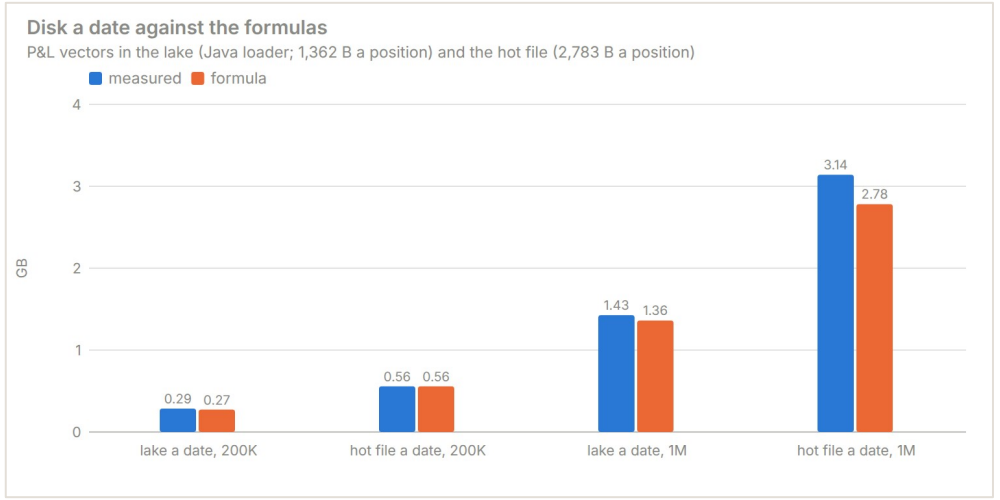
[1] Measured on one workstation (24 threads, 61 GB, NVMe), Java 21, G1: BENCHMARK.md section 4; 1,000,000 positions × 509 scenarios, two dates, 27,024 nodes

Heap and disk at both sizes, against the formula

Heap and resident memory, two dates open, GB[1]



Disk a date: the lake and the hot file



The heap a date[1]

Node members as runs, aggregates mapped from the hot file

At 1,000,000 positions and 27,024 nodes a date takes about 0.6–0.9 GB of heap, against 3.2 GB before, when each node's members were a bitmap as long as the date. The formula gives about 0.77 GB a date; two dates fit a 6 GB heap.

The capacity model's 80 MB of heap a date per 100,000 positions, 0.8 GB at a million, holds.

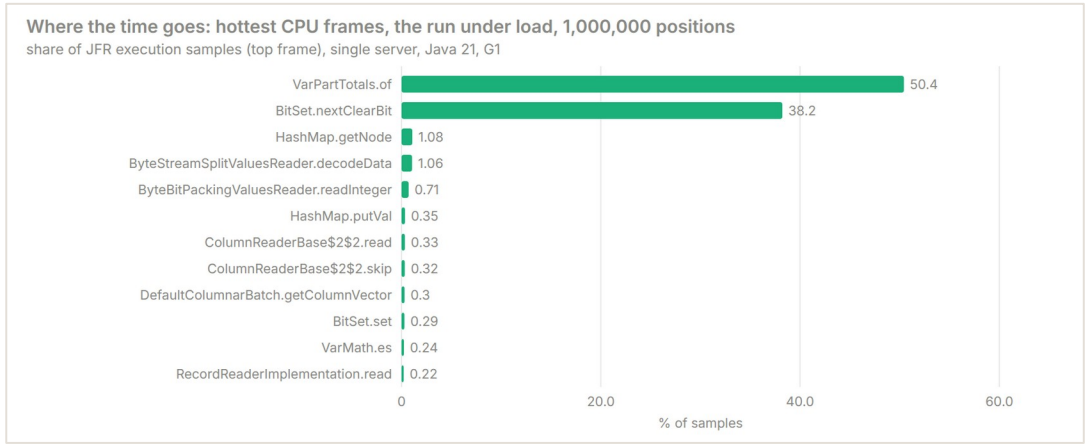
Disk a date[1]

The lake: 1,362 B a position, 5% above it at both sizes. The hot file: 2,783 B a position, met at 200,000, 13% above at 1,000,000 where 27,024 nodes add 639 MB of aggregates. Resident memory is about three times the heap: the mapped hot files count in it.

[1] Measured on one workstation (24 threads, 61 GB, NVMe), Java 21, G1: BENCHMARK.md section 4; the formula: VAR_GUIDE.md 27.1

Where the time went, and what the memory work changed

Before: hottest CPU frames under load, 1,000,000 positions[1]



- Before, at -Xmx16g: summing a scope's parts position by position, with its bit-set scan, took 89% of the CPU under load
- Now, at -Xmx6g: the bank minus a desk 0.20 s at p50 with 50 users, against 5.7 s
- Heap: about 0.6–0.9 GB a date, against 3.2 GB
- A date's first open: decoding the lake's Parquet is most of its CPU

What changed, and what comes next

1 A scope's parts summed by runs: done
the totals walk runs, not positions; the bank minus a desk no longer grows with the bank

2 The heap a date under a gigabyte: done[2]
node members as runs, aggregates mapped from the hot file: 3.2 GB a date became about 0.6–0.9 GB

3 Ten dates in memory: done
least recently used out first; month-, quarter- and year-ends pinned; a byte budget as the safety net

4 Many scopes in one pass; OpenBLAS bundled, off
each tile of rows read once for every scope; OpenBLAS gives the same VaR, turned on per node after measuring; profiling comes with the native work later

[1] Measured before the memory work: JFR execution samples, one server, Java 21, G1 (BENCHMARK.md 4.9) [2] Measured after it, one server at -Xmx6g (BENCHMARK.md section 4)

Many scopes in one pass; OpenBLAS bundled, and off as shipped

MEASURED **1,000,000 positions × 509 scenarios, 100 scopes summed at once[1]**

Shape of the 100 scopes	One at a time	One pass, Java	One pass, OpenBLAS
Cube cells (each row in one)	291.1 ms	124.7 ms · 2.3×	121.2 ms · 2.4×
Overlapping portfolios	3,209.8 ms	2,645.4 ms · 1.2×	1,028.4 ms · 3.1×
Desks (contiguous ranges)	110.3 ms	33.0 ms · 3.3×	34.5 ms · 3.2×

At 200,000 positions[1]: cube cells 2.0× on both paths; overlapping portfolios 1.9× on Java and 2.4× on OpenBLAS; desks 2.2× on Java. At 10 or 1,000 scopes the gain is small (0.9× to 1.2×).

The same bits

The Java path reads each tile of rows once for every scope and gives the same bits as one scope at a time; the benchmark checks every vector. OpenBLAS agrees within a relative 1e-12, with the same VaR scenario date.

Read it with care

One workstation, with a release check running beside it: the ratios are the result, not the milliseconds. The dgemm thresholds were tuned on this machine; measure again on the bank's hardware.

OpenBLAS: bundled, off as shipped

Linux and Windows x86-64, nothing to install. A 250,000-position run gave the same VaR to the dollar on and off, with no clear end-to-end gain at request level[2].

On per node, after measuring

Admin → Engines turns it on live, after a self-test; a failure falls back to Java with a warning, never a stop. In a cluster, one member on Java takes the many-scope sums to Java.

Kept honest

Every release's VaR check runs on Java (bit-exact against the previous figures) and with OpenBLAS (within 1e-12, the same VaR dates).

Next

Profiling comes with the native work of a later release; the request path is where the next gain is looked for.

[1] Measured 2026-11-03, AMD Ryzen AI 9 HX 370 (24 threads, 61 GB), ManyScopeBenchmark, best of the repeats (BENCHMARK.md 3, 17) [2] CHANGELOG: OpenBLAS off as shipped

Bank scale, from first principles and the measured figures

Per business date	Arithmetic	Result	Basis
Vectors, float64 (the lake)[1]	2.3M × 509 × 8 B	9.37 GB	arithmetic
Vectors, float32 (hot rows)[1]	2.3M × 509 × 4 B	4.68 GB	arithmetic
Hot file, USD book[2]	237.7 MB per 100k × 23	≈ 5.5 GB	*estimated*
Heap, one open date[2]	≈ 80 MB per 100k × 23	≈ 1.8 GB	*estimated*
First load, after the lake read[2]	0.8–1.2 s per 100k × 23	≈ 18–28 s	*estimated*
Reopen a date from its file[2]	23 ms per 100k × 23	≈ 0.5 s	*estimated*
Whole bank from rows, resident[3]	4.68 GB at ≈ 60 GB/s	≈ 80 ms	*estimated*
... cold, from NVMe[3]	4.68 GB at ≈ 3 GB/s	≈ 1.6 s	*estimated*
A node, any size[3]	its float64 aggregate	≈ 0.5 ms	**measured**
4 extra currencies, disk[4]	250 MB × 4 per 100k × 23	≈ 23 GB a date	*estimated*
... 5 retained dates; build[4]	23 GB × 5; 4.3 s per 100k × 23	≈ 115 GB; ≈ 99 s	*estimated*
USD hot files, 60 dates[5]	5.5 GB × 60	≈ 330 GB disk	*estimated*
Lake, a year (Parquet ≈ 0.75×)[6]	9.37 GB × 0.75 × 252	≈ 1.8 TB; ≈ 12 TB in 7 y	*estimated*

Position grain keeps rows = positions: no trade matrix in the hot file. *Estimated = scaled linearly (× 23) from the measured 100,000-trade figures (VAR_GUIDE 27.3, 15.11); not measured at 2.3M.*

15,000 portfolios[7]

- A portfolio is the sum of its positions: the cost is all portfolios' positions together × 509 additions.
- Assumed 130 positions each: 1.95M rows, 3.97 GB of float32 → ≈ 0.07 s at 60 GB/s, ≈ 0.8 s scattered; per currency, estimated.
- Roll-ups: a HIERARCHY portfolio is a node aggregate; whole books cost one add each; a node minus a few rows is aggregate − rows (built).
- Store them: 15k × 509 × 8 B ≈ 61 MB a date a currency; float32 halves it (proposed).

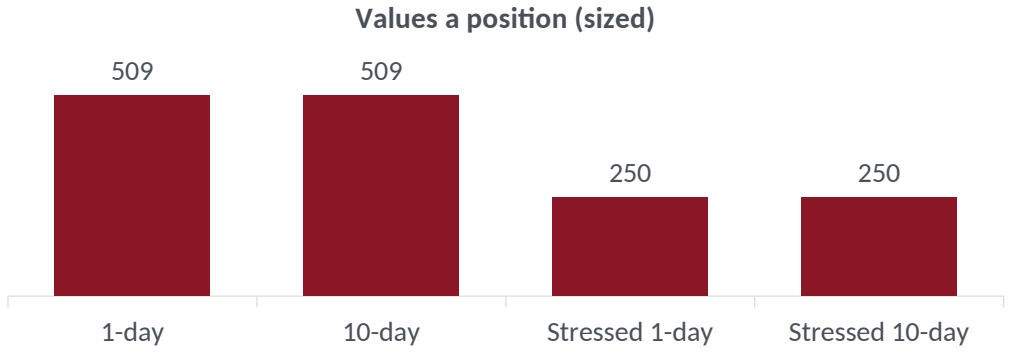
Totals and intraday[8]

- Currency totals: 510 × 8 B per node per trade currency; (5,000 MGMT and LEGAL + ≈ 700 limit-letter nodes) × 10 ≈ 233 MB (assumed shape).
- Per changed trade or position: ≈ 11 node vectors (three trees) × 5 currency books ≈ 55 additions of 509: microseconds; the lake append dominates (≈ 1 s a batch).
- A headline in any currency: a node's few trade-currency sums, converted: microseconds.

[1–8] How these figures were derived: A-1, in the appendix (Sizing)

10-day and stressed VaR: the added workload

Measure	Vector a position	× 1-day
1-day VaR and ES	509 scenarios (2 years)	1.00
10-day (b), a full-revaluation set	500 (509 − 9), sized 509	+ 1.00
Stressed 1-day VaR, ES	≈ 250 (12 months, e.g. 2008–09)	+ 0.49
Stressed 10-day	≈ 250 overlapping	+ 0.49
Total, sized[1]	1,518 values a position	2.98



Per date and server	1-day only	All four	Label
Vectors, float64[2]	9.37 GB	27.9 GB	ESTIMATED
Rows, float32[2]	4.68 GB	14.0 GB	ESTIMATED
USD hot file[3]	5.47 GB	16.3 GB	ESTIMATED
Heap, an open date[4]	1.84 GB	2.23 GB	ESTIMATED
First open: lake read[5]	18–70 s	54–209 s	ESTIMATED
First open: build[5]	18–28 s	55–82 s	ESTIMATED
Other currency, asked[6]	5.75 GB	+ 11.4 GB	ASSUMPTION
Local NVMe a server[7]	458 GB	1,183 GB	ESTIMATED
RAM a server[7]	76 GB	151 GB	ESTIMATED
Lake, 7 years[7]	16.1 TB	48.2 TB	ESTIMATED

Method (ASSUMPTION): 10-day by (b), overlapping returns, the conservative case; (a) √10 × 1-day adds no data, × 1.49 only.
Currencies (ASSUMPTION): USD builds all four sets with the date; the others build 1-day ahead and the rest on first ask.

About 3× the 1-day footprint per date. With (a) √10 instead[8]: × 1.49, vectors 18.5 TB over 7 years. Sums per measure are unchanged: a node 0.5 ms, the whole bank ≈ 80 ms for a 509-wide set, ≈ 40 ms for a stressed one (estimated).

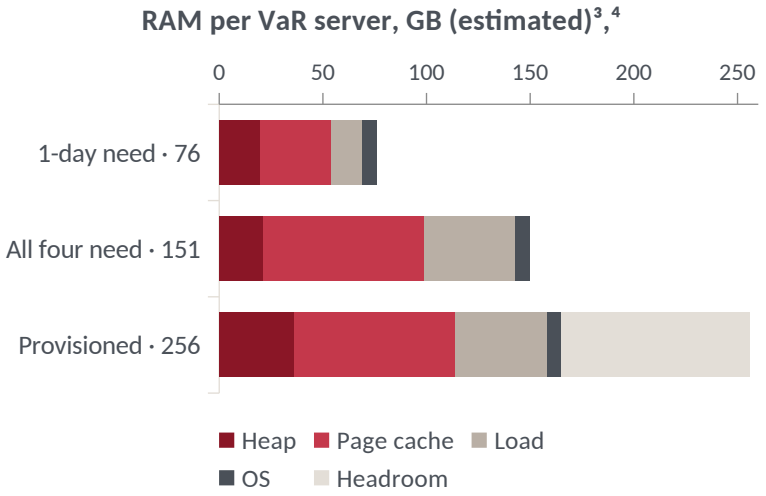
[1–8] How these figures were derived: A-2, in the appendix (10-day and stressed VaR)

Capacity: processes and memory, 1-day, 10-day and stressed

Processes, recommended one server holds each date's whole VaR

Role	No.	Holds and does
Official EOD server	1	captures every node of every tree and measure; publishes (DRISHTI_VAR_OFFICIAL here only)
Intraday writer	1	Kafka PV topic, one lake batch a 5-minute window, and files (DRISHTI_VAR_PV_STREAM here only)
UI/API VaR replicas	2	each maps today and yesterday whole, applies intraday; Drishti*net failover
Console hosts	2	4 workers each; load balancer, no stickiness; one session secret
Cluster, sharding	–	servers as one; a date's VaR split by shard (next slides)

Per VaR server	Need	Provisioned	Large (2×)[8]
vCPU[6]	32	64	96
-Xmx[1]	21 GB	36g	60g
Memory budget, MB[2]	14,000	23,000	46,000
RAM[4]	151 GB	256 GB	512 GB
Local NVMe (tier.dir)[5]	1.18 TB	3.8 TB	5.7 TB



CPU: 32 vCPU needed → 64[6]

- Sums are bandwidth-bound: 24 threads gave ≈ 60 GB/s (MEASURED); all four sets of the bank ≈ 233 ms
- + 8 vCPU for a load, currency builds, intraday, requests; 32 ÷ 0.61 = 52 → 64

39% headroom: the projected peak uses at most 61% of what is provisioned: need ÷ 0.61 (× 1.64). Set DRISHTI_VAR_MEMORY_MB: the default ¼ of the heap (9 GB at 36g) refuses a 14 GB date.

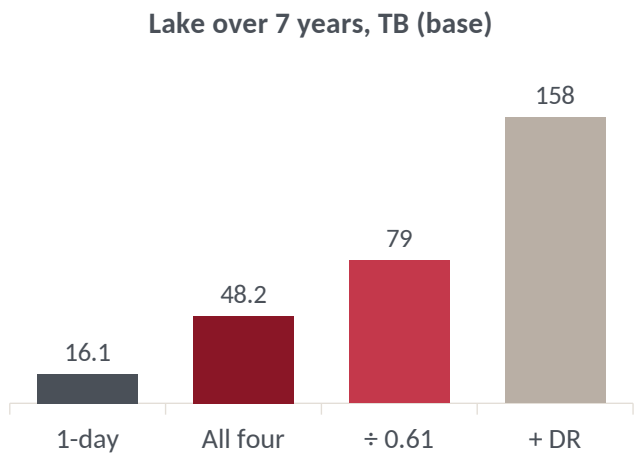
Consoles[7]: 2 hosts × 4 workers; 40 pages/s a worker (MEASURED); 50 users at once (ASSUMPTION): peak 25 pages/s ÷ 0.61 = 41 against one host's 160.

[1–8] How these figures were derived: A-2, in the appendix (Processes and memory)

Capacity: storage over 7 years

Item	Arithmetic (all four sets)	1-day	All four	Label
P&L vectors, float64[1]	27.9 GB × 0.75 × 1,764 dates	12.4 TB	37.0 TB	ESTIMATED
Row columns (terms)[2]	2.3M × ≈ 100 B × 1,764 × 4 sets	0.4 TB	1.6 TB	ASSUMPTION
Scenario PVs as well[2]	the vectors again, if both kept	+ 12.4	+ 37.0	ASSUMPTION
Trades, if held[3]	1.5 KB × 2.3M × 1,764	6.1 TB	6.1 TB	ESTIMATED
Positions (membership)[3]	41 B × 2.3M × 1,764	0.17 TB	0.17 TB	ESTIMATED
Official results[4]	≈ 29 KB × 5,700 nodes × 1,764 × 2.98	0.29 TB	0.88 TB	ESTIMATED
Intraday batches[5]	230k × 9.1 KB × 1,764 × 2	2.5 TB	7.4 TB	ASSUMPTION
Restatements[5]	2% of dates × 20.9 GB × 1,764	0.25 TB	0.74 TB	ASSUMPTION
Delta log, checkpoints[6]	daily 0.14 × 2.98 (often: 4.9)	0.14 TB	0.42 TB	ESTIMATED
Need, base[7]	37.0 + 1.6 + 7.4 + 0.9 + 0.2 + 0.7 + 0.4	16.1 TB	48.2 TB	ESTIMATED
Provisioned (÷ 0.61)[7]	39% headroom; DR doubles it	26.4 TB	79 TB	ESTIMATED
Need, upper[7]	+ PVs, trades, frequent checkpoints	36.1 TB	95.7 TB	ESTIMATED

With a DR copy: 158 TB provisioned (base), 314 TB (upper). Delta or Iceberg, not both. 7 × 252 = 1,764 dates; Parquet ≈ 0.75 × raw; × 2.98 = all four sets.



Local NVMe, per server[9]

- USD 16.3 GB × 60 dates 978 + currencies 161 + a load 44 = 1,183 GB ÷ 0.61 → 3.8 TB; rebuilt from the lake, never on EFS

The 250-day window[8]

- 1-day and 10-day cut to 250: 1,518 → 1,000 values (× 0.66); float32 14.0 → 9.2 GB; NVMe ≈ 780 GB
- The lake shrinks (37.0 → 24.3 TB) only if the bank sends 250-day sets

[1–9] How these figures were derived: A-3, in the appendix (Storage over 7 years)

Capacity for a sharded cluster: 6 owners, 2 coordinators, 2 consoles

The scenario a bank of 2.3 million positions and 15,000 portfolios, with 1-day, 10-day and stressed VaR: 1,518 values a position, the conservative case of the single-server slides.

Role	Count	Heap (-Xmx)	RAM a server	vCPU	Local NVMe	Label
VaR shard owners	6	36g[2]	128 GB (need 69)[3]	16[3]	~650 GB (need 394)[1]	ESTIMATED
Coordinators (coordinator-only, no shards)[4]	2, failover	16–20 GB	32 GB	8	~50 GB	ASSUMPTION
Consoles[4]	2, load balancer	–	8 GB	4	–	ASSUMPTION

```
# the market-risk pack's sharding: block
sharding:
  enabled: true
  key: [LEGAL_ENTITY, DESK]
  replicas: 2       # two zones
  target-shard-mb: 2048
# coordinators: coordinator-only: true
# Java 25 -XX:+UseCompactObjectHeaders
```

Each shard on two servers

- Shards are legal entity and desk ranges near 2 GB; a desk is never split
- Any one server can fail: the other replica answers
- Replacing a server moves about 1/6 of the rows

Smaller options

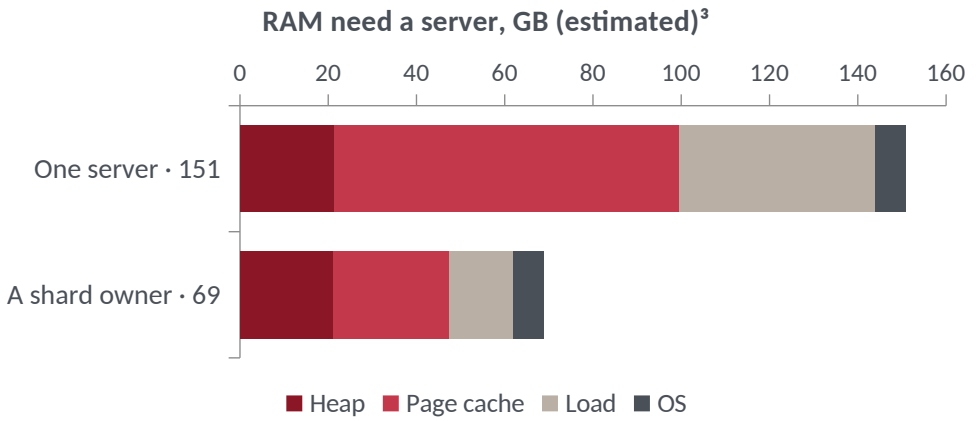
- **1-day VaR only**[5]**:** 4 owners, 24g heap, 96 GB RAM (need 47), plus 2 coordinators
- **Minimum start**[6]**:** 3 owners and 1 coordinator, 36g heap; 192 GB RAM with 2 replicas (need 110), 128 GB with 1 (less failure headroom)

DESIGNED **Full-revaluation 10-day vector sets are designed, not built;** with stressed VaR they triple the footprint, so they are sized here. VaR sharding itself is built (off by default). 1-day only, six owners would need about 39 GB each.[5]

Estimated, not measured at 2.3M. Every figure is scaled × 23 from the measured 100,000-trade benchmark; the 200K benchmark is in the evidence chapter. A 2.3M load on one real bank server comes before ordering hardware.

[1–6] How these figures were derived: A-3, in the appendix (A sharded cluster)

Sharding divides the cache and the disk, not the heap



One server against six owners	One server	An owner
RAM need	151 GB	69 GB
vCPU	32 needed	16 (96 in all)
Local NVMe need	1.18 TB	394 GB

ESTIMATED the nightly official run over 15,000 portfolios, spread over 6 owners: minutes[7]

Divided by the shards

- Page cache for the hot files: 78.4 GB × 2 replicas ÷ 6 owners = 26.1 GB[1]
- A load's spill files: 44.2 × 1/3 = 14.7 GB; NVMe 1,183 × 1/3 = 394 GB

Not divided: the heap

- Every member keeps the date's skeleton for all 2.3M positions (ids, columns, book and node aggregates), so it can answer any scope: about 2 GB an open date
- Two dates, the answer cache, × 1.5: 21.2 GB need → -Xmx36g with the 39% headroom; 24g for 1-day only[2],[5]

An owner: 21 + 26 + 15 + 7 = 69 GB[3]

- ÷ 0.61 = 113 → 128 GB machines (96 GB for 1-day only)
- 15,000 portfolios are light: memberships about 31 MB[7]; each portfolio a sum of its rows, in parallel across shards

Failure and growth. With 2 replicas any one server can fail and every scope still answers; replacing one moves about 1/6 of the rows. More positions add owners, not bigger machines. All figures ESTIMATED (× 23 from the measured 100K benchmark); the 200K benchmark is in the evidence chapter; a 2.3M load on one real server comes next.

[1–7] How these figures were derived: A-3, in the appendix (A sharded cluster)

How many people at once: what risk users can expect

300–500[5] ESTIMATED

people active at once (1,500–5,000 named)

330–650 req/s[2] ESTIMATED

sustained to peak, the 6 + 2 + 2 cluster

The console ESTIMATED

is the first limit, not the VaR servers

Measured[1] one 24-core server, 8 GB heap, a 1.16M-trade lake

What	Result
Mixed reads, 100 users	≈ 140 req/s, 0 failures
Single trade p95, 10 / 50 users	0.08 s / ≈ 1 s; then queues
One console, 50 users	≈ 69 req/s; search ≈ 4 s (open)
VaR of a node, one user	under 1 ms
Bank minus a desk · 50K-position scope	2.6 ms · 52 ms
Whole-bank filter, 1M trades	58 ms

Mix: 80% trades, 10% counterparty-trade pages, 5% netting sets, 5% searches.

Estimated for the cluster 8 servers, each answers any scope

Layer	Capacity	Basis
Servers[2]	~650 req/s peak, ~330 sustained	140 measured on 24 cores → ~93 a 16-vCPU, ~47 an 8-vCPU server
VaR queries[3]	thousands/s for nodes; ~10–25/s whole-bank filters	nodes from totals; a filter reads ~4.7 GB over 6 owners
Consoles (2 × 4 vCPU)[4]	~100–140 page loads/s	69 measured a console
Active users[5]	~300–500	consoles first: a page every 10 s, 3–5 calls: 0.3–0.5 req/s each
Named users[6]	~1,500–5,000	10–20% active at once

What limits it first 1 the consoles: add more behind the load balancer (no sticky sessions needed); 4 handle about double. 2 many whole-bank ad-hoc filters at once, the morning risk meeting; node and hierarchy questions are cheap. 3 live tabs and streams: each open live view holds a connection.

VaR under many users. Measured by the 200K benchmark (50 users, a realistic VaR mix, single server and sharded cluster): the evidence chapter. Then again at 2.3M on a bank server.

[1–6] How these figures were derived: A-4, in the appendix (Concurrency and throughput)

Storage and deployment

10



Delta for history and reproducibility, Drishti*net for scale-out

Partition by business_date; 7 years of time travel
full run replaces a date; restatement = new commit

Vector column list<double>, float64
standard Parquet; a sum reads one column

Compaction + checkpoints (lake maintain)
many small intraday commits stay fast

Sorted by LE and desk; ZSTD + byte-stream split
the parallel loader: shard-aligned files, a node a run of rows

Z-order or liquid clustering by book and portfolio PROPOSED
skip files for a desk's or portfolio's read

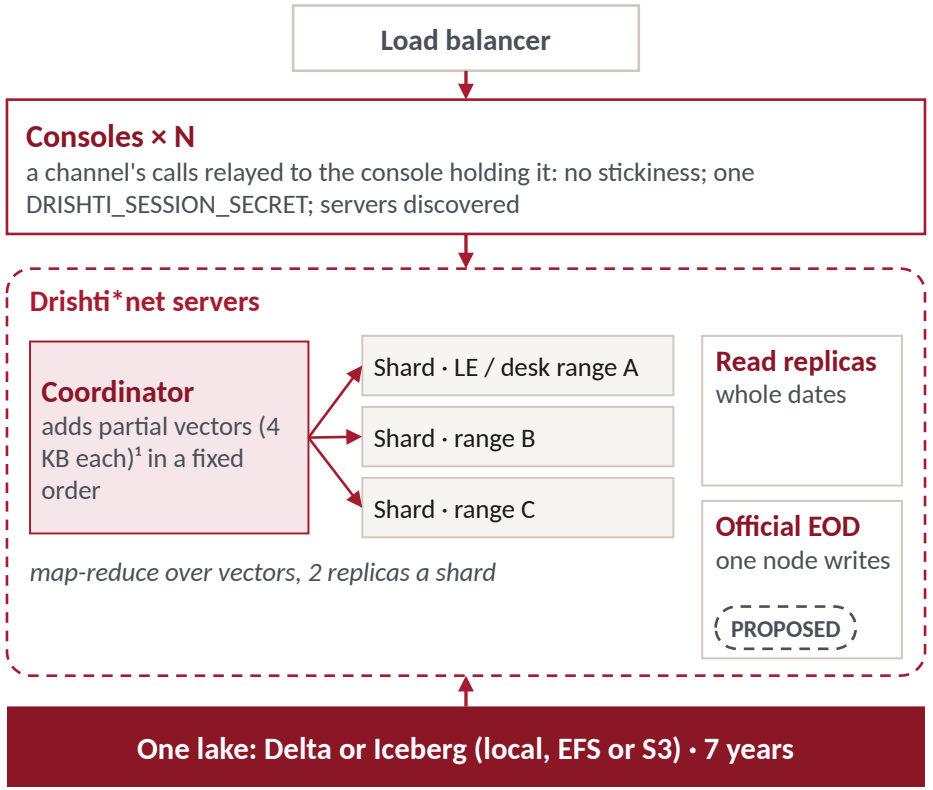
Published node and portfolio vector table PROPOSED
BI and peers read sums, not 2.3M rows

Change data feed on the input tables PROPOSED
follow intraday deltas downstream

Delta inputs on S3 (reading)
the Delta plugin reads s3:// tables

Iceberg for VaR, intraday and exposure
equal to Delta to the cent; Delta the default

Official results to Delta or Iceberg, folder or S3
append-only; BI's readers follow the format



To reach 2.3M positions and 15,000 portfolios, still to do

1 A 2.3M load on a real server
2 Clustering by book and portfolio (Z-order)
3 The 15,000 portfolio vectors built in a batch
4 The memory-mapped skeleton: less heap an owner
5 The Glue and S3 Tables runs on AWS

Built: VaR sharding, the relay across consoles, the loader, Iceberg, results to S3

[1] A partial vector: 509 scenarios × 8 B = 4,072 B a node a shard (VAR_SHARDING)

Delta Lake or Iceberg for VaR: performance and recommendation

VaR answers are format-independent. Once a date is loaded, the engine computes from its memory-mapped hot files; the lake format matters only for loading, intraday commits, history and other engines.

≈ 0.5 ms[1]
a node's VaR and ES · measured

≈ 80 ms[2]
the whole bank at 2.3M positions · estimated

Where they differ	Delta Lake	Iceberg	Edge
First load of a date	replays its log from the last checkpoint	plans from manifests	about the same: both Parquet, Drishti's own readers
Intraday micro-batches (Kafka every 5 min, ≈ 120 commits a day)	a small log entry per commit, then checkpoints	rewrites its metadata file each commit; ≈ 0.2 GB[3] by year 7 (estimated): needs snapshot and metadata management	**Delta**
Time travel and restatements	rebuilds a version from its log and checkpoints	jumps to a snapshot	**Iceberg, slightly**
Caches after another day's commit	equivalent proposed	built	**Iceberg**
Other engines (Spark, Athena, Trino, Snowflake, Databricks)	strongest with Spark and Databricks	broader multi-engine support	depends on the platform
Correctness	identical: the equality tests prove Delta = Iceberg to the cent		same

Recommendation

Delta is the default (product decision), above all with Kafka intraday. Choose Iceberg where the data platform standardises on it, or for heavy time travel and multi-engine access. Either is a configuration switch.

Built VaR, intraday and exposure on Iceberg; official results to Iceberg and S3; equal to Delta to the cent

Measured (demo)[4] 14 risk tables, 955,124 rows moved to Iceberg in 579 s

Not built yet a 2.3M benchmark on both formats; Delta ↔ Iceberg conversion

[1] VarTierBenchmark (VAR_GUIDE §27.3) [2] 4.68 GB of float32 rows at 2.3M ÷ 60 GB/s = 78 ms [3] 120 commits a day × 1,764 dates × ≈ 1 KB [4] CHANGELOG, Iceberg

Delta or Iceberg, on premises or on AWS

	Delta Lake	Iceberg
On premises	VaR inputs from Delta tables Official results: append-only Delta + JSONL Lake maintenance; 7-year floor for risk	VaR, intraday, exposure on Iceberg Official results to Iceberg Maintenance, orphans, 7-year floor
AWS	Reading Delta tables in S3 Official publish to an EFS Delta root Official results to s3://	Iceberg on S3; AWS scripts Admin → Lakes, disk caches Glue and S3 Tables: first AWS run NOT RUN

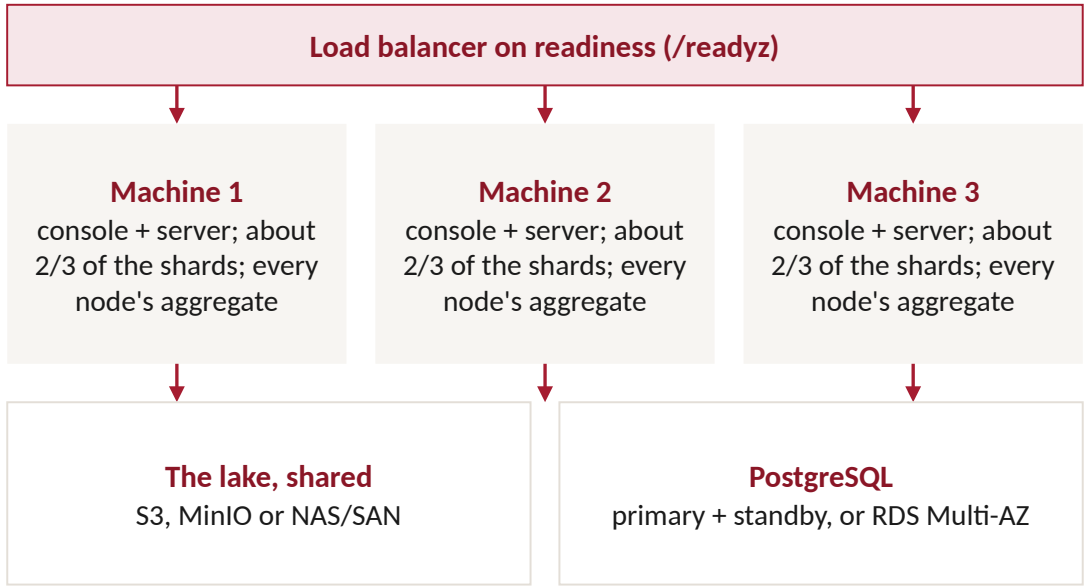
Above the storage plugins, one product packs, data contracts, connector names, engines, BI, UI and Drishti*net are the same on Delta and Iceberg; VaR and exposure are equal to the cent on both (VAR BANK USD 112,041,646[1] on the demo data). Delta is the default.

The AWS deployment and its cost are in the appendix; the companion deck Drishti on AWS (the lake, Delta or Iceberg) has the detail.

[1] VarFormatEqualityTest: the same lines written to Delta and to Iceberg give the same VaR, to the cent, on the demo data

Three machines, on premises or on AWS

The smallest highly available Drishti: three machines, the rest shared



- One fails: every shard still has a replica; the others rebuild the second copy from the lake
- Upgrades one at a time: drain, stop, upgrade, ready; two copies throughout
- No consensus protocol: a split leaves read answers identical; writes go to the shared database
- Two machines: pilots and UAT only (no spare)

A machine (the heap measured at 1M, the rest extrapolated)[1]

	On premises	AWS
Machines	3 × 16 cores, 64 GB, 500 GB NVMe	3 × m7i.4xlarge, 16 vCPU, 64 GB; gp3 500 GB
Lake	S3-compatible (MinIO) or NAS/SAN	S3
Database	PostgreSQL + standby	RDS PostgreSQL Multi-AZ
Balancer	a pair + VIP, or F5	ALB across three zones

- 64 GB carries 1-day VaR to 2M (heap 4.3 GB, -Xmx 12g; 6g at 1M, measured); every horizon: 128 GB to about 1M, 192 GB at 2M
- The lake is small: about 1–3 TB for several years at 1–2M positions (1-day)

AWS, 1-day VaR to 2M, a month (Canada (Central))[2]

On demand: USD 3,300–4,000[3]

1-year Savings Plan: USD 2,800–3,500[4]

3-year Savings Plan: USD 2,300–3,000

Every horizon to 1M (3 × m7i.8xlarge): USD 5,400–6,100 on demand. Check every price in the AWS Pricing Calculator.

[1]–[4] How these figures were derived: A-6, in the appendix (Three machines)

Drishti Elements

11

Every screen drawn from one set of JavaScript libraries

Library	What it holds
drishti-core	formats, the theme, the API client, dialogs, the filter model
drishti-inputs	date and range pickers on the bank's calendars, select, multi-select, combobox, forms from a schema, the code editor
drishti-grid	the grid with pivot, cell editing and write-back; an AG Grid-style adapter
drishti-chart	every chart kind; maps from the world to every country's first-level regions
drishti-data	data sources, live and Arrow clients
drishti-tree · drishti-graph	hierarchies; networks of linked records
drishti-insight	Explain, search, Approvals
drishti-designer	the designer kit: canvas, palette, inspector, outline, live YAML, the designer shell
drishti-renderer	Sutras drawn anywhere, inside the console or another application
drishti-risk	the banking module's views; served only where the module is licensed

One release train

Eleven libraries, each with its own version, released together; the gallery (/elements) shows every component live, and the API reference is generated from the sources.

Every screen, none bespoke

Every page of the console is drawn with them. Guard tests fail the build when a page draws its own select, form, chart or designer: each guard stands at zero.

Fewer lines, more product

The screens' own code fell from 81,053 to 79,764 lines while every new screen of the train was added; the libraries hold 41,295 lines in 186 ES modules.

Eighteen parallel implementations folded into one, the old ones deleted

Search and keys
one search (Alt+K) over entities, commands, screens and Help; one keymap, ? lists the keys

Approvals
every four-eyes decision in one queue: Sutras, record edits, BI, compute, functions, sign-offs, settings

Alerts
rules on one entity and on measures: one engine, one stream

Explain
hover, click or E on any figure: formula, inputs and lineage, why it moved, the workbook

Build home
everything people build in one rail, with what each item uses and what uses it

Sitemap
one file draws the menus, the Admin tabs, the BI rail, Help's sections and the footer

Charts
one chart engine; Vega-Lite visuals translated, Vega retired

Grid, tree, graph, form, select
one of each, from the libraries

The exposure what-if
computed in Java; the Python engine retired

Limits and pre-deal
one limit rule; one pre-deal service

Public data
the per-pack scripts folded into drishti-feeds

Editions
folded into lenses

Why it matters. Two ways to do one job means two places to fix, two things to learn and two answers that can disagree. Each fold deleted the old way in the same step.

The designer kit, and the libraries inside the bank, never from a CDN

The designer kit (drishti-designer)

The layout canvas, palette, inspector, outline and a YAML pane kept in step with the model; the designer shell: Visual / YAML / Both, versions, validate, preview, history, impact, propose for review.

The Sutra designer, the BI report designer, the theme designer and the table and model editors are all built from it.

The same controls everywhere

A business-date picker that reads T-1, ME or 6 Nov 2026 and names the bank's holidays; multi-select with chips and search; labels above fields, problems in words, numbers read loosely (1.5m, 250k); 44 px targets on a phone.

Why it matters. A bank's own intranet pages can show the same grid, chart, Explain or live view as Drishti, from files the bank keeps, so nothing reaches the public internet and every byte can be checked.

Form	How
Two files for one page	drishti.py elements pack --libs core,grid,chart
Release archive	drishti.py elements dist: SHA-384 integrity hashes, npm tarballs for the bank's own registry
WebJar	./mvnw -P elements -pl drishti-elements package, for the bank's Maven repository
From the server	the Drishti server you run serves them
Banking views	drishti-risk only with --include-domain banking

- Nothing is published anywhere public; there is no CDN to point at.
- The same sources make the same bytes: a build can be reproduced and compared.
- Pages that embed the earlier elements keep working, byte for byte.

Operating it

12



The release drill: from about 91 to about 77 minutes, same proof

Lane	Before	After (measured alone)	What did it
Browser	about 88 min	74 min	shared read-only servers under a write guard, snapshots split, drawn signals instead of sleeps
Containers	about 95 min	10 min	images built and booted once; health, the VaR golden through the API, four console pages
Java 21	about 43 min	29 min	Docker hidden from the copy's tests, the copy built once
Gate	3.3 min	3.3 min	unchanged

Same proof, less waiting

- the VaR goldens are the proof of the numbers; a 12-case matrix runs in about 3 s
- a fix drill reruns only the lanes the change touches
- a page walk times every page against a 10 s budget

Projected full drill

- about 77 minutes on a loaded machine, about 57 on a quiet one, against about 91 before
- measured on a shared machine; confirmed by each release's full drill

From the full run to intraday, every business date

1 Full run lands

after the close: scenario set, every live trade, hierarchies, portfolios; JSONL into the lake, then the load signal

2 Date loads

every row checked; positions derived; nodes summed; the hot file built; currencies built ahead

3 Official record

every node of every tree captured from the full run, published append-only, with JSONL

4 Intraday

changed trades or positions only, files or 5-minute Kafka windows; answers say intraday; the record stays

Monitoring, as built	What it shows
GET /api/v1/var/status	loaded dates, trades, positions, nodes, load time, rejected, excluded, batch ids, refused dates
GET .../var/intraday/streams	topic, state, waiting trades, offsets, last ten windows, newest rejections
Provenance on every answer	inputs by system, rejections by code, intraday batches applied
Admin pages	Audit, Lakes (maintenance), the connector's Apply now
drishti.py var status	the same status from the command line

What to check each morning

- The newest date is loaded, from one run (one BATCH_ID)
- Refused rows and refused dates are zero, or explained
- The official record exists; no publishError
- Each system arrived in its expected form and grain
- Limit use: amber 75%, red 90%; the backtest light
- Intraday streams: nothing waiting long; no rejections

Not checked by the engine today: the batch's due time and its completeness against the trades booked (planned in the VaR guide); agree them with the pricing-system teams.

Fewer documents, one home for every fact

172
documents under docs/, from 360

22
tutorials, one per major feature

32
guides and references

4
Knowledge pages: a glossary and three primers

Track	Tutorials, one per major feature
Start and analyse	01 the first 15 minutes · 07 pivot, cubes and Explain · 08 reports and dashboards · 09 the Notebook · 10 the AI assistant · 11 correct and approve
Risk	02 VaR · 03 exposure · 04 FRTB-SA · 05 SIMM · 06 groupings · 12 a day of bank data
World and public data	13 world data · 14 the public sector
Data and build	15 connect · 16 a lake of your own · 17 data out · 18 tables and models · 19 a pack and a screen · 20 embed
Operate	21 production · 22 a cluster of three

Every step three ways

Each tutorial step shows the screen, the command line and the YAML, on the shipped demo data.

Help by domain and lens

A Help page belongs to a module and may name its lenses: a module that is off for a person is not listed, searched or opened. Every old address still opens the page that took its subject over.

Link, don't copy

A field lives in its data contract, a record in its JSON example; guides and tutorials link both. Every contract has complete JSON examples, checked against its fields.

Checked in the gate

A register of features with their guide, tutorial, domain and lenses; counts capped; no copied record and no version history in a guide.

Ready means ready; drain before stop; upgrade one node at a time

Each node, in turn: the rolling upgrade runner does exactly this



Order: consoles → BI and data-serving → coordinators → shard owners, one zone at a time. Any failure stops the run and names the node; resume with --from.

Readiness, for routing; liveness, for restarts

Readiness	HTTP	When
`UP`	200	joined, newest date open, owned shards ready
`DOWN`	503	joining, a date opening, or a shard not ready
`OUT_OF_SERVICE`	503	drained before a stop

- Liveness (the JVM answers) drives container restarts: a node waiting for a date is never restarted for waiting
- GET /api/v1/admin/node says what readiness waits for, and whether the node is drained and safe to stop
- Each gate can be switched off: drishti.node.readiness.joining, .engines, .shards

The runner and the stop hook

- tools/rolling-upgrade.sh --inventory my-cluster.yaml --version <v> --dry-run prints every step; without --dry-run it runs them (Windows: rolling-upgrade.ps1)
- The inventory (deploy/rolling.example.yaml): platform vm or ecs, the waits (drain 600 s, ready 1,800 s), zones, the nodes' real addresses
- The stop hook drains a node on every stop (systemd, Docker, ECS); Admin → Cluster has Drain for a stop
- The drain needs an administrator's token with the node:drain scope

Every server measures itself and raises its own alerts

Platform metrics, every 15 seconds

What	Prometheus names (examples)
Caches, the memory budget	`drishti_cache_bytes`, `drishti_memory_budget_*`
Disk folders and free space	`drishti_disk_folder_bytes`, `drishti_disk_free_bytes`
Sources and feeds	`drishti_source_up`, `drishti_feeds_attention`
Loads	`drishti_loads_last_ready_age_seconds`
Pages and alerts	`drishti_view_seconds`, `drishti_alerts_open`

- Exported on /actuator/prometheus with the JVM's and the HTTP server's own
- The CIO lens reads the same names
- Admin → Health shows them, with the alerts open now

With or without Prometheus: each server evaluates the rules itself; deploy/prometheus/drishti-alerts.yaml is the same rules in PromQL for a site's own Alertmanager.

The alert rules as shipped (config/alerts/platform-alerts.yaml)

Rule	Opens when	Severity
`cache-near-limit`	above 95% of its limit, an hour	warn
`heap-after-gc-high`	heap after GC above 85%, 10 min	warn
`direct-memory-high`	direct memory above 80%	critical
`disk-free-under-guard`	free disk under the guard	critical
`source-down`	a source down for 5 min	critical
`feeds-need-attention`	a load late, missing or failed	warn
`views-slow`	pages above 150 ms at p99, 5 min	warn
`member-suspect`	a member silent for 2 min	warn

- When one opens: logged, counted, and every administrator told in the inbox; listed on Health until it closes
- Rules are configuration: a metric, a threshold, for how long, a severity and a summary line; read again when changed

Tidy by design: scopes, one settings document, archives, API references

Admin → Ad-hoc scopes

every person's saved VaR baskets in one list

- Owners delete their own; an administrator may delete anyone's, with an optional reason
- Every delete audited (var-basket-delete: who, whose, the reason); the owner told in the inbox
- Also from the command line and the API (drishti.py var basket-delete)

One configuration document

docs/admin/CONFIGURATION.md, and Help → Configuration

- Every setting of every kind: key, type, default, environment variable, meaning, restart or live
- Generated from the files themselves and checked on every build: never out of date
- Administrators see the value in effect beside each default, and where it came from; secrets never shown

Loaders read archives

zip, tar.gz, tgz and tar, read in place, local or s3://

- JSON Lines members, plain or compressed, read in name order; other files skipped with a note
- No folders: one dataset named after the file; one level of folders: a dataset per folder
- One rule in the Python and the Java loaders, tested on the same archives

API references in the console

Help → Developer

- The REST API (OpenAPI), the renderer's TypeScript API
- Java and Python references generated from the code: plugin and connector SPI, shard machinery, compute SDK, drishti.py and its client
- The build fails on an undocumented public member

Each engine on or off per server; off means gone everywhere

Engine	Command or place	As shipped	Needs
VaR	VAR, SMAT, PLAY VAR, regulatory VaR	on	—
Exposure	EXP	on	—
Sensitivities	the sensitivities API, risk factors	on	—
FRTB-SA	FRSA, PLAY FRTB	on	VaR, sensitivities
SIMM	SIMM, PLAY SIMM	on	sensitivities
Pivot	PIVOT, Views → Pivot	on	—
MR cube	MCUBE	on	VaR, pivot
Data inbox	Admin → Data loads → Inbox	off	—
In-memory lake	memory-only tables	off	—
Arrow Flight	Admin → Data out	off	—
Linear algebra: OpenBLAS	many-scope sums	off	—

One file per engine

config/engines/<engine>.yaml, switched from Admin → Engines or the command line; written, audited and kept in History.

Off is gone everywhere

Commands, menus, Help, pages, playgrounds and the API; dependencies checked first; in a cluster a request goes to a node that runs the engine.

Busy protection

When memory is short, new heavy work (a new date, a download, a BI run, a bulk load) waits politely and retries; what is open keeps working. An operator may override, with a reason.

Every number reproducible, every day backtested

Reproducibility, as built

Versions on every answer
inputs' data versions, batch ids, scenario set, method, engine version

Digests
the position rules' text and digest, the membership hash

Append-only official records
a restatement adds a result version; the earlier is kept

7-year retention
vectors with time travel, captures and records; maintenance never goes below

Recomputable
the same vectors and engine version give the same number years later; a capture keeps the vector

Exact arithmetic
a node's figures equal the lake's to the bit; components add up to VaR

Backtesting at 99% losses bigger than VaR are breaches

Green 0–4	Amber 5–9	Red 10+

Kupiec test too many or too few breaches?

Christoffersen test do breaches cluster?

Official mode a node's official history: each day's hypothetical P&L against the day before's VaR

Replay today's members over past scenario days: in-sample, labelled not a regulatory backtest

Demo, replayed[1] FX desk: 3 breaches in 249 days, green; the bank: 5, amber

Short histories limits scale with the days; not a regulatory zone before 250

Not built, to disclose age-weighted and filtered methods, ES backtests, the regulatory backtest. The hypothetical P&L is the engine's own; a bank's own hypothetical P&L table replacing it is planned.

Kupiec test (Paul Kupiec, 1995): is the number of VaR breaches (losses bigger than VaR) consistent with 99%, about 2–3 in 250 days?
Christoffersen test (Peter Christoffersen, 1998): do breaches cluster together, a sign the model reacts too slowly when markets turn?
Basel traffic light: breaches in 250 days at 99%: 0–4 green, 5–9 amber, 10+ red.

[1] Replayed on the demo's synthetic data, 249 days (VAR_GUIDE §7); zones by the Basel traffic light for 250 days at 99%

Independent checks, entitlements, audit, change control

Independent recomputation

PROPOSED

- A checker recomputes the VaR and ES of the bank and every desk from the files sent, and compares with the engine
- Exits non-zero on any difference: fit for a daily job
- The official record names its inputs' versions and batch ids: a recomputation reads exactly that data

Entitlements

- Per role and node: positions, or aggregate only (drishti.var.access)
- A scope is computed over the trades you may see; the rest counted
- Masks apply; answers cached per view, never shared across rights
- Saving baskets and captures needs the token scope risk:write

Audit

- Every headline, capture, basket change, reload and official run (Admin → Audit)
- A headline's entry: date, data version, view, VaR, ES, trades, hidden, members' digest, time
- Reloads and official captures: administrators only; official numbers kept 7 years

Change control of settings

PROPOSED

- Window, confidence, rank rule, currencies, pins: in application.yaml, shown by GET /status
- Every answer is labelled with its confidence and window; per-request windows are never official
- Pins read at start: apply from the next date loaded; provenance records them

PROPOSED marks the practice: a daily independent job, and comparing a setting change per request before making it; the mechanisms are built.

Nothing unbuilt looks built: one registry, one mechanism

One registry

config/under-construction.yaml lists everything that depends on unbuilt work: what it is, where it shows, and whether it is hidden or marked.

The server rewrites a page's Sutra when it matches, so pages, phone lines, pivots and the client renderer read the same answer; the console reads the same file for menus and Help.

What a person sees

“Under construction” in place of the figure, or nothing at all. The data stays in the lake; only what is displayed changes.

Checked

A generated document (UNDER_CONSTRUCTION.md), checked by a test on both sides: every place it names exists.

Registered today	
SA-CCR figures EAD, replacement cost, add-ons, multiplier, RWA: stored, not computed	NOT BUILT
CVA figures CVA, DVA, FVA, KVA, CS01 and their parts: stored, not computed	NOT BUILT
Credit-limit usage the used amount and utilisation as stored, not measured by the exposure engine	NOT BUILT
Counterparty net MTM, stored the computed one, from the trade valuations, stays beside it	NOT BUILT
Pre-deal: SA-CCR and EAD limits SA-CCR's incremental EAD, soft limits, the maximum size (the credit-limit decision is built)	NOT BUILT
P&L explain waterfall the stored steps; the computed explain is built	NOT BUILT
The CCR cube a Pivot preset, until the counterparty engines compute its figures	NOT BUILT
Wrong-way risk, collateral, margin calls their contracts, planned	NOT BUILT
Arrow Flight on three nodes, end to end not measured yet; single-node reads and streams are run	NOT RUN

Modules, lenses and access

13



The core knows no industry: each industry is a module

Licence which modules a customer may run → **Module** installed, licensed and enabled → **Lens** the experience a person works in → **Function** a command word, the finest unit of access → **Permission** roles, row rules and masks inside

Module	What it brings
core	connectors, lakes, Sutras, search, the Pivot, BI, tables and models, approvals, alerts, Explain, Admin; GLEIF's legal entities
banking	VaR and ES, exposure and the pre-deal check, sensitivities, FRTB-SA, SIMM, official results; drishti-risk
capital markets	US equities (EDGAR filings and facts, indices), funds and ETFs, the market data's functions
economy	economies, forecasts, governments, elections, news, and the public sector

Off is gone

A module that is off loads no engine, job, pack or API; its pages, menus, commands, type-ahead, Help pages and Build items are gone for everyone. Switched in config/modules/<module>.yaml or the admin API, audited.

One trust point

Every layer asks the server, which asks the registry: the console and the browser never decide on their own.

Nothing changes on upgrade

Without an installed licence every module is on, so an existing installation keeps everything it had.

A lens is the view a person works in, and only that

Module	Lenses	For
banking	Risk · Trading · Market · CRO · Market-risk cube · Counterparty-risk cube	risk managers, traders, the CRO's desk, the two risk cubes
economy	World · Public sector	economists, public-sector analysts
core	CIO · Oversight	the technology owner, audit and oversight

A file per lens

config/lenses/<module>/<lens>.yaml: the modules it needs, the functions it bundles, its menus, Home cards and chips. Offered only when every module it needs is on.

Functions, one by one

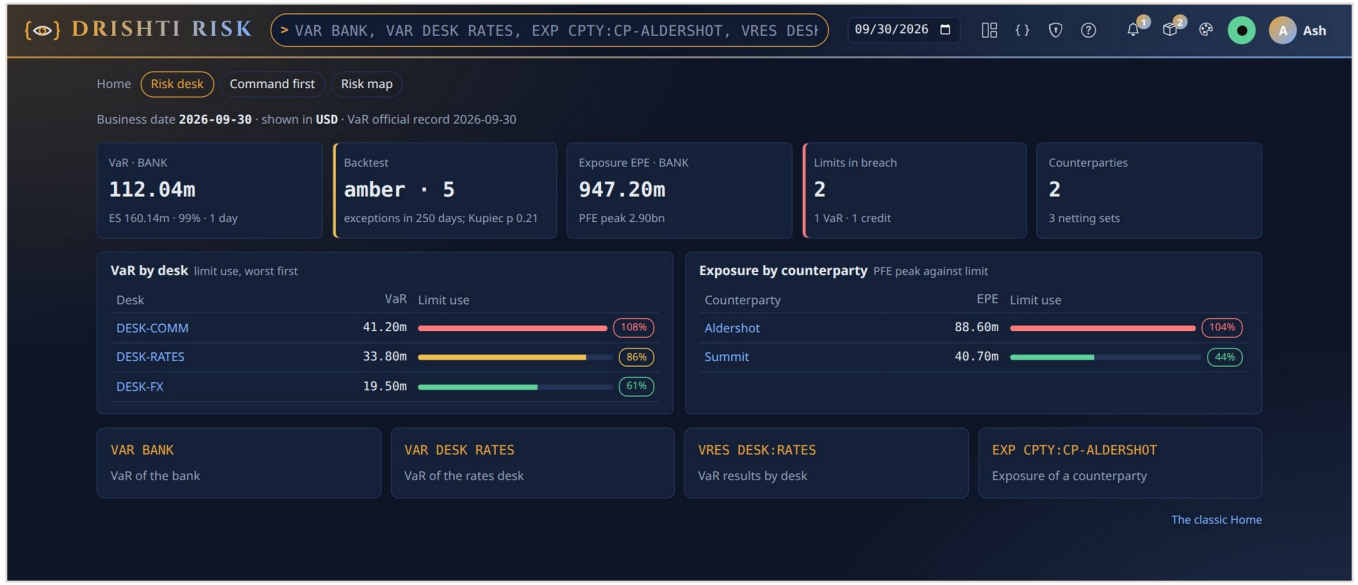
Each command word belongs to one module; roles are granted or denied single functions on top of a lens. Typed anyway, a refused function answers in words, and the refusal is audited.

Editions are lenses

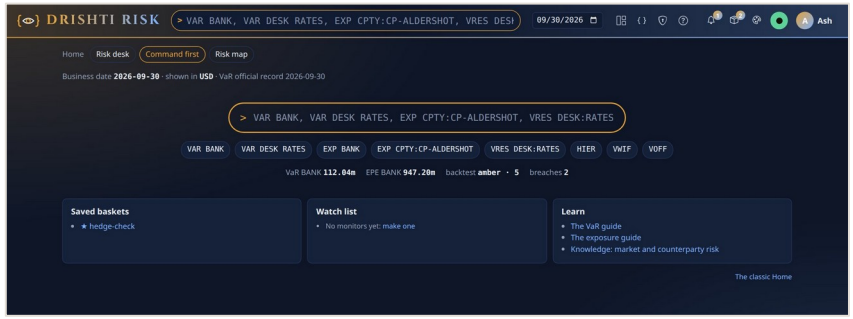
drishti.edition still runs a whole instance as one lens (for example risk); drishti.lenses offers several. A site's own edition file moves to config/lenses/<module>/.

Why it matters. Each person sees the work they do, and only what they may use: a trader's lens has no CRO board, a public-sector analyst never meets a VaR limit. Each person's modules, lenses and refused functions are one API call (GET /api/v1/modules).

Drishti Risk's three landings: desk, command and map



A · Risk desk (the default): VaR, backtest, exposure, limits in breach; desks and counterparties



B · Command first: the command line, chips, baskets



C · Risk map: a hierarchy with VaR and limit use, a heat map, Today

Every figure is the engines' own answer

- The landings read the console's VaR and exposure routes (scope, tree, intraday streams, official records, baskets); no new store, no copy of a figure
- The choice of landing is kept per person; a role can be given its landing (home.landing-by-role)
- Pictures from the browser tests' stand-in server: VaR 112.04m and EPE 947.20m are the demo bank's 2026-09-30 figures; the desk and counterparty rows are stand-ins

Drishti Market: the market data that feeds VaR

Why a VaR user cares. VaR is read from scenario P&L that the pricing systems build from the bank's curves, vol surfaces and fixings. When that market data is stale or broken, so is the VaR built on it. Drishti Market shows that data, and checks it.

Asset class	Commands	What it holds
Rates	`CRV · REPO · IRV · CPV · FIX · INF · INFC`	OIS and projection curves, repo curves, swaption and cap vols, fixings, inflation
FX	`FX · FXF · FXV`	spot, forward curves, vol surfaces
Credit and bonds	`CDS · BND`	credit curves and spreads, bonds
Equity and commodities	`EQ · EQX · DIV · EQV · CMD · CMDC · CMDV`	equities, indices, dividends, equity vols; commodities, their curves and vols
Correlation	`CORR · CORRC · EQB`	correlations and baskets

One file: config/lenses/banking/market.yaml

- the market-data pack and core (issuers, holiday calendars), allow-only: its 22 commands, nothing else
- no trades and no VaR: VAR, EXP and the risk and trade commands are hidden
- Home: Market desk (the default), Explorer, Quality
- the engines are unchanged: nothing in them moved for the edition

For a VaR user: a lens beside Drishti Risk

- On a full instance, drishti.lenses: risk,world,market offers Open Drishti Market next to Open Drishti Risk; inside one, Switch to … opens the other in one step, no restart
- A lens hides; it does not secure: what a person may read stays with roles, entitlements and masks

Market data quality: the checks behind VaR's inputs

Checked on the business date, worst first; each item opens its command. Tiles: Critical, Warnings, Clean (the share of items with no exception); a Checks run card gives each check's items and flags.

1 · Stale *warning*

- a curve's AS_OF, or the date its source holds it for, older than the business date by more than quality.stale-days (2 business days; weekends and holidays skipped)
- curves and surfaces one by one; every other kind by its latest load

2 · Gaps: missing tenors *warning*

- a curve missing a tenor that every other curve of its type has (a type of three or more)
- IR curves by CURVE_TYPE; repo, credit, FX forward and inflation curves by kind

3 · Discount factors and forwards *critical · warning*

- critical: a discount factor that does not fall with maturity, or is outside (0, 1]
- warning: a negative forward rate in FORWARDS

4 · Vol surfaces: sanity *critical*

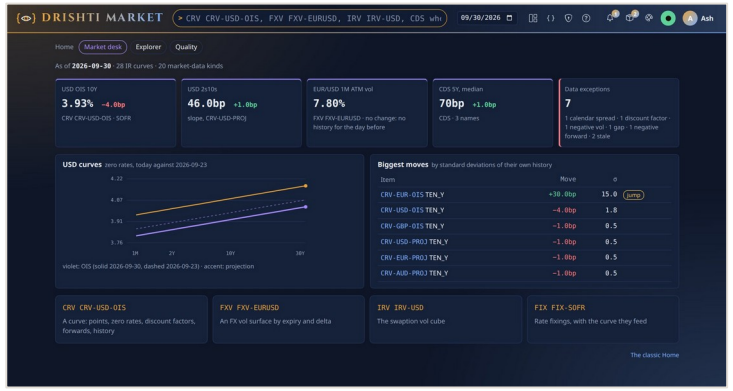
- a vol at or below zero (risk reversals and butterflies may be negative)
- ATM total variance ($vol^2 \times T$) falling with expiry, on the FX and equity surfaces

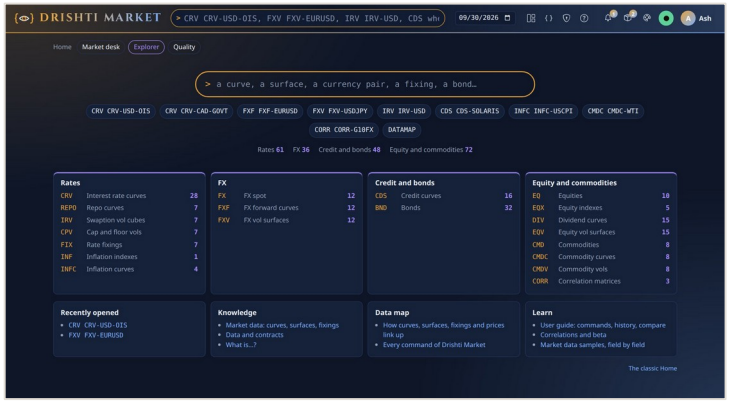
PLANNED

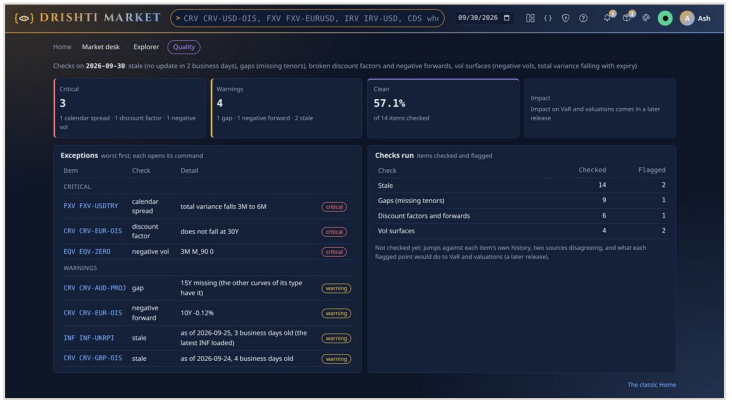
Impact on VaR

- who uses each flagged point and the VaR at stake; jumps over n σ as a quality check; two sources disagreeing (the lake holds one source per kind today)
- Today the Quality landing says so in place of the figure: impact on VaR and valuations comes later

Drishti Market's three landings: desk, explorer and quality







A · Market desk (the default)

- tiles: USD OIS 10Y, USD 2s10s, EUR/USD 1M ATM vol, CDS 5Y median, data exceptions
- USD curves, today against a week before
- biggest moves over 21 business days, in σ of each item's own moves

B · Explorer

- a big command line and the edition's chips
- counts by asset class: rates, FX, credit and bonds, equity and commodities
- the commands by asset class; recently opened; Knowledge, Data map, Learn

C · Quality

- the four checks, worst first: stale, gaps, discount factors and forwards, vol surfaces
- Critical, Warnings, Clean; the Checks run card
- the impact on VaR: a note, not a figure (planned)

Every figure is the pack's own answer. Worked out in the console from the market data's search, documents and history; read-only, cached two minutes per person. A day-over-day change only from real data on both days; otherwise the tile says so.

Pictures from the browser tests' stand-in server, with known problems seeded on purpose; the demo data's checks are all clean.

Drishti World: the macro context for risk

Why a risk user cares. A VaR or exposure move has a story outside the bank: an economy's growth, inflation or debt, the day's news, a company's filings, an entity's owners, a fund's holdings. Drishti World puts that public context one command away.

Pack	Commands	The context it gives
Economies	`ECON · MACRO · FCST · LABR · FISC · ECG · REGE · JUR · PARTY · OFFC · ELEC · REGN · SOCI`	economies and regions: growth, inflation, debt, labour, fiscal; jurisdictions, parties, elections
News and markets	`TOPIC · NEWS · CO · QUOTE`	news by topic; companies; a market snapshot of quotes
Companies	`USEQ · IDX · SIC · EDGAR · FILING · FACT · PXH`	US companies, indices, sectors, SEC filings and facts, price history
Legal entities	`LEI · LEIG · LEIR · LEIC · LEIM`	GLEIF: entities, groups, parents and children
Funds	`FUND · HELD · TRACK`	funds and ETFs, what they hold, the index each tracks

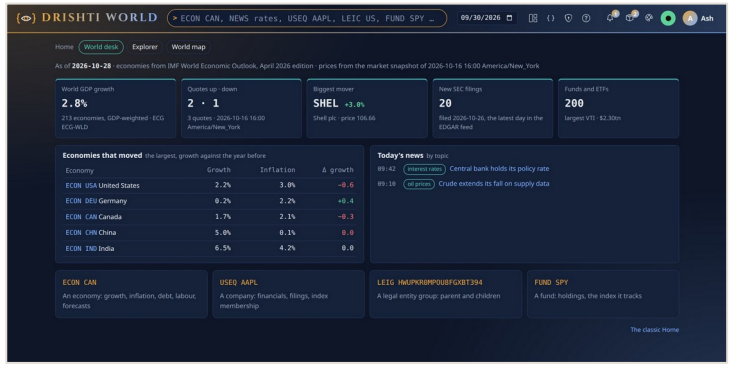
One file: config/lenses/economy/world.yaml

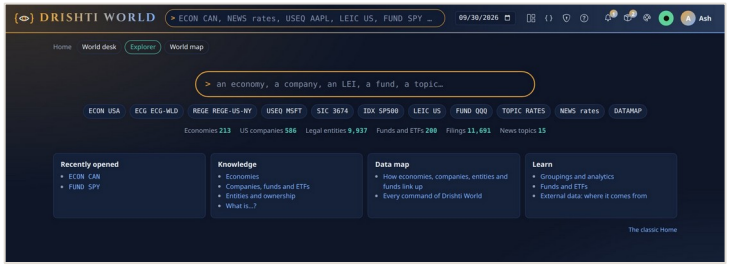
- the public-data packs, allow-only: world-economy, news-and-markets, us-equities, gleif, funds
- their 32 commands; VAR, EXP and every risk command hidden
- Home: World desk (the default), Explorer, World map
- a lens beside Drishti Risk: Switch to … in one step

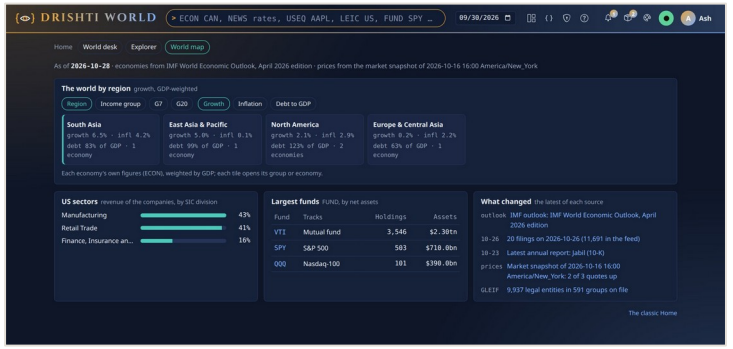
Public data, inside the bank

- The IMF figures and the market prices in these packs are shown inside the bank; showing Drishti World outside it needs the product owner's decision first

Drishti World's three landings: desk, explorer and map







A · World desk (the default)

- tiles: World GDP growth, quotes up and down, the biggest mover, new SEC filings, funds and ETFs
- economies that moved: the twelve largest
- today's news by topic

B · Explorer

- a big command line and the edition's chips
- counts: economies, US companies, legal entities, funds and ETFs, filings, news topics
- recently opened; Knowledge, Data map, Learn

C · World map

- by region, income group, G7 or G20; by growth, inflation or debt to GDP, GDP-weighted
- US sectors; the largest funds
- what changed: each source's latest

How it reads. One thin read-only route and the server's own search grammar (ECON where GDP_USD_BN > 0 order by GDP_USD_BN desc limit 12); a command the person's edition or lens does not show is refused (DRS-4023). Each panel paints when its answer arrives.

A teal accent, so World reads apart from Risk at a glance. Pictures from the browser tests' stand-in server.

Regulatory reporting

14

Canada and the US: the reports that need VaR or ES

Today. Canada: FRTB since November 2023; every bank reports the standardised approach, internal models need OSFI's approval. US: VaR and stressed VaR (Basel 2.5) in force; an ES-based re-proposal of March 2026 is not final.

Canada (OSFI)	How often	VaR or ES it needs
BCAR 90.010	Quarterly	standardised for all; ES 97.5% and SES, latest and 60-day, on internal models
Pillar 3 MR1, MR2	Quarterly	MR2: ES, SES, DRC, latest, average, high, low; exceptions against 99% VaR
Internal-model tests	Daily	1-day VaR 99% and 97.5% against actual and hypothetical P&L; the PLA test
Annual report	Yearly	trading VaR, 1 day 99%, by risk type: year-end, average, high, low
ICAAP, stress tests	Yearly	no prescribed VaR or ES

US (Fed, OCC, FDIC)	How often	VaR or ES it needs
Market risk rule	Daily	VaR 99% 10-day and stressed VaR; 60-day and 12-week averages × 3 to 4
FFIEC 102	Quarterly	VaR and stressed VaR, latest and averages; public
§217.212	Quarterly	high, low, mean, period-end VaR and stressed VaR; by risk type; backtest
Volcker metrics	Quarterly	per desk, daily: limits and use, VaR 1-day 99%, P&L attribution, positions, volumes
FR Y-9C, FR Y-14Q	Quarterly	market-risk RWA (HC-R item 27); the market shock: not VaR

A Canadian bank with a US holding company. It files both sides: the US holding company follows the US capital rule like a US bank holding company; FFIEC 102 applies at $1 billion of trading assets and liabilities, Volcker at $20 billion across the bank's US operations.

The building blocks, and what Drishti has today

Building block	Needed by	Status	Drishti today
VaR 99% and ES 97.5%, 1 day	MD&A, Volcker, every backtest		the bank to a trade; any node or trade set
By desk and legal entity	all; diversification in most		every hierarchy node, with diversification
VaR limits and their use	Volcker		a limit per node; use in every official record
Kept and reproducible	all		official runs of every node, 7 years, versions
Backtest: hypothetical P&L	all		every node: exceptions, traffic light, tests
Backtest: actual P&L, 97.5%	US rule, OSFI internal models		built; reads the bank's daily P&L feed
10-day VaR, stressed VaR	US rule, FFIEC 102, §217.212		√10 or the sum; stressed: SVAR, 12-week average
High, low, average, 60-day	MD&A, FFIEC 102, §217.212		period statistics from the official records
VaR by risk type	MD&A, §217.212, FFIEC 102		built; reads a risk-type hierarchy from the bank
Other limits, P&L attribution	Volcker	NOT BUILT	P&L explain designed, not built
Standardised approach	BCAR 90.010, Pillar 3 MR1		FRTB-SA for any scope, from sensitivities
ES by liquidity horizon, PLA	banks on internal models	NOT BUILT	with NMRF and DRC

Reading it. A Canadian bank on the standardised approach reports VaR in its annual report; its US holding company needs the full VaR and stressed VaR set. Built rows carry no pill.

From the official run to the filing, reproducible as of a date

Inputs	Official run	Computed measures	Report layer	Filing
▪ trades, P&L vectors, scenario sets	▪ end of day, every node	▪ VaR, ES, backtest, diversification	▪ BI reports and paginated reports	▪ the bank's own filing channels
▪ hierarchies, limits	▪ inputs' versions and batch ids	▪ 10-day and stressed VaR	▪ extracts: Excel, CSV, PDF	▪ the bank's review and approval
▪ daily P&L from product control	▪ a restatement is a new version	▪ period statistics, averages	▪ the published official table	▪ Drishti does not file

Example: a quarter's US VaR figures

- the official records of the quarter's business days, each node's VaR at its own result version
- 10-day VaR: the latest and the 60-day average × the multiplier from 250 days of exceptions
- stressed VaR: the latest and the 12-week average; high, low, mean and period end by risk type
- laid out as the report's table; exported for the bank's filing

Reproducible as of a date

- each record names its inputs' versions, batch ids and position rules
- the members of every node kept with the record
- append-only, 7 years, Delta or Iceberg
- recomputed from the same vectors and engine: the same number

Roadmap and rollout

15

ROADMAP

What is built, and what comes next

Built · unlabelled

Any input: PVs or P&L, trade or position
files and Kafka; intraday as new − old; mixed per system

1-day, 10-day and stressed VaR; parametric beside it
√10 or the sum; SVAR; SMAT; why VaR moved; the VaR Playground

Any currency; official records, 7 years
every hierarchy; Delta or Iceberg, folder or S3

Totals over every position; sharded cluster
1,000,000 positions in a 6 GB heap; many scopes at once; engines per server

Regulatory VaR; FRTB-SA, SIMM; the MR cube
backtests, sign-off, the stress window; each measure with its playground

The pre-deal check; one limit rule
approve, refer or decline from the exposure paths; within, near, breach everywhere

BI on one table layer; Pivot → Report
the editors; the Notebook; one Python runtime on the server

Drishti Elements; one way per job
every screen on eleven libraries; one search, Approvals, Explain, Build home

Domain modules, lenses, the access registry
banking, capital markets, economy; ten lenses; functions one by one

A lake of your own; feeds; the docs rewritten
thirteen feeds, the public sector; one guide and one tutorial per feature

Not built yet

Trust, RBAC and controls — PLANNED
security on by default; data and module licensing; one breach queue

Memory and speed — PLANNED
the full drill under an hour; measured everywhere, then fixed

Counterparty engines and stress — PLANNED
SA-CCR in pre-deal; the CCR cube; a stress engine

Collateral and margin — PLANNED
valuation and eligibility; margin calls; wrong-way flags

CVA, then hardening — PLANNED
BA-CVA from SA-CCR; FVA; adversarial and soak tests

Lakehouse interoperability — PLANNED
Unity Catalog, Delta Sharing, plain Delta and Iceberg tables

Onboarding: role doors — PLANNED
four role doors; the remaining guides and tutorials

Arrow Flight on three nodes, end to end — NOT RUN
single-node reads and streams are run

Glue and S3 Tables on AWS — NOT RUN
code written; first live run on an account

PROPOSED **At scale:** lake clustering by book and portfolio (Z-order or liquid clustering); a published table of node and portfolio vectors, built in an overnight batch.

Not scheduled: QuantLib; the PostgreSQL wire protocol; FRTB internal models. New domains come as modules, as the public sector did.

ROADMAP · WHAT COMES NEXT, IN ORDER

Lean & streamlined: trust and speed, then new engines

1 Next: Trust, RBAC, security and controls

PLANNED

- Security on by default, one entitlement service for every door, visibility by role; data licensing and module licensing on the clean modules
- Run status and sign-off, one breach and escalation queue, computed credit-limit usage, number honesty; the pilot kit

2 Memory and speed

- the full drill under an hour
- measured everywhere, then fixed where it matters

3 Counterparty engines and stress

- SA-CCR with incremental EAD in pre-deal
- the CCR cube; a stress engine

4 Collateral and margin

- valuation and eligibility
- margin calls and disputes; wrong-way flags

5 CVA

- BA-CVA from SA-CCR exposure
- FVA

6 Hardening

- adversarial tests at every door
- soak tests at production volumes

7 Onboarding

- four role doors
- the remaining guides and tutorials

8 More domains, as modules

- each with its packs, lens and feeds
- anti-money laundering next in line

Not scheduled: QuantLib; the PostgreSQL wire protocol; FRTB internal models, SA-CVA, full xVA. New domains come as modules, as the public sector did.

Regulatory reporting in phases: built, then later, then on approval

1 · The measures
built

Builds
- the stress window selection tool
- a daily actual and hypothetical P&L feed
- backtests: 99% and 97.5%, both P&Ls, multiplier
- period statistics; signed report runs

Unlocks
US rule figures; FFIEC 102 and §217.212 inputs; the annual report's statistics

2 · FRTB standardised
built

Builds
- sensitivities-based method
- default risk charge, residual risk add-on
- PLAY FRTB: every choice a switch

Unlocks
BCAR 90.010, Pillar 3 MR1; the US standardised approach if adopted

3 · The report packs
later

PLANNED

Builds
- VaR by risk type
- BI reports and extracts in each report's layout
- Volcker metrics per desk

Unlocks
annual report VaR table, Pillar 3 tables, FFIEC 102, §217.212, Volcker

4 · FRTB internal models
only on approval

NOT BUILT

Builds
- ES by liquidity horizon, stressed calibration
- PLA test; NMRF and SES; DRC

Unlocks
BCAR 90.010 section A; Pillar 3 MR2; model approval

Why the standardised approach first. Every Canadian bank reports the standardised approach, modelled or not; internal models serve only a bank that applies.

The bank supplies: daily actual and hypothetical P&L; 10-day and stress-window P&L vectors; P&L by risk class; desk definitions and its limit catalogue; sensitivities; real-price observations for phase 4.

A phased rollout with a parallel run

0 Prepare

- Environment: Delta or Iceberg, on premises or AWS
- Settings and entitlements agreed
- A 2.3M load on the target server

1 Pilot desk

- The desk's systems onboarded
- Daily full runs and official records
- Parallel run against the existing VaR

2 A legal entity

- All its desks and systems
- LEGAL tree and limits
- Intraday switched on

3 The bank

- Every system, every tree
- ≈ 15,000 portfolios; ≈ 700 limit letters
- BI reads the official results

4 Cutover

- Sign-off
- Drishti's records become the numbers of record
- The existing VaR retired

Parallel run and reconciliation

- Each node, each date: the existing VaR beside Drishti's official record (the var-official table)
- Explain every difference by its cause: inputs (refused, excluded, unknown ids), method (window, confidence, rank rule), FX (scenario FX, not today's rate), membership (overlaps counted once), model
- Track the backtest light on the official history as it builds up

Sign-off criteria

PROPOSED

- Agreed tolerance per node; every difference explained
- No unexplained refusals over an agreed run of days
- Model validation of the method settings and the controls
- Runbook rehearsed: a restatement, a failed publish, a restart
- Backtest light not red; entitlements and audit reviewed

A proposed plan: generic, no dates. Each phase uses built capabilities; stressed and 10-day VaR are among them.

Appendix

A

What a pricing system sends

Every record JSONL: one JSON object a line, UTF-8, .jsonl or .jsonl.gz; files or Kafka

Field	Values	Rule
`VALUE_TYPE`	PV or PNL	required
`GRAIN`	TRADE or POSITION, with its id	required
`CURRENCY`	ISO code; PVs in their own	required
`SOURCE_SYSTEM`	the pricing system that sent it	required
`SCENARIO_SET_ID`	the date's set (demo: HS-509)	all, in order
`BATCH_ID · RUN_KIND`	FULL or INTRADAY	one full run a date
`COMPUTED_AT`	UTC instant	the latest wins
`LEGAL_ENTITY · DESK · TT_BOOK`	where it sits	needed to load
`LIMIT_LETTER`	a level column of a hierarchy	absent: tree skipped
`BASE_PV · SCENARIO_PV`	the PV form	failed: null
`PNL (+ PV)`	the P&L form, USD	PV makes FX exact
`TRADE_STATUS · MATURITY_DATE`	live trades count	CANCELLED: cancel

Files

the full run, and small intraday batches; lake build, then the load signal

Kafka

JSONL text per message, one or more records; one lake batch per 5-minute window

Intraday and cancels

- RUN_KIND INTRADAY, its own BATCH_ID, a later COMPUTED_AT, the morning's scenario set
- A cancel: TRADE_STATUS CANCELLED, the vector null
- Same form and grain as the system's full run

```
# optional per-system pin (read at start)
drishti.var.inputs:
  SYS_A:   {form: pv,  grain: position}
  SYS_B:   {form: pnl, grain: trade}
  default: {form: any, grain: any}
```

A null scenario PV refuses the whole trade (DRS-5206), never zero-filled. Field names are the as-built contracts'; older files are brought to the record with drishti.py convert --set ….

A checklist for each pricing system

1. **Choose form and grain**
   PVs or P&L, by trade or by position: one form and one grain per system per date
2. **Map the record's fields**
   VALUE_TYPE, GRAIN + id, CURRENCY, SOURCE_SYSTEM; LEGAL_ENTITY, DESK, TT_BOOK; status, maturity
3. **Price over the date's scenario set**
   every scenario in the set's order; a failed one null, never zero
4. **Scenario FX for its currencies**
   send fx-scenario, or let Drishti derive it from the FX history
5. **Pick the route**
   JSONL files for the full run; files or a Kafka topic for intraday
6. **Set the optional pin**
   drishti.var.inputs: a record that disagrees is refused (DRS-5209)
7. **Recompute independently**
   from the sent files, compare bank and desk VaR and ES with the engine
8. **Clear the refusals**
   provenance lists every refused row by code; aim for none before go-live

PROPOSED

Ready when the system's records load with no refusals, the independent recomputation agrees to the cent, and How this number was built shows the system with its form and grain.

When the data's structure changes

A new field in the source data

No schema change. Each record's full JSON document is kept in the `doc` column, so a new field is readable at once; older rows simply lack it.

A new promoted column

Iceberg: in-place schema evolution by field id: add, rename, reorder, widen.

Delta: in-place add of a nullable column, old files read as empty, optional backfill from the documents. Until then, adding one to an existing table rewrites the affected dates.

On upgrade: unedited connector files follow the pack; edited ones gain its new keys and columns, edits kept.

PLANNED Delta in-place add

History is never rewritten

Time travel shows each version as it was; official records are append-only. Adding a field never invalidates the lake, and a figure of record can still be recomputed from its inputs.

```
# before: the source sends A and B
{"id":"T1", "doc":{"A":1,"B":2}, "A":1}
# after: the source adds C; nothing to migrate, C is in doc at once
{"id":"T2", "doc":{"A":3,"B":4,"C":5}, "A":3}
# C promoted to a column: new rows carry it, old files read it as empty (null)
```

Hardware plan with 39% headroom

1-day, 10-day and stressed VaR at 2.3M positions

provisioned = need ÷ 0.61

Item	No.	Need, each	Provisioned, each	Large (2×)
VaR servers (EOD, intraday, 2 UI)[1]	4	one machine type	identical, any role	5 (+ history)
vCPU	–	32	64	96
RAM	–	151 GB	256 GB	512 GB
Heap (-Xmx)	–	21 GB	36g	60g
VaR memory budget	–	14,000 MB	23,000 MB	46,000 MB
Local NVMe (tier.dir)	–	1.18 TB	2 × 1.9 TB	4 × 1.425 TB
Console hosts[2]	2	25 pages/s	4 vCPU, 8 GB, 4 workers	the same
Lake, 7 years (object store)[3]	1	48.2 TB	79 TB; 158 with DR	158; 316 with DR
Shared state (EFS or NFS)[3]	1	0.3 TB	0.5 TB	0.5 TB
Network, a date's first open[4]	–	20.9 GB, ≤ 0.4 GB/s	≥ 10 Gbit/s	25 Gbit/s
Intraday traffic[5]	–	17.5 MB a window	< 1 MB/s a server	< 2 MB/s
Kafka PV topic, 7 days[6]	–	5.5 GB a day; 39 GB	64 GB a replica	128 GB

Does one server hold a date?[7]

Yes. All four sets need 151 GB: one 256 GB server per role, 512 GB at 2×.

Sharding is needed beyond ≈ 5M positions (a date takes 4–10 min to open) or ≈ 10M (over 1 TB of RAM).

Assumptions to confirm

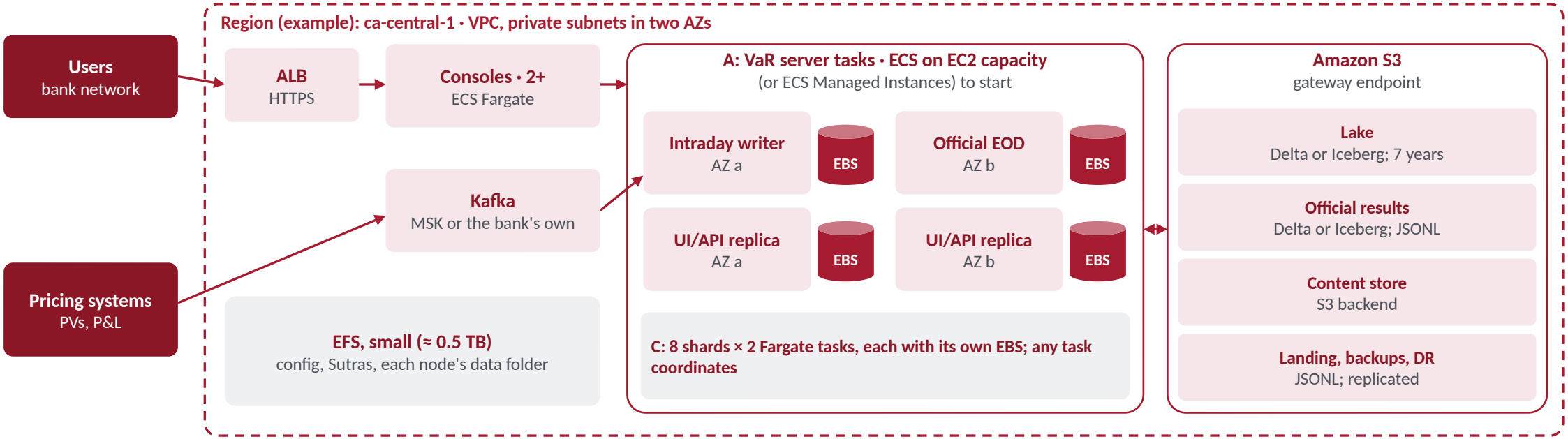
ASSUMPTION

- 50 of 200 users at once; peaks 5×
- 10-day by overlapping returns; stressed 1- and 10-day
- Intraday 230k rows a day, 120 windows
- 5,700 nodes; 2% restated; a DR copy
- Kafka 24 KB a record, kept 7 days

AWS example: m6id.16xlarge (64 vCPU, 256 GiB, 2 × 1.9 TB NVMe, 25 Gbit/s); large r6id.24xlarge; Fargate consoles; S3.
On premises: 2 × 16 cores (64 threads), 256 GB on every memory channel, 2 × 1.92 TB NVMe, 25 GbE; an S3-compatible object store.

[1–7] How these figures were derived: A-4, in the appendix (Hardware plan)

Drishti on AWS: ECS, Fargate, S3 first, as little EFS as possible



VaR tier	Shape (full case: 1-day, 10-day, stressed)	Fits, with 39% headroom[2]	Status
A · ECS on EC2 capacity[1]	4 × m7i.16xlarge (64 vCPU, 256 GiB), 2 TB EBS gp3 each; the rest on Fargate	151 GB need → 256 GB machine	
B · √10 10-day on Fargate[3],[5]	4 Fargate tasks of 32 vCPU · 244 GB (the largest), 1.1 TB gp3	94 GB → 155 GB: memory yes; CPU to measure	PLANNED
C · sharded on Fargate[4]	8 shards × 2 tasks of 8 vCPU · 44 GB, 250 GB gp3 each, one S3 lake	26 GB a task → 43 GB	

Recommendation: Fargate for the consoles and lake tasks; A for the VaR servers to start (buildable today on Delta or Iceberg); C, Fargate everywhere, with VaR sharding (built; to run on Fargate). Hot files on an EBS gp3 volume per task: disposable, rebuilt from the lake. EFS only for what must be shared.

PROPOSED

[1–5] How these figures were derived: A-5, in the appendix (AWS: the VaR tier)

AWS cost estimate for a large bank

ASSUMPTION

Monthly, year 7, 39% headroom[1]	1-day only	Full	Full, a year
VaR servers (4, EC2 capacity)[2]	$9,124	$10,498	$125,976
EBS gp3 for hot files (4)[2]	$898	$1,320	$15,840
Consoles (2, Fargate)[3]	$317	$317	$3,805
Lake loads, maintenance (Fargate)[3]	$31	$31	$369
S3: lake, results, content store[4]	$258	$752	$9,021
DR copy (S3 replication)[4]	$252	$753	$9,038
EFS and its backup[5]	$190	$190	$2,280
Amazon MSK (3 brokers)[5]	$531	$531	$6,375
Load balancer (ALB)[5]	$31	$31	$371
NAT and interface endpoints[5]	$173	$173	$2,074
Data transfer[5]	$92	$92	$1,104
CloudWatch logs and metrics[5]	$217	$217	$2,609
Total, on-demand	$12,119	**$14,910**	**$178,914**
with a 1-year Savings Plan[6]	$9,596	$12,017	$144,201
with a 3-year Savings Plan[6]	$7,443	$9,553	$114,637

Prices: AWS public on-demand list prices for the example Region, read 2026-10-27 (ASSUMPTION); quantities from the capacity slides (ESTIMATED). Verify before quoting.

The VaR tier: A, B or C[7]

servers and EBS, $ a month

Option	On-demand	3-yr plan	+ Spot
A: EC2 capacity	$11,818	$6,618	–
B: √10 on Fargate	$8,633	$5,200	–
C: sharded, 24 × 7	$7,373	–	–
C: scale-in off-hours[8]	$5,050	$2,966	**$2,785**

Honest on both sides[8]

- One full copy costs more on Fargate: 8 tasks $3,640 against one EC2 server $2,954 a month
- C wins by needing 2 copies, not 4, scaling replicas in off-hours, Spot for second replicas, growing by adding tasks
- C beats A once A needs more than 2.5 full copies; A needs 4

Recommendation

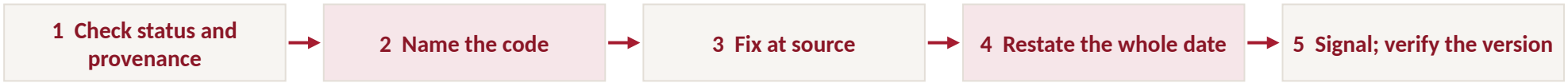
A to start; C once sharding runs on Fargate: the cheapest ($2,785 against $6,618 a month with 3-year plans) and fully ECS-native.

[1]–[8] How these figures were derived: A-5, in the appendix (AWS cost)

When something goes wrong, and the runbook

Situation	What the engine does	What the operator does
Bad rows (DRS-5201, 5206, 5207, 5208, 5209)	refuses each row, names it; never zero-fills	fix the rows, restate the date
A trade twice on a date (DRS-5202)	does not load the date	restate with one row per trade
Two runs or sets on a date (DRS-5203)	answers the date before, with a warning	restate as one run; signal
Second form or grain from a system (DRS-5210)	refuses: never a double count	fix the system's output
A date over the memory budget (DRS-5204)	503; never loads part of a date	raise memory-budget-mb or heap
Restatement	reloads; next result version; old kept	rewrite the whole date, new BATCH_ID
Publish fails (disk, root)	records publishError; capture stands	fix the root; capture again
Damaged or old hot file	reads the lake, rebuilds the file	nothing
Restart, or a Kafka replay	rebuilds from the lake; writes nothing twice	nothing

Runbook



The engine never guesses: a bad row is refused and named; a date that cannot load as one run is not loaded, and the day before is answered with a warning rather than a wrong number.

GLOSSARY

The terms used in this deck

Term	Meaning
VaR	the loss not exceeded at the confidence: 99% = minus the 6th worst of 509
ES	expected shortfall: minus the mean of the 13 worst P&Ls at 97.5%
Scenario	a historical day's market move, applied to today's trades
Scenario set	the scenario days of a date (HS-509), oldest first
Scenario PV	a trade's or position's value under one scenario, in its own currency
Path P&L	the P&L vector: one P&L per scenario (509 numbers)
Form · grain	PV or P&L · one record a trade, or one a position
Scope	any set of trades or positions: bank, LE, desk, book, portfolio, rule, basket
Node	a point of a hierarchy; its vector precomputed
Limit letter	a node of LL, a hierarchy by columns; limit from var-limit
Position	trades grouped by the pack's rules, or sent whole
Portfolio	a stored scope: HIERARCHY, RULE or BASKET

Term	Meaning
Component VaR	a part's share of VaR (Euler); parts add up to it
Incremental VaR	VaR(scope) − VaR(scope without the part)
Marginal VaR	component VaR per million of size
Diversification	the sum of standalone VaRs minus the VaR
Scenario FX	each scenario's exchange rates, used to convert
Reporting currency	the currency an answer is in; five built ahead
Hot file	a date's vectors, memory-mapped from local disk
Official record	a node's end-of-day VaR and ES, append-only
Result version	1, 2, 3 … as a date is restated
Provenance	how a number was built: inputs, versions, refusals
Breach (exception)	a day whose loss exceeds the VaR tested against
Headroom 39%	provisioned = need ÷ 0.61

How the numbers were derived

How it scales

[1] A row of 509 scenarios: 509 × 8 B = 4,072 B in float64 (the lake, the sums), 509 × 4 B = 2,036 B in float32 (the hot file's rows) (VAR_GUIDE §27.1).

[2] 21.4 MB a reporting currency a date: measured on the demo lake, 8,566 trades (VAR_GUIDE §15.11).

[3] Measured on one workstation (24 threads, 61 GB, NVMe): VarTierBenchmark, 100,000 trades × 5 dates (VAR_GUIDE §27.3); VarScaleBenchmark, 1,000,000 trades in memory (§27.2); the demo date's load, 9,572 trades, on the server (§26.7). 60 GB/s = 203.6 MB of float32 rows in 3.4 ms; 70 GB/s = 1M × 509 × 8 B in 58 ms.

How it scales: the algorithms

[1] A scope's set: one bit a trade, N ÷ 8 bytes: 200,000 ÷ 8 = 25,000 B ≈ 25 KB; 2.3M ÷ 8 = 287,500 B ≈ 290 KB (VAR_GUIDE §27a.1). The standalone cache: 8 B a position and lens: 200,000 × 8 = 1.6 MB; 2.3M × 8 = 18.4 MB ≈ 18 MB (§27a.3).

[2] Is a position whole: a scan of its own bits, (to − from) ÷ 64 machine words (a position of up to 64 trades: one word); a scope: one pass over its positions. A standalone VaR keeps the k worst of S values, O(S × k): k = ⌈509 × 0.01⌉ = 6 at 99% (§27a.1, §27a.3).

[3] Measured, one user, p50 (BENCHMARK.md §4.5): a node 3 ms at 200,000 and 4 ms at 1,000,000 positions, its aggregate whatever its size; the bank minus a desk 68 ms and 1.46 s, linear in the scope's positions while its parts are summed position by position: × 2.3 = 3.4 s at 2.3M (EXTRAPOLATED, §12.1), the walk that summing by runs removes.

Sizing

[1] Vectors: 2.3M positions (ASSUMPTION, the bank's volumes) × 509 scenarios × 8 B = 9.37 GB a date in float64; × 4 B = 4.68 GB in float32 (VAR_GUIDE §27.1).

[2] Measured per 100,000 trades by VarTierBenchmark (VAR_GUIDE §27.3) and scaled linearly × 23 (2.3M ÷ 100,000): hot file 237.7 MB → 5.47 GB; heap 80 MB an open date → 1.84 GB; first load 0.8–1.2 s → 18–28 s; reopen 23 ms → 0.5 s.

[3] Whole bank: 4.68 GB of float32 rows ÷ 60 GB/s (measured, VAR_GUIDE §27.3) = 78 ms; cold ÷ 3 GB/s (NVMe, the guide's basis) = 1.6 s. A node: 0.48 ms, measured, from its aggregate, whatever its size.

[4] Currencies: 250 MB a currency per 100,000 trades (the guide's extrapolation of 21.4 MB measured on 8,566, VAR_GUIDE §15.11) × 23 × 4 = 23 GB a date; × 5 retained dates = 115 GB; build 4.3 s × 23 = 99 s.

[5] USD hot files: 5.47 GB × 60 dates kept (drishti.var.tier.disk-dates) = 328 GB of local disk.

[6] Lake: 9.37 GB × 0.75 (Parquet, the design's estimate) × 252 business dates = 1.77 TB a year; × 7 years = 12.4 TB.

[7] Portfolios: 15,000 × 130 positions each (ASSUMPTION) = 1.95M rows × 2,036 B = 3.97 GB; ÷ 60 GB/s = 0.07 s, ÷ 5 GB/s scattered (the design's figure) = 0.8 s; stored: 15,000 × 509 × 8 B = 61 MB.

[8] Currency totals: 510 × 8 B × 5,700 nodes (5,000 MGMT and LEGAL, ASSUMPTION, + 700 limit letters) × 10 trade currencies (ASSUMPTION) = 233 MB; a changed position touches ≈ 11 nodes (three trees) × 5 books = 55 additions.

How the numbers were derived (continued)

10-day and stressed VaR

[1] Width: 509 (1-day, 2 years) + 509 (10-day overlapping: 509 − 9 = 500, sized as 509) + 250 + 250 (stressed 1-day and 10-day over 12 months, ASSUMPTION) = 1,518 values a position; ÷ 509 = × 2.98. Method (b) is the bank's to confirm.

[2] Vectors: 2.3M × 1,518 × 8 B = 27.93 GB a date (1-day 9.37); float32 half: 13.97 GB (4.68).

[3] USD hot file: 5.47 GB (237.7 MB per 100,000 measured × 23) × 2.98 = 16.30 GB.

[4] Heap of an open date: 1.84 GB (80 MB per 100,000 × 23) + its aggregates 0.20 GB (8.6 MB per 100,000 × 23, VAR_GUIDE §27.3) × (2.98 − 1) = 2.23 GB.

[5] First open: lake read 18–70 s (the guide's 10–40 s for ≈ 4 GB of Parquet, scaled to 7 GB) × 2.98 = 54–209 s; build 0.8–1.2 s per 100,000 × 23 = 18–28 s × 2.98 = 54–84 s.

[6] Other currency: 1-day built ahead 5.75 GB (250 MB per 100,000 × 23); its 10-day and stressed books on first ask (ASSUMPTION): 5.75 × (2.98 − 1) = 11.4 GB.

[7] Per server: NVMe and RAM as on the processes-and-memory derivations (458 and 1,183 GB; 76 and 151 GB); lake over 7 years as on the storage derivations (16.1 and 48.2 TB).

[8] Option (a), √10 × 1-day: width 509 + 250 = 759 (× 1.49); vectors over 7 years 12.39 TB × 1.49 = 18.5 TB. Sums: a 509-wide set 4.68 GB ÷ 60 GB/s = 78 ms; a 250-wide set 2.3 GB → 38 ms.

Processes and memory

[1] Heap: two USD dates with all four sets 2 × 2.23 + two other-currency 1-day books 2 × 1.84 (ASSUMPTION) + a load's sort 2 + the rest of the server 4 (ASSUMPTION) = 14.14 GB × 1.5 (generational ZGC) = 21.2 GB need; ÷ 0.61 = 34.8 → -Xmx36g. 1-day: 13.36 × 1.5 = 20.0 GB.

[2] Memory budget (drishti.var.memory-budget-mb, the float32 copy of one date): 2.3M × 1,518 × 4 B = 13,966 MB → 14,000 need; ÷ 0.61 = 22,894 → 23,000 MB; large 2× → 46,000.

[3] Page cache: two USD dates 2 × 16.30 + four 1-day currency books 23 + two currencies' 10-day and stressed books 2 × 11.4 (ASSUMPTION) = 78.4 GB (1-day 33.9). A load: spill 27.93 + new hot file 16.30 = 44.2 GB (1-day 14.8). OS and native 7 GB (ASSUMPTION).

[4] RAM: 21.2 + 78.4 + 44.2 + 7 = 150.9 GB need (1-day 75.8); ÷ 0.61 = 247 → 256 GB. Headroom (39%) is the deck's rule: the projected peak uses at most 61% of what is provisioned.

[5] Local NVMe: USD hot files 16.30 × 60 + 1-day currency books 23 × 5 dates + 4 × 11.4 + a load 44.2 = 1,183 GB; ÷ 0.61 = 1,940 → 2 × 1.9 TB; large 2× → 4 × 1.425 TB.

[6] CPU: 24 threads reached ≈ 60 GB/s (measured, VAR_GUIDE §27.2) + 8 for a load, currency builds, intraday and requests (ASSUMPTION) = 32; ÷ 0.61 = 52 → 64. All four sets of the bank: 78 + 78 + 38 + 38 ≈ 233 ms.

[7] Consoles: 40 pages/s a worker (measured, SCALE_BENCHMARK §8); 50 of 200 users at once, a page every 10 s, peaks 5× (ASSUMPTION) = 25 pages/s ÷ 0.61 = 41 against one host's 4 × 40 = 160.

[8] Large (2× positions): heap 36.4 ÷ 0.61 → 60g; RAM 291 ÷ 0.61 = 477 → 512 GB; vCPU 48 ÷ 0.61 → 96.

How the numbers were derived (continued)

Storage over 7 years

[1] Vectors: 27.93 GB a date × 0.75 (Parquet, the design's estimate) × 1,764 dates (7 × 252) = 36.95 TB (1-day 12.39); all four = 1-day × 2.98.

[2] Row columns: ≈ 100 B a row (ASSUMPTION) × 2.3M × 1,764 = 0.41 TB a set × 4 = 1.62 TB. Scenario PVs kept as well (ASSUMPTION, upper case): the vectors again, + 37.0 TB.

[3] Trades: 1.5 KB a trade a day (measured, SCALE_BENCHMARK §8) × 2.3M × 1,764 = 6.09 TB. Membership: 40.8 B a position (measured on the demo: 349,855 B ÷ 8,566) × 2.3M × 1,764 = 0.17 TB.

[4] Official results: 27,527 B capture + 352 B record + 321 B published row (measured on the demo) + ≈ 1 KB JSONL (ASSUMPTION) = 29.2 KB × 5,700 nodes (ASSUMPTION) × 1,764 = 0.29 TB; × 2.98 = 0.88 TB.

[5] Intraday: 230,000 positions changed a day (10%, ASSUMPTION) × 1,518 × 8 B × 0.75 = 9.1 KB → 2.09 GB a day × 1,764 × 2 (compaction keeps the old files) = 7.39 TB (1-day 2.48). Restatements: 2% of dates (ASSUMPTION) × 20.9 GB × 1,764 = 0.74 TB.

[6] Delta checkpoints: one a day ≈ 0.14 TB over 7 years (the design's estimate) × 2.98 = 0.42 TB; one every 10 commits at 120 commits a day: 4.9 TB (upper case). Scenario FX under 1 GB; Iceberg metadata, if chosen, ≈ 12 GB (the product owner's estimate).

[7] Need, base: the rows above = 48.2 TB (1-day 16.1); ÷ 0.61 = 79 TB; a DR copy doubles it: 158 TB. Upper: + PVs 37.0 + trades 6.1 + frequent checkpoints 4.5 = 95.7 TB; ÷ 0.61 = 157; DR 314.

[8] 250-day window: 0.66 × the 1,518 values (1,000 kept): float32 14.0 → 9.2 GB; vectors over 7 years 37.0 → 24.3 TB, only if the bank sends 250-day sets.

[9] Local NVMe a server: USD hot files 16.3 × 60 = 978 + currency books 161 + a load 44 = 1,183 GB; ÷ 0.61 = 1,940 → 2 × 1.9 TB. Rebuilt from the lake.

A sharded cluster

[1] Six owners, two replicas: each owner holds 2 ÷ 6 = 1/3 of what divides: page cache 78.4 × 1/3 = 26.1 GB; a load 44.2 × 1/3 = 14.7 GB; NVMe 1,183 × 1/3 = 394 GB, ÷ 0.61 = 647 → ~650 GB.

[2] The heap does not divide: every member keeps the date's skeleton for all 2.3M positions (VAR_SHARDING §13): 21.2 GB, the single server's; ÷ 0.61 = 34.8 → -Xmx36g.

[3] An owner's RAM: 21.2 + 26.1 + 14.7 + 7 = 69.1 GB need; ÷ 0.61 = 113 → 128 GB. vCPU 16 an owner (ASSUMPTION; 96 in all).

[4] Coordinators (coordinator-only) and consoles: ASSUMPTION; a coordinator keeps the skeleton without the rows (16–20 GB heap).

[5] 1-day only, 4 owners: heap (2 × 1.84 + 2 + 4) × 1.5 = 14.5 → 16 GB; cache 33.9 × 2/4 = 17.0; load 14.8 × 2/4 = 7.4; OS 7: 47.4 GB; ÷ 0.61 = 78 → 96 GB; heap ÷ 0.61 → 24g. Six owners, 1-day: 16 + 33.9/3 + 14.8/3 + 7 = 39 GB.

[6] Minimum start, 3 owners × 2 replicas (2/3 each): 21.2 + (78.4 + 44.2) × 2/3 + 7 = 110 GB; ÷ 0.61 = 180 → 192 GB; with 1 replica a third each: 69 → 128 GB.

[7] Portfolios: 15,000 × 130 positions (ASSUMPTION) = 1.95M ids × 16 B = 31 MB. The nightly official run over them, "minutes", is a judgement, not computed.

How the numbers were derived (continued)

Concurrency and throughput

[1] Measured: SCALE_STRATEGY §4.4, round 3 (24 cores, -Xmx8g, 1.16M trades a day): 133 / 141 / 122 req/s at 10 / 50 / 100 users, 0 failures, single trade p95 0.08 s at 10 users; §12: one console 69 req/s at 50 users; VAR_GUIDE §27.2: a node under 1 ms, 2.6 ms, 52 ms, 58 ms.

[2] Servers: 140 req/s on 24 cores, scaled by vCPU: × 16/24 = 93 an owner, × 8/24 = 47 a coordinator; 6 × 93 + 2 × 47 = 653 → ~650 peak; half kept free (ASSUMPTION) = 327 → ~330 sustained.

[3] Whole-bank filters: 4.68 GB of float32 rows each (1-day); 6 owners × 60 × 16/24 = 40 GB/s = 240 GB/s → 51 a second at most; 10–25 quoted (20–50% of it, the rest of the load sharing memory bandwidth: ASSUMPTION). Nodes come from totals: thousands a second.

[4] Consoles: 2 × 69 req/s (measured) = 138 → ~100–140 page loads a second.

[5] Active people: the consoles bind first: 138 req/s ÷ 0.3–0.5 req/s a busy person (a page every 10 s, 3–5 calls, ASSUMPTION) = 276–460 → ~300–500. The servers' ~330 sustained would carry more.

[6] Named users: active ÷ 10–20% active at once (ASSUMPTION): 300 ÷ 0.2 = 1,500 to 500 ÷ 0.1 = 5,000.

Hardware plan

[1] VaR servers: 4, one machine type (official EOD, intraday writer, two UI/API replicas); vCPU, RAM, heap, memory budget and NVMe as on the processes-and-memory derivations: provisioned = need ÷ 0.61, up to the next size; large = 2× positions.

[2] Consoles: 25 pages/s at peak (ASSUMPTION: 50 of 200 users, a page every 10 s, peaks 5×) ÷ 0.61 = 41 against 160 a host.

[3] Lake: 48.2 TB need ÷ 0.61 = 79 TB; × 2 with a DR copy; large × 2. Shared state: ≈ 300 GB as built (config, Sutras, each node's data folder; ASSUMPTION) ÷ 0.61 = 0.49 → 0.5 TB.

[4] Network: a first open reads 27.93 × 0.75 = 20.9 GB of Parquet at ≤ 0.4 GB/s (decoding, the guide's 10–40 s for 4 GB) = 3.2 Gbit/s ÷ 0.61 = 5.2 → ≥ 10 Gbit/s.

[5] Intraday: 2.09 GB a day (storage derivations) ÷ 120 five-minute windows = 17.5 MB a window, < 1 MB/s a server.

[6] Kafka: 230,000 records a day × 24 KB (1,518 numbers as JSON, ASSUMPTION; the guide: 4–8 KB for 509) = 5.5 GB a day × 7 days = 38.6 GB ÷ 0.61 = 63 → 64 GB a replica.

[7] Sharding beyond ≈ 5M positions: (151 − 13 fixed) GB ÷ 2.3M ÷ 0.61 = 98 GB of RAM a million positions, so 512 GB holds ≈ 5.2M; a first open of 2–5 min at 2.3M takes 4–10 min at 5M.

How the numbers were derived (continued)

AWS: the VaR tier

[1] A, ECS on EC2 capacity: 151 GB need ÷ 0.61 = 247 → m7i.16xlarge (64 vCPU, 256 GiB); EBS gp3 2 TB (1,183 ÷ 0.61 = 1,940 GB).

[2] 1-day only on one Fargate task: 75.8 GB, 32 vCPU need; ÷ 0.61 = 124 GB, 52 vCPU: the 32 vCPU · 244 GB task holds the memory (31%) but keeps the CPU headroom only if the peak is ≤ 32 × 0.61 = 19.5 vCPU. Full case: 151 ÷ 244 = 62%, over 61%.

[3] B, √10 for 10-day: width 759 (× 1.49): heap, page cache, load and OS = 94 GB; ÷ 0.61 = 155 GB ≤ 244 GB; 1.1 TB gp3.

[4] C, 8 shards: (151 − 6 heap base − 7 OS) ÷ 8 + 6 + 3 = 26 GB ÷ 0.61 = 43 → an 8 vCPU, 44 GB task; CPU 32 ÷ 8 + 1 = 5 ÷ 0.61 = 8.2 → 8; gp3 1,183 ÷ 8 ÷ 0.61 = 242 → 250 GB.

[5] Fargate and EBS limits: AWS documentation, read 2026-10-27 (32 vCPU with up to 244 GB; one EBS volume a task; gp3 up to 64 TiB, 80,000 IOPS, 2,000 MiB/s).

AWS cost

[1] Prices: AWS public on-demand list prices, example Region ca-central-1, read 2026-10-27 from the Price List files (ASSUMPTION: verify before quoting); 730 hours a month; quantities from the capacity derivations (ESTIMATED).

[2] VaR servers: 4 × m7i.16xlarge $3.5952 an hour × 730 = $10,498 (1-day: c7i.16xlarge $3.1248 → $9,124). EBS gp3 a server: 2,000 GB × $0.088 + 13,000 IOPS × $0.0055 + 1,875 MiB/s × $0.044 = $330 × 4 (1-day 800 GB: $224 × 4).

[3] Consoles: 2 × (4 vCPU × $0.04456 + 8 GB × $0.004865) × 730 = $317; lake tasks 60 h × (8 × $0.04456 + 32 × $0.004865) = $31.

[4] S3: the lake grows 6.9 TB a year (1-day 2.3); year 7: newest year Standard $0.025 + six older years Standard-IA $0.0138 per GB-month (a lifecycle rule, ASSUMPTION) = $752; DR: the same + $0.02/GB replication = $753.

[5] EFS 500 GB × $0.33 + $25 backup = $190; MSK 3 × $0.2275 × 730 + 300 GB × $0.11 = $531; ALB, NAT and endpoints, transfer, CloudWatch as listed; Secrets Manager and KMS $4, in the total.

[6] Savings Plans (Compute, no upfront, the same price files): 1 year −26.9% EC2, −20.0% Fargate; 3 years −49.5%, −45.0%; applied to EC2 and Fargate only.

[7] VaR tier: A as above $11,818; B 4 × (32 × $0.04456 + 244 × $0.004865 = $2.613 an hour) × 730 + 4 × 1.1 TB gp3 = $8,633; C 16 tasks × $0.5705 × 730 + 16 × $38.50 gp3 + the intraday task $93 = $7,373.

[8] C scale-in: second replicas 12 h × 22 days = 264 of 730 hours (ASSUMPTION) = $5,050; 3-year $2,966; second replicas on Spot at 60% off (ASSUMPTION) = $2,785. One full copy: Fargate 8 × ($416 + $38.50) = $3,640 against EC2 $2,954.

How the numbers were derived (continued)

Three machines

[1] Per member of three, replicas 2 (each holds 2/3 of the shards), two dates open; the heap scaled from one server measured at 1M (-Xmx6g, 2.2 GB with two dates): H = 128 MB + 641 B × P + 2 × 722 B × P = 2.2 GB at 1M, 4.3 GB at 2M; -Xmx 2.7 × H (the measured ratio) = 11.6 → 12g; RAM = H + 0.13 GB + two open dates of hot files (2,783 B × P × 2/3) + a load + 7 GB = 28 GB ÷ 0.61 → 48 GB; every horizon (the capacity model × P ÷ 2.3M): 55 GB at 1M, 97 GB at 2M → 192 GB.

[2] AWS list prices, Canada (Central), on demand, 730 h (ASSUMPTION: verify before quoting). 1-day VaR to 2M: 3 × m7i.4xlarge (a quarter of m7i.16xlarge) = $1,968, gp3 3 × 500 GB = $231, about 3,300–4,000 with RDS. Every horizon to 1M: m7i.8xlarge = m7i.16xlarge $3.5952 ÷ 2 × 3 × 730 = $3,937; gp3 3 × 1 TB, 6,000 IOPS, 500 MiB/s = $363; S3 2 TB $60; ALB, zones, loads, NAT and endpoints, CloudWatch, secrets as the cost lines = $476.

[3] RDS PostgreSQL Multi-AZ $600–1,300 (ASSUMPTION, not in the price constants: db.m7g.xlarge to 2xlarge, us-east-1 list + ~10%): total $5,436–6,136, about 5,400–6,100.

[4] Compute Savings Plans, no upfront, on EC2 and Fargate only: 1 year −26.9% / −20.0% = −$1,065; 3 years −49.5% / −45.0% = −$1,964.

RELATED DOCUMENTS

Where to read more

Companion decks

Drishti: Design and Evidence
what Drishti is, its design, editions, BI, AI, security and the evidence

Drishti on AWS: the lake, Delta or Iceberg
the AWS architecture in detail: setup, storage, caching, maintenance, cost and runbooks, for either format

Drishti VaR engine
this deck: method, inputs, how it scales, evidence, operating it, editions, regulatory reporting, roadmap

Document	What it holds
`docs/guides/VAR_GUIDE.md`	the complete guide: method and contract
`docs/tutorials/02-var-from-the-bank-to-a-trade.md`	VaR from the bank to a trade, hands-on
`docs/guides/TABLES_GUIDE.md`	tables and models: what BI reads
`PIVOT_GUIDE.md section 29`	the MR cube on the pivot engine
`docs/guides/FRTB_SA_GUIDE.md`	FRTB-SA; SIMM_GUIDE.md beside it
`DATA_ARCHITECTURE.md section 13`	VaR sharded across servers
`docs/admin/BENCHMARK.md`	every measured figure, with its machine
`docs/guides/DATA_LAKE_GUIDE.md`	lakes and loading, with real examples
`docs/guides/FLIGHT_GUIDE.md`	data out: Arrow Flight and Flight SQL
`docs/guides/REGULATORY_VAR_REPORTING.md`	the reports that need VaR or ES
`ADMIN_GUIDE.md section 12`	editions and lenses
`docs/admin/UNDER_CONSTRUCTION.md`	what is not built yet, and where
`docs/architecture/IMPLEMENTATION_PLAN.md`	what is built, and what comes next

Drishti दृष्टि

Thank you

See what your data hasn't told you yet.

ANY DATA. ANY DOMAIN. ONE GRAMMAR.

One engine, every input: PVs or P&L, by trade or by position, any currency, files or Kafka.

Ashutosh Sinha • November 2026 • VaR engine